

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
UTEC

Curso: Base de Datos I (CS2041), Laboratorio 14

Ciclo 2026-2

Proyecto final

Agricultura de Precisión y Sensores IoT Agropecuarios

Integrantes

Lescano Garamendi, Juan Carlos Sebastián : 202510447

Rojas Meza, Kiara Luz : 202510620

Profesor

Ojeda Rios, Brenner Humberto

Lima, Perú

Índice

1. Requisitos	4
1.1. Introducción	4
1.2. Descripción general del problema y de la organización	4
1.3. Necesidad y usos de la base de datos	4
1.4. ¿Cómo se resuelve el problema hoy?	5
1.4.1. ¿Cómo se almacenan y procesan los datos hoy?	5
1.4.2. Flujo de datos	5
1.5. Descripción detallada del sistema	6
1.5.1. Objetos de información actuales	6
1.5.2. Características y funcionalidades esperadas	6
1.5.3. Tipos de usuarios existentes y necesarios	7
1.5.4. Tipos de consulta y actualizaciones	7
1.5.5. Tamaño estimado de la base de datos	7
1.6. Objetivos del proyecto	8
1.7. Referencias del proyecto	8
1.8. Eventualidades	9
1.8.1. Problemas que pudieran encontrarse en el proyecto	9
1.8.2. Límites y alcances del proyecto	9
2. Modelo Entidad-Relación	9
2.1. Reglas semánticas	9
2.2. Modelo Entidad-Relación	11
2.3. Especificaciones y consideraciones sobre el modelo	11
3. Modelo Relacional	12
3.1. Modelo Relacional	12
3.2. Especificaciones de transformación	13
3.2.1. Entidades	13
3.2.2. Entidades débiles	14
3.2.3. Entidades superclase/subclases	14
3.2.4. Relaciones binarias	14
3.2.5. Relaciones ternarias	15
3.2.6. Normalización	16
3.3. Diccionario de datos	16

4. Implementación de la base de datos	24
4.1. Creación de Tablas en PostgreSQL	24
4.2. Carga de datos	30
4.3. Simulación de Datos Faltantes	32
4.4. Views	33
4.5. Procedimientos Almacenados	35
4.6. Triggers	37
4.6.1. Pruebas de funcionamiento	40
5. Optimización y Experimentación	42
5.1. Consultas SQL para el experimento	42
5.1.1. Descripción de las 5 consultas intermedias seleccionadas (no triviales)	42
5.1.2. Implementación de consultas en SQL	42
5.2. Metodología del experimento	44
5.3. Optimización de consultas	45
5.3.1. Plan de índices para Consulta 1	45
5.3.2. Plan de índices para Consulta 2	45
5.3.3. Plan de índices para Consulta 3	46
5.3.4. Plan de índices para Consulta 4	46
5.3.5. Plan de índices para Consulta 5	46
5.4. Plataforma de Pruebas	47
5.5. Medición de tiempos	47
5.5.1. Sin índices	47
5.5.2. Con índices	48
5.5.3. Comparación entre tipos de índice	49
5.5.4. Costo de los índices en las operaciones de escritura	50
5.6. Resultados	52
5.6.1. Consulta 1	52
5.6.2. Consulta 2	54
5.6.3. Consulta 3	56
5.6.4. Consulta 4	58
5.6.5. Consulta 5	60
5.7. Análisis y Discusión	62
5.8. Complejidad operacional por encima del millón de registros	64
6. Conclusiones	66

7. Anexos	66
7.1. Video de demostración	66
7.2. Respaldos de la base de datos	67
7.3. Scripts del proyecto	67
7.4. Generador de datos	67

1. Requisitos

1.1. Introducción

La agricultura de precisión parte de una idea sencilla: un campo no es homogéneo. Dos sectores de la misma parcela pueden tener suelos, pendientes y capacidades de retención de agua distintas, así que regarlos y fertilizarlos por igual desperdicia agua en uno y estresa al cultivo en el otro. Para manejar cada sector por separado hace falta medir, y hoy esa medición la hacen sensores de suelo conectados por radio que reportan humedad, temperatura, conductividad eléctrica y pH varias veces por hora.

En la costa peruana el agua es el recurso que limita la producción. Los fundos agroexportadores de Ica, La Libertad o Piura riegan con agua de pozo o de canal, pagan por ella y deben justificar su uso ante la Autoridad Nacional del Agua. En ese contexto, saber cuántos litros cuesta producir un kilogramo de espárrago o de arándano deja de ser un dato académico y se vuelve un indicador de gestión.

Este proyecto diseña e implementa la base de datos de una empresa agroexportadora que ya instaló sensores y controladores de riego en sus fundos, pero que guarda la información en lugares desconectados entre sí. Construimos el modelo conceptual, el modelo relacional y el modelo físico en PostgreSQL 16, poblamos la base con datos sintéticos en cuatro volúmenes (mil, diez mil, cien mil y un millón de registros) y medimos el efecto de distintos índices sobre cinco consultas de nivel intermedio.

1.2. Descripción general del problema y de la organización

El caso de estudio es Agroandina del Pacífico S.A.C.¹, una empresa con 12 fundos repartidos en 12 departamentos del país. En la costa produce espárrago, arándano, palta Hass, uva de mesa, mango, alcachofa y pimiento piquillo; en Arequipa, quinua y papa; y en la selva alta, café y cacao. Maneja 600 parcelas que suman cerca de 10 553 hectáreas, divididas en 2 400 zonas de manejo.

La empresa tiene instalados 2 200 sensores de suelo y 600 controladores de válvulas. Los sensores provienen de varios fabricantes y cada uno envía sus datos a una plataforma web propia. Los controladores registran los riegos en la consola del proveedor del sistema. Las cosechas se anotan en hojas de cálculo por fundo y las aplicaciones de fertilizantes y plaguicidas se escriben en cuadernos de campo que un asistente transcribe cada semana.

El problema concreto es que ninguna de estas fuentes comparte un identificador común. Un sensor se reconoce por su número de serie, una parcela por un código interno que cambia de un fundo a otro y una cosecha por la fecha y el nombre del supervisor. Cruzar los riegos de una parcela con lo que esa parcela produjo toma días de trabajo manual y el resultado suele tener errores.

1.3. Necesidad y usos de la base de datos

La gerencia agrícola necesita un repositorio único que permita responder preguntas que hoy no puede contestar a tiempo:

- Qué parcelas están por debajo de la humedad mínima que su cultivo tolera y desde cuándo.
- Cuánta agua consume cada campaña por kilogramo cosechado, dato que además pide la certificación GLOBALG.A.P. y que sirve para renovar licencias de uso de agua.

¹Nombre ficticio. El caso se construyó con información pública del sector agroexportador peruano y con fichas técnicas de sensores comerciales.

- Qué sensores reportan valores anómalos con demasiada frecuencia, lo que suele indicar un equipo descalibrado antes que un problema del suelo.
- Qué parcelas rinden más dentro de cada fundo y con qué cultivo.
- Si el personal de riego reacciona a las alertas de humedad crítica dentro de un plazo razonable.

También se requiere trazabilidad: para exportar hay que demostrar qué producto se aplicó en qué zona, quién lo aplicó y que se respetó el periodo de carencia antes de cosechar.

1.4. ¿Cómo se resuelve el problema hoy?

1.4.1. ¿Cómo se almacenan y procesan los datos hoy?

Hoy cada tipo de dato vive en un sistema distinto. Las lecturas de los sensores se descargan como archivos CSV desde los portales de los fabricantes, con columnas y formatos de fecha diferentes según la marca. Los riegos se exportan desde la consola de los controladores. Las cosechas están en un libro de Excel por fundo, con una hoja por campaña. Los cuadernos de aplicación se transcriben a otro Excel compartido y los avisos de humedad baja llegan por un grupo de WhatsApp en el que participan los operadores de riego.

Un analista une estos archivos a mano una vez por semana. Como no hay claves comunes, el cruce se hace por nombres y fechas aproximadas, lo que produce duplicados cuando un mismo sensor se reinstala en otra zona y omisiones cuando un supervisor escribe mal el código de la parcela.

1.4.2. Flujo de datos

La Figura 1 resume el flujo actual. La información pasa por al menos tres transcripciones antes de llegar a quien decide, y el ciclo completo tarda una semana.

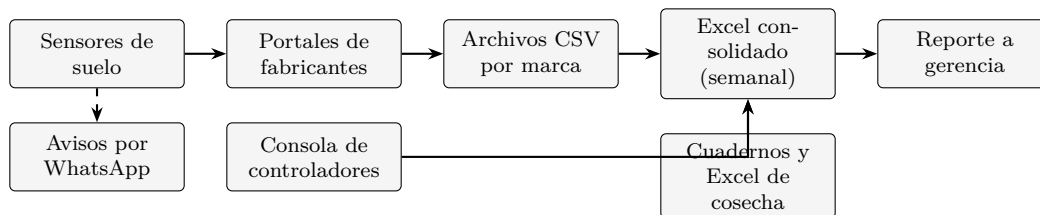


Figura 1: Flujo de datos actual.

Con la base de datos propuesta el flujo cambia (Figura 2). Un servicio de ingesta recibe los mensajes del gateway LoRaWAN y llama al procedimiento `sp_registrar_lectura`. Al insertarse la lectura, un trigger evalúa si la humedad está por debajo del mínimo del cultivo vigente y, si es así, crea la alerta. Los controladores registran cada riego en `Evento_Riego`, los supervisores registran cosechas y aplicaciones desde el campo, y la gerencia consulta vistas que ya traen los indicadores calculados.

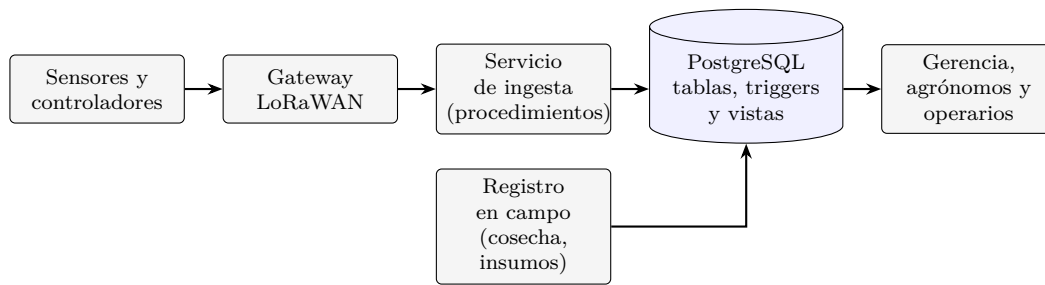


Figura 2: Flujo de datos con la base de datos propuesta.

1.5. Descripción detallada del sistema

1.5.1. Objetos de información actuales

Los objetos de información que la empresa ya maneja, aunque dispersos, son los siguientes:

- **Fundos y parcelas.** Nombre, ubicación política, coordenadas, área, tipo de suelo y pendiente. Cambian muy poco.
- **Zonas de manejo.** Sectores de riego dentro de cada parcela. Se redefinen cuando se rediseña el sistema de riego.
- **Catálogo de cultivos.** Rangos óptimos de humedad, pH y conductividad, y duración del ciclo. Se toman de fichas técnicas y de la guía FAO-56 de evapotranspiración.
- **Campañas.** Qué cultivo se sembró en qué parcela y entre qué fechas.
- **Dispositivos.** Número de serie, modelo, fabricante, fecha de instalación y zona. Los sensores añaden tipo y profundidad; los controladores, el sistema de riego que accionan y el caudal de la válvula.
- **Lecturas de suelo.** Humedad volumétrica, temperatura, pH, conductividad y batería, con marca de tiempo. Es el objeto más voluminoso: un sensor que reporta cada 15 minutos genera 96 lecturas diarias.
- **Riegos, alertas, cosechas, aplicaciones de insumos y mantenimientos,** cada uno con su fecha, su responsable y sus cantidades.

1.5.2. Características y funcionalidades esperadas

Se espera un repositorio central en el que cada lectura, riego y cosecha quede asociado sin ambigüedad a una zona, una parcela y una campaña. Las reglas del negocio deben vivir en la base de datos y no en la buena voluntad de quien digita: rangos válidos, fechas coherentes, campañas que no se superpongan, alertas que se generen solas y un rendimiento por hectárea que se actualice con cada cosecha.

En cuanto a desempeño, las consultas operativas (alertas pendientes, estado hídrico de una zona) deben responder en menos de un segundo, y los reportes analíticos sobre toda la serie, en pocos segundos. La carga masiva de datos históricos debe poder hacerse en minutos.

1.5.3. Tipos de usuarios existentes y necesarios

La Tabla 1 resume los perfiles y lo que cada uno puede hacer. El script opcional `05_rolles.sql` crea estos roles en PostgreSQL con los permisos indicados.

Perfil	Acceso
Gerencia agrícola	Solo lectura sobre las vistas de producción, eficiencia hídrica y alertas. No ve datos personales de los operarios.
Ingeniero agrónomo	Lectura de todo el esquema. Crea y cierra campañas, mantiene el catálogo de cultivos e insumos.
Operador de riego	Registra eventos de riego y marca alertas como atendidas. Consulta lecturas y el estado hídrico.
Técnico de campo	Registra aplicaciones de insumos, cosechas y mantenimientos de equipos.
Plataforma IoT	Cuenta de servicio. Solo puede ejecutar <code>sp_registrar_lectura</code> ; no tiene acceso directo a las tablas.
Administrador de BD	Control total: estructura, respaldos, índices y usuarios.

Tabla 1: Perfiles de usuario.

1.5.4. Tipos de consulta y actualizaciones

Las consultas más frecuentes son de tres tipos. Las operativas buscan pocas filas recientes: la última lectura de una zona, las alertas sin atender o los riegos de hoy. Las de seguimiento agregan un periodo corto, por ejemplo la humedad promedio del último mes por parcela. Las analíticas recorren toda la historia: eficiencia hídrica por campaña, ranking de rendimiento o proporción de lecturas anómalas por sensor. Las cinco consultas del experimento (Sección 5.1) cubren estos tres tipos.

Las actualizaciones tienen ritmos muy distintos, como muestra la Tabla 2.

Dato	Frecuencia	Forma de carga
Lecturas de suelo	Cada 5 a 60 min por sensor	Insertión continua vía procedimiento; nunca se modifican.
Alertas	Según las lecturas	Las crea un trigger; el operador solo actualiza la atención.
Eventos de riego	Varias veces al día	Insertión automática desde el controlador o manual.
Cosechas	Diaria en temporada	Insertión por el supervisor; recalcula el rendimiento.
Aplicaciones de insumos	Semanal	Insertión por el técnico de campo.
Campañas	Por temporada	Alta y cambio de estado por el agrónomo (auditado).
Fundos, parcelas, cultivos	Rara vez	Carga inicial y ajustes puntuales.

Tabla 2: Ritmo de carga y modificación de los datos.

1.5.5. Tamaño estimado de la base de datos

El escenario de un millón de registros ocupa 170.2 MB en disco con todos sus índices. La tabla `Lectura_Suelo` concentra el 77.0% del espacio ocupado por las tablas (Tabla 3). Cada lectura ocupa en promedio 171 bytes entre datos e índices.

Tabla	Filas	Datos (MB)	Índices (MB)	Total (MB)
Lectura_Suelo	765 861	49.0	75.7	124.8
Evento_Riego	120 000	9.7	9.9	19.6
Alerta	30 000	2.8	3.1	5.9
Cosecha	25 000	1.8	2.6	4.4
Aplicacion_Insumo	40 000	2.7	1.6	4.3
Mantenimiento	8 000	0.7	0.3	1.0
Dispositivo	2 800	0.2	0.3	0.5
Campania	1 500	0.1	0.2	0.3
Otras 10 tablas	6 839	0.5	0.4	1.2
Total				161.9

Tabla 3: Espacio ocupado por tabla en el escenario de un millón.

Ese millón de registros es, sin embargo, una muestra. Si los 2 200 sensores reportaran realmente cada 15 minutos, la empresa generaría unos 77.1 millones de lecturas al año, es decir, alrededor de 13.2 GB anuales solo en esa tabla con el esquema de índices actual. Ese orden de magnitud condiciona las decisiones de la Sección 5.8.

1.6. Objetivos del proyecto

El objetivo general es diseñar e implementar en PostgreSQL una base de datos que integre la información de sensores, riego, campañas y cosechas de la empresa, y evaluar experimentalmente cómo responde a volúmenes crecientes de datos.

Los objetivos específicos son:

1. Modelar el dominio con una jerarquía Is-A, entidades débiles y una relación ternaria, y transformarlo a un esquema relacional en tercera forma normal.
2. Trasladar las reglas del negocio a restricciones, procedimientos y triggers para que la base de datos las haga cumplir por sí misma.
3. Generar datos sintéticos coherentes con la realidad agronómica peruana en cuatro volúmenes.
4. Medir cinco consultas intermedias con y sin índices, comparar tipos de índice y cuantificar cuánto encarecen los índices las operaciones de escritura.
5. Proponer una estrategia de crecimiento para volúmenes superiores al millón de registros.

1.7. Referencias del proyecto

Los rangos óptimos por cultivo y la relación entre humedad del suelo y estrés hídrico se tomaron de la guía FAO-56 y de fichas técnicas públicas. Los modelos de sensores y controladores corresponden a equipos comerciales reales, pero sus datos son simulados. Para el diseño y la implementación seguimos la bibliografía del curso y la documentación oficial de PostgreSQL.

- Allen, R., Pereira, L., Raes, D. y Smith, M. (1998). *Crop evapotranspiration: guidelines for computing crop water requirements*. FAO Irrigation and Drainage Paper 56.

- Garcia-Molina, H., Ullman, J. y Widom, J. (2009). *Database Systems: The Complete Book* (2.^a ed.). Pearson.
- Elmasri, R. y Navathe, S. (2016). *Fundamentals of Database Systems* (7.^a ed.). Pearson.
- PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>
- Autoridad Nacional del Agua. <https://www.gob.pe/ana>
- Ministerio de Desarrollo Agrario y Riego. <https://www.gob.pe/midagri>
- GLOBALG.A.P. Integrated Farm Assurance Standard. <https://www.globalgap.org>
- METER Group. Ficha técnica del sensor TEROS 12. <https://www.metergroup.com>

1.8. Eventualidades

1.8.1. Problemas que pudieran encontrarse en el proyecto

El primer problema es el acceso a datos reales. Las lecturas de sensores de una empresa privada son información comercial y no están publicadas, por eso trabajamos con datos sintéticos contruados con parámetros reales. El riesgo es que la simulación no reproduzca fenómenos que sí ocurren en campo, como la deriva gradual de un sensor o la estacionalidad de la humedad entre invierno y verano.

Otros problemas previsibles son técnicos. Los sensores pueden enviar dos lecturas con la misma marca de tiempo si su reloj se desfasa, lo que chocaría con la clave primaria; por eso el procedimiento de registro ignora duplicados con `ON CONFLICT DO NOTHING`. También hay pérdidas de transmisión que dejan huecos en las series, y cambios de formato cuando un proveedor actualiza su plataforma. Por último, todas las mediciones se hicieron en una sola máquina, así que los tiempos absolutos dependen de ese equipo; lo que se puede comparar con confianza son las proporciones entre escenarios y entre configuraciones.

1.8.2. Límites y alcances del proyecto

El proyecto cubre el modelo conceptual, el relacional y el físico, la carga de datos en cuatro volúmenes, las vistas, procedimientos y triggers, y el experimento de optimización con sus variantes. No incluye el servicio de ingesta en tiempo real ni una aplicación web, tampoco datos meteorológicos o imágenes satelitales, ni algoritmos de recomendación de riego. Estos elementos se mencionan como trabajo futuro porque el esquema está preparado para recibirlos.

2. Modelo Entidad-Relación

2.1. Reglas semánticas

Cada regla indica entre paréntesis el mecanismo que la hace cumplir en la implementación. Las multiplicidades se expresan como mínimo y máximo de participación.

- R1.** Un fundo tiene una o más parcelas y cada parcela pertenece a exactamente un fundo. El código de la parcela se repite entre fundos, pero no dentro del mismo fundo (FK, NOT NULL, UNIQUE).

- R2.** Cada parcela se divide en una o más zonas de manejo. Una zona no existe fuera de su parcela y se identifica por su número dentro de ella: es una entidad débil (PK compuesta, ON DELETE CASCADE).
- R3.** Un cultivo puede sembrarse en muchas campañas y una parcela puede tener muchas campañas a lo largo del tiempo; cada campaña corresponde a exactamente una parcela y a exactamente un cultivo (FK, NOT NULL).
- R4.** En una misma parcela, dos campañas no pueden superponerse en el tiempo. En consecuencia, en una fecha dada hay a lo sumo una campaña en curso por parcela (trigger `trg_campania_sin_solape`).
- R5.** La fecha de fin estimada de una campaña es posterior a su fecha de siembra (CHECK).
- R6.** Una parcela puede tener cero o más sistemas de riego; las parcelas de secano de la selva no tienen ninguno. Cada sistema abastece a una sola parcela (FK).
- R7.** Todo dispositivo está instalado en exactamente una zona de manejo; una zona puede tener cero o más dispositivos (FK compuesta).
- R8.** Todo dispositivo es un sensor o un controlador, y nunca ambos a la vez. La especialización es total y disjunta (trigger `trg_fn_subclase_exclusiva` y procedimiento `sp_registrar_dispositivo`).
- R9.** Cada controlador acciona exactamente un sistema de riego, que debe ser el de la parcela donde está instalado (FK y verificación de carga).
- R10.** Un sensor emite cero o más lecturas y cada lectura pertenece a un solo sensor. La lectura se identifica por el minuto en que se tomó dentro de ese sensor: es una entidad débil y un sensor no puede reportar dos veces en el mismo minuto (PK compuesta).
- R11.** Las variables presentes en una lectura dependen del tipo de sensor. Un sensor de pH no mide humedad y uno de temperatura solo mide temperatura; en esos casos la columna queda nula por diseño.
- R12.** Toda alerta se origina en exactamente una lectura, y una lectura origina a lo sumo una alerta de cada tipo (FK compuesta hacia la lectura, UNIQUE).
- R13.** Si la humedad de una lectura cae por debajo del mínimo del cultivo en curso, o la batería baja de 15 %, se genera la alerta correspondiente sin intervención humana (trigger `trg_alerta_lectura`).
- R14.** Una alerta atendida debe tener fecha de atención, y una no atendida no puede tenerla (CHECK).
- R15.** Un evento de riego usa exactamente un sistema sobre exactamente una zona, y esa zona debe pertenecer a la parcela que abastece el sistema. Puede tener un operario responsable o ninguno si fue automático (FK y trigger `trg_evento_riego`).
- R16.** En una zona no pueden empezar dos riegos en el mismo minuto (UNIQUE).
- R17.** Si el controlador no informa el volumen aplicado, este se calcula como caudal nominal por duración (trigger `trg_evento_riego`).
- R18.** Una campaña tiene cero o más registros de cosecha y cada cosecha pertenece a una sola campaña. Las campañas perdidas no registran cosecha (FK y verificación de carga).
- R19.** El rendimiento de una campaña es un atributo derivado: kilogramos cosechados entre el área de la parcela (trigger `trg_rendimiento`).
- R20.** En una aplicación de insumo intervienen exactamente una zona, un insumo y un operario en un instante dado. La misma terna puede repetirse en otro momento, pero no en el mismo (relación ternaria, PK de cinco columnas).

- R21.** Un operario puede dar mantenimiento a muchos dispositivos y un dispositivo puede ser atendido por muchos operarios. Un operario no registra dos mantenimientos del mismo equipo en la misma fecha (relación N:M, PK compuesta).
- R22.** Todo operario tiene un DNI de ocho dígitos que no se repite, y su teléfono, si existe, tiene nueve dígitos (UNIQUE, CHECK con expresión regular).
- R23.** Los valores medidos respetan rangos físicos: humedad entre 0 y 100 %, pH entre 0 y 14, conductividad no negativa, temperatura entre -10 y 60 °C (CHECK).
- R24.** Todo cambio de estado de una campaña queda registrado con usuario y fecha (trigger `trg_auditoria_campania`).

2.2. Modelo Entidad-Relación

La Figura 3 presenta el diagrama. Usamos notación de Chen: rectángulos para entidades, óvalos para atributos, rombos para relaciones y triángulos para la especialización. Las claves se subrayan con línea continua y las claves parciales de las entidades débiles, con línea discontinua. Siguiendo a Garcia-Molina, Ullman y Widom, una flecha que llega a una entidad indica que cada instancia del otro lado se relaciona con a lo sumo una instancia de ella. Siguiendo a Elmasri y Navathe, la línea doble indica participación total. Las entidades débiles y sus relaciones identificadoras se dibujan con borde doble y el atributo derivado, con óvalo punteado.

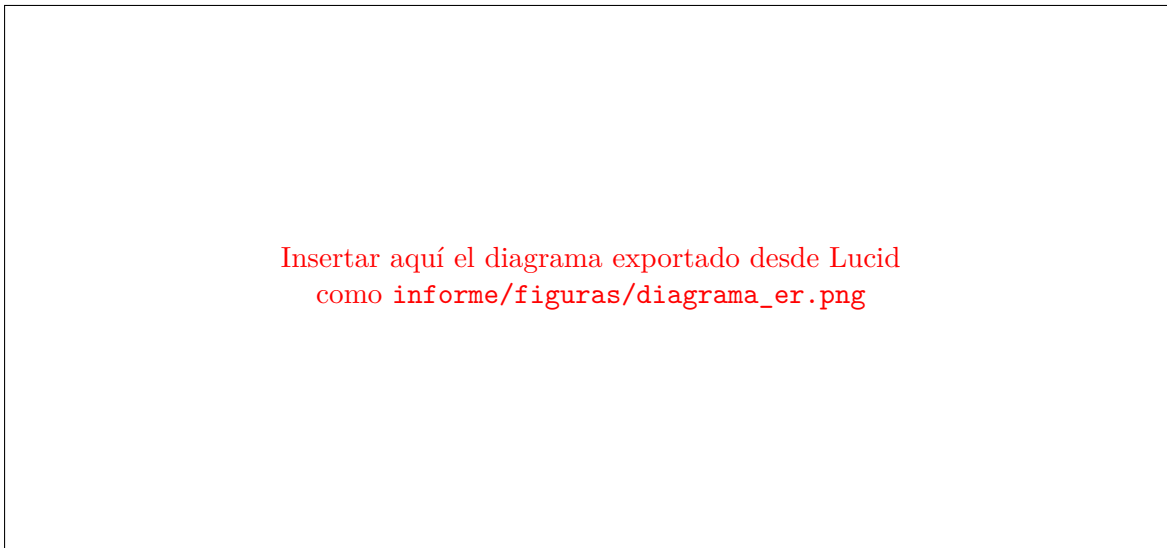


Figura 3: Modelo entidad-relación.

2.3. Especificaciones y consideraciones sobre el modelo

Campaña como entidad. Una campaña podría verse como la relación *se siembra* entre **Parcela** y **Cultivo**. La modelamos como entidad porque tiene identidad propia, porque la misma parcela repite el mismo cultivo en años distintos y porque otras entidades dependen de ella: las cosechas se registran contra una campaña concreta, no contra un par parcela-cultivo.

Dos entidades débiles. **Zona_Manejo** depende de **Parcela**: la zona 3 solo tiene sentido dentro de su parcela. **Lectura_Suelo** depende de **Sensor**: una medición aislada del equipo que la tomó no significa

nada, y el instante basta para distinguirla dentro de ese equipo. Tratar la lectura como débil evita además un identificador artificial en la tabla más grande del esquema, lo que ahorra espacio y un índice.

Especialización total y disjunta. Dispositivo generaliza a Sensor y Controlador. Es total porque la empresa no instala equipos que no sean de uno de los dos tipos, y disjunta porque un nodo que mide y a la vez acciona válvulas se registra como dos dispositivos. Cada subclase tiene atributos y relaciones propias: solo los sensores emiten lecturas y solo los controladores accionan un sistema de riego.

Relación ternaria. Aplica relaciona Zona_Manejo, Insumo y Operario. No se puede reemplazar por tres relaciones binarias sin perder información: saber qué operario aplicó qué insumo y en qué zona trabajó cada operario no permite reconstruir quién aplicó qué cosa dónde. La terna es de muchos a muchos a muchos, por eso ninguno de sus arcos lleva flecha.

Atributos compuestos y derivados. La ubicación del fundo (departamento, provincia, distrito) y los rangos óptimos del cultivo (mínimo y máximo de humedad y de pH) son atributos compuestos que se aplanan en columnas simples. El rendimiento por hectárea de la campaña es derivado; se almacena por rendimiento de consulta y lo mantiene un trigger.

Tabla fuera del modelo conceptual. Auditoria_Campania es una bitácora técnica alimentada por un trigger. No representa un objeto del negocio, por lo que no aparece en el diagrama.

Relaciones del diagrama. Además de las ya descritas, el modelo incluye: *tiene* (Fundo–Parcela), *se divide en* (Parcela–Zona, identificadora), *se siembra en* (Campaña–Parcela), *corresponde a* (Campaña–Cultivo), *labora en* (Operario–Fundo), *abastece* (Sistema–Parcela), *instalado en* (Dispositivo–Zona), *acciona* (Controlador–Sistema), *registra* (Sensor–Lectura, identificadora), *origina* (Lectura–Alerta), *atiende* (Operario–Alerta), *ejecuta* (Sistema–Evento), *riega* (Evento–Zona), *opera* (Operario–Evento), *produce* (Campaña–Cosecha), *recoge* (Operario–Cosecha) y *mantiene* (Operario–Dispositivo, N:M). En total el modelo tiene 15 entidades, contando las dos subclases, y 18 relaciones.

3. Modelo Relacional

3.1. Modelo Relacional

El esquema tiene 18 relaciones: 15 provienen de las entidades del modelo conceptual, dos implementan relaciones con atributos propios (Aplicacion_Insumo y Mantenimiento) y una es la bitácora técnica Auditoria_Campania. Las claves primarias van subrayadas y las claves foráneas en cursiva, con una flecha hacia la tabla referenciada. La lista se generó desde el catálogo de PostgreSQL, por lo que coincide exactamente con lo implementado.

- FUNDO(id_fundo, nombre, departamento, provincia, distrito, area_total_ha, latitud, longitud, fecha_registro)
- PARCELA(id_parcela, *id_fundo* → FUNDO, codigo_parcela, area_ha, tipo_suelo, pendiente_pct, fecha_habilitacion)
- ZONA_MANEJO(*id_parcela* → PARCELA, num_zona, area_ha, descripcion)
- CULTIVO(id_cultivo, nombre_comun, nombre_cientifico, ciclo_dias, humedad_optima_min, humedad_optima_max, ph_optimo_min, ph_optimo_max, ce_maxima_ds_m)
- OPERARIO(id_operario, dni, nombre, apellido, rol, telefono, *id_fundo* → FUNDO, fecha_ingreso)

- **CAMPANIA**(id_campania, id_parcela→PARCELA, id_cultivo→CULTIVO, fecha_siembra, fecha_fin_estimada, densidad_plantas_ha, estado, rendimiento_kg_ha)
- **SISTEMA_RIEGO**(id_sistema, id_parcela→PARCELA, tipo, caudal_nominal_lph, fuente_agua, fecha_instalacion, estado)
- **DISPOSITIVO**(id_dispositivo, codigo_serie, modelo, fabricante, fecha_instalacion, estado, id_parcela→ZONA_MANEJO, num_zona→ZONA_MANEJO)
- **SENSOR**(id_dispositivo→DISPOSITIVO, tipo_sensor, profundidad_cm, frecuencia_min, unidad_medida, rango_min, rango_max)
- **CONTROLADOR**(id_dispositivo→DISPOSITIVO, id_sistema→SISTEMA_RIEGO, caudal_max_lps, tipo_valvula)
- **LECTURA_SUELO**(id_dispositivo→SENSOR, fecha_hora, humedad_pct, temperatura_c, ph, conductividad_ds_m, bateria_pct)
- **EVENTO_RIEGO**(id_evento, id_sistema→SISTEMA_RIEGO, id_parcela→ZONA_MANEJO, num_zona→ZONA_MANEJO, id_operario→OPERARIO, fecha_hora_inicio, duracion_min, volumen_litros, modo)
- **ALERTA**(id_alerta, id_dispositivo→LECTURA_SUELO, fecha_hora→LECTURA_SUELO, tipo, severidad, valor_registrado, atendida, fecha_atencion, id_operario→OPERARIO)
- **COSECHA**(id_cosecha, id_campania→CAMPANIA, id_operario→OPERARIO, fecha, cantidad_kg, categoria, humedad_producto_pct, precio_kg_soles)
- **INSUMO**(id_insumo, nombre, tipo, unidad, costo_unitario, periodo_carencia_dias)
- **APLICACION_INSUMO**(id_parcela→ZONA_MANEJO, num_zona→ZONA_MANEJO, id_insumo→INSUMO, id_operario→OPERARIO, fecha_hora, dosis, metodo)
- **MANTENIMIENTO**(id_dispositivo→DISPOSITIVO, id_operario→OPERARIO, fecha, tipo, observacion, costo)
- **AUDITORIA_CAMPANIA**(id_auditoria, id_campania, estado_anterior, estado_nuevo, usuario_bd, fecha_cambio)

3.2. Especificaciones de transformación

3.2.1. Entidades

Cada entidad fuerte se convirtió en una tabla con sus atributos simples: **Fundo**, **Parcela**, **Cultivo**, **Operario**, **Campania**, **Sistema_Riego**, **Dispositivo**, **Evento_Riego**, **Alerta**, **Cosecha** e **Insumo**. Los atributos compuestos se aplanaron: la ubicación del fundo quedó en tres columnas y cada rango óptimo del cultivo en un par mínimo-máximo con un **CHECK** que exige mínimo menor que máximo.

Como claves usamos identificadores numéricos generados por secuencia (**SERIAL**) y no las claves naturales, porque los códigos de la empresa cambian o se repiten: un código de parcela se reutiliza entre fundos y un equipo reinstalado conserva su número de serie. Las claves naturales no se descartaron; quedaron como restricciones **UNIQUE** (**codigo_serie**, **dni**, **nombre_comun** y el par **id_fundo**, **codigo_parcela**), de modo que la base sigue impidiendo duplicados reales.

3.2.2. Entidades débiles

Una entidad débil se transforma en una tabla cuya clave primaria une la clave de la entidad propietaria con la clave parcial. La relación identificadora no genera tabla: queda absorbida en la clave foránea hacia la propietaria, declarada con **ON DELETE CASCADE** porque la entidad débil no puede existir sin su dueña.

- **Zona_Manejo**(id_parcela, num_zona, ...): id_parcela es a la vez parte de la clave y clave foránea hacia **Parcela**.
- **Lectura_Suelo**(id_dispositivo, fecha_hora, ...): la clave foránea apunta a **Sensor** y no a **Dispositivo**. Así la base rechaza por sí sola una lectura atribuida a un controlador.

Las tablas que dependen de una entidad débil heredan su clave compuesta. **Dispositivo**, **Evento_Riego** y **Aplicacion_Insumo** referencian la zona con el par (id_parcela, num_zona), y **Alerta** referencia la lectura con (id_dispositivo, fecha_hora).

3.2.3. Entidades superclase/subclases

Para la especialización de **Dispositivo** evaluamos las tres estrategias habituales (Tabla 4).

Estrategia	A favor	En contra
Una sola tabla con columna discriminadora	Ninguna unión para leer un equipo.	Columnas nulas para los atributos de la otra subclase; no se puede exigir NOT NULL al tipo de sensor ni al sistema del controlador; la lectura no podría apuntar solo a sensores.
Tabla para la superclase y una por subclase (elegida)	Cada subclase declara sus propias restricciones y relaciones; las claves foráneas son precisas.	Ver un equipo completo exige una unión por clave primaria, que es barata.
Solo tablas para las subclases	Sin uniones dentro de cada tipo.	Los atributos comunes se repiten; Mantenimiento necesitaría dos claves foráneas; no se puede garantizar un número de serie único entre ambas tablas.

Tabla 4: Estrategias para la jerarquía Is-A.

En la estrategia elegida, la clave primaria de **Sensor** y de **Controlador** es también clave foránea hacia **Dispositivo**. El modelo relacional no ofrece una forma declarativa de exigir cobertura total y disjunta, así que la completamos con dos mecanismos. La disjunción la garantiza el trigger **trg_fn_subclase_exclusiva**, que rechaza registrar como controlador un equipo que ya es sensor y viceversa. La cobertura total la garantiza el procedimiento **sp_registrar_dispositivo**, que inserta la superclase y la subclase en la misma transacción: si la segunda inserción falla, la primera se revierte y no queda un dispositivo sin tipo (prueba P2 de la Sección 4.6.1).

3.2.4. Relaciones binarias

Las relaciones uno a muchos se implementaron con una clave foránea en el lado “muchos”. Si la participación de ese lado es total, la columna es NOT NULL; si es parcial, admite nulos. La política de

borrado sigue el significado de cada vínculo: **CASCADE** en las dependencias de existencia (entidades débiles, subclases y registros que no tienen sentido sin su padre), **RESTRICT** donde un borrado dejaría sin historia datos del negocio (no se puede borrar un cultivo con campañas) y **SET NULL** en las referencias opcionales a operarios, para conservar el riego o la cosecha aunque el trabajador se retire.

Relación	Tipo	Implementación
tiene (Fundo–Parcela)	1:N	Parcela.id_fundo NOT NULL
labora en (Fundo–Operario)	1:N	Operario.id_fundo
se siembra en (Parcela–Campaña)	1:N	Campania.id_parcela NOT NULL
corresponde a (Cultivo–Campaña)	1:N	Campania.id_cultivo NOT NULL
abastece (Parcela–Sistema)	1:N	Sistema_Riego.id_parcela NOT NULL
instalado en (Zona–Dispositivo)	1:N	Dispositivo(id_parcela, num_zona) NOT NULL
acciona (Sistema–Controlador)	1:N	Controlador.id_sistema NOT NULL
origina (Lectura–Alerta)	1:N	Alerta(id_dispositivo, fecha_hora) NOT NULL
atiende (Operario–Alerta)	1:N	Alerta.id_operario, admite nulos
ejecuta (Sistema–Evento)	1:N	Evento_Riego.id_sistema NOT NULL
riega (Zona–Evento)	1:N	Evento_Riego(id_parcela, num_zona) NOT NULL
opera (Operario–Evento)	1:N	Evento_Riego.id_operario, admite nulos
produce (Campaña–Cosecha)	1:N	Cosecha.id_campania NOT NULL
recoge (Operario–Cosecha)	1:N	Cosecha.id_operario, admite nulos
mantiene (Operario–Dispositivo)	N:M	Tabla Mantenimiento con PK (id_dispositivo, id_operario, fecha)
se divide en, registra	1:N ident.	Absorbidas en la clave de la entidad débil

Tabla 5: Transformación de las relaciones binarias.

En **Mantenimiento** la fecha forma parte de la clave porque el mismo técnico vuelve a atender el mismo equipo en visitas distintas; sin ella, la tabla solo admitiría una intervención por pareja en toda la vida del equipo.

3.2.5. Relaciones ternarias

La relación *aplica* se transformó en la tabla **Aplicacion_Insumo**, con una clave foránea hacia cada participante: la zona (compuesta), el insumo y el operario. Por ser de muchos a muchos a muchos, la clave primaria reúne las tres claves foráneas. Le añadimos **fecha_hora** porque la misma terna se repite en el tiempo: el mismo operario aplica el mismo fertilizante en la misma zona cada semana. Los atributos de la relación (**dosis** y **metodo**) dependen de la aplicación completa y no de uno solo de los participantes, lo que confirma que la tabla está en segunda forma normal.

3.2.6. Normalización

Todas las tablas cumplen la primera forma normal (valores atómicos, sin grupos repetidos) y la segunda (en las tablas de clave compuesta, cada atributo no clave depende de la clave completa). Revisamos la tercera forma normal tabla por tabla y encontramos tres casos que merecen explicación:

- **Evento_Riego está en 3FN pero no en FNBC.** Como un sistema abastece a una sola parcela, se cumple la dependencia $id_sistema \rightarrow id_parcela$, y $id_sistema$ no es superclave. Sin embargo, $id_parcela$ es atributo primo, porque forma parte de la clave candidata ($id_parcela, num_zona, fecha_hora_inicio$); por eso la tabla sí cumple 3FN. Descomponerla para alcanzar FNBC impediría declarar en la misma tabla la clave foránea hacia la zona y la unicidad por zona e instante, es decir, se perdería la preservación de dependencias. Preferimos 3FN y hacemos cumplir la dependencia con el trigger `trg_evento_riego`.
- **Ubicación del fundo.** En el Perú el nombre de un distrito no identifica su provincia (en nuestros datos hay un distrito Bellavista en Sullana y otro en Jaén), así que no existe la dependencia $distrito \rightarrow provincia$. Sí podría argumentarse $provincia \rightarrow departamento$. Con doce fundos la redundancia es despreciable; si la tabla creciera, la solución sería reemplazar las tres columnas por el código UBIGEO del INEI con su propia tabla.
- **Rendimiento de la campaña.** `rendimiento_kg_ha` se deriva de las cosechas y del área, por lo que es una redundancia controlada. La mantenemos porque se consulta mucho más de lo que cambia y la actualiza un trigger en cada inserción, modificación o borrado de cosechas.

3.3. Diccionario de datos

El diccionario se generó automáticamente desde el catálogo de PostgreSQL y desde los comentarios del script `01b_comentarios.sql`. Para cada tabla se indican el tipo de dato, si admite nulos, su papel en las claves y su significado. Debajo de cada tabla se listan sus restricciones `CHECK`.

Fundo

Predio agrícola de la empresa, ubicado en un distrito del Perú.

Columna	Tipo	Nulo	Clave	Descripción
<code>id_fundo</code>	SERIAL	No	PK	Identificador del fundo.
<code>nombre</code>	VARCHAR(60)	No	UQ	Nombre comercial del fundo.
<code>departamento</code>	VARCHAR(40)	No	UQ	Departamento donde se ubica.
<code>provincia</code>	VARCHAR(40)	No		Provincia donde se ubica.
<code>distrito</code>	VARCHAR(40)	Sí		Distrito donde se ubica.
<code>area_total_ha</code>	NUMERIC(8,2)	No		Superficie total del predio en hectáreas.
<code>latitud</code>	NUMERIC(9,6)	Sí		Latitud WGS84 del punto de referencia.
<code>longitud</code>	NUMERIC(9,6)	Sí		Longitud WGS84 del punto de referencia.
<code>fecha_registro</code>	DATE	No		Fecha en que el fundo se registró en el sistema. Por defecto: <code>CURRENT_DATE</code> .

Restricciones CHECK: `ck_fundo_area: ((area_total_ha > (0))); ck_fundo_lat: (((latitud >= ('-19')) AND (latitud <= (1))));` `ck_fundo_lon: (((longitud >= ('-82')) AND (longitud <= ('-68'))))`

Parcela

Unidad productiva dentro de un fundo; es la base de cada campaña.

Columna	Tipo	Nulo	Clave	Descripción
id_parcela	SERIAL	No	PK	Identificador de la parcela.
id_fundo	INTEGER	No	FK → Fundo, UQ	Fundo al que pertenece.
codigo_parcela	VARCHAR(15)	No	UQ	Código interno, único dentro del fundo (P-001, P-002...).
area_ha	NUMERIC(7,2)	No		Área cultivable en hectáreas.
tipo_suelo	VARCHAR(20)	No		Clase textural del suelo.
pendiente_pct	NUMERIC(4,1)	Sí		Pendiente media del terreno en porcentaje.
fecha_habilitacion	DATE	Sí		Fecha en que la parcela entró en producción.

Restricciones CHECK: ck_parcela_area: ((area_ha > (0))); ck_parcela_pend: (((pendiente_pct >= (0)) AND (pendiente_pct <= (100)))); ck_parcela_suelo: (((tipo_suelo) = ANY ((ARRAY['Arenoso', 'Franco', 'Franco arenoso', 'Franco arcilloso', 'Arcilloso']))))

Zona_Manejo

Subdivisión de una parcela con manejo homogéneo de riego y nutrición (entidad débil).

Columna	Tipo	Nulo	Clave	Descripción
id_parcela	INTEGER	No	PK, FK → Parcela	Parcela propietaria (parte de la clave).
num_zona	SMALLINT	No	PK	Número correlativo de la zona dentro de la parcela (clave parcial).
area_ha	NUMERIC(6,2)	No		Área de la zona en hectáreas.
descripcion	VARCHAR(80)	Sí		Descripción libre de la zona.

Restricciones CHECK: ck_zona_area: ((area_ha > (0))); ck_zona_num: ((num_zona > 0))

Cultivo

Catálogo de especies con sus rangos agronómicos óptimos.

Columna	Tipo	Nulo	Clave	Descripción
id_cultivo	SERIAL	No	PK	Identificador del cultivo.
nombre_comun	VARCHAR(40)	No	UQ	Nombre comercial de la especie o variedad.
nombre_cientifico	VARCHAR(60)	Sí		Nombre científico.
ciclo_dias	INTEGER	No		Duración típica de la campaña en días.
humedad_optima_min	NUMERIC(5,2)	No		Humedad volumétrica mínima recomendada (%).
humedad_optima_max	NUMERIC(5,2)	No		Humedad volumétrica máxima recomendada (%).
ph_optimo_min	NUMERIC(4,2)	No		pH mínimo recomendado.
ph_optimo_max	NUMERIC(4,2)	No		pH máximo recomendado.

Columna	Tipo	Nulo	Clave	Descripción
ce_maxima_ds_m	NUMERIC(5,2)	Sí		Conductividad eléctrica máxima tolerada (dS/m).

Restricciones CHECK: ck_cultivo_cic: ((ciclo_dias > 0)); ck_cultivo_hum: ((humedad_optima_min < humedad_optima_max)); ck_cultivo_ph: (((ph_optimo_min < ph_optimo_max) AND (ph_optimo_min >= (0)) AND (ph_optimo_max <= (14))))

Operario

Personal de campo que riega, aplica insumos, cosecha y mantiene equipos.

Columna	Tipo	Nulo	Clave	Descripción
id_operario	SERIAL	No	PK	Identificador del operario.
dni	CHARACTER(8)	No	UQ	Documento Nacional de Identidad (8 dígitos, único).
nombre	VARCHAR(40)	No		Nombres.
apellido	VARCHAR(40)	No		Apellidos.
rol	VARCHAR(25)	No		Función que desempeña en el fundo.
telefono	VARCHAR(9)	Sí		Celular de contacto (9 dígitos).
id_fundo	INTEGER	Sí	FK → Fundo	Fundo donde labora habitualmente.
fecha_ingreso	DATE	Sí		Fecha de ingreso a la empresa.

Restricciones CHECK: ck_operario_dni: ((dni ~ '^[0-9]{8}\$')); ck_operario_rol: (((rol) = ANY ((ARRAY['Tecnico de campo', 'Ingeniero agronomo', 'Operador de riego', 'Supervisor']))))); ck_operario_tel: (((telefono IS NULL) OR ((telefono) ~ '^[0-9]{9}\$')))

Campania

Periodo de siembra de un cultivo en una parcela.

Columna	Tipo	Nulo	Clave	Descripción
id_campania	SERIAL	No	PK	Identificador de la campaña.
id_parcela	INTEGER	No	FK → Parcela, UQ	Parcela sembrada.
id_cultivo	INTEGER	No	FK → Cultivo	Cultivo sembrado.
fecha_siembra	DATE	No	UQ	Fecha de siembra o inicio de campaña.
fecha_fin_estimada	DATE	No		Fecha estimada de término (siembra + ciclo).
densidad_plantas_ha	INTEGER	Sí		Plantas por hectárea.
estado	VARCHAR(15)	No		Planificada, En curso, Cerrada o Perdida. Por defecto: 'En curso'.
rendimiento_kg_ha	NUMERIC(10,2)	No		Atributo derivado: kg cosechados / área; lo mantiene un trigger. Por defecto: 0.

Restricciones CHECK: ck_campania_dens: (((densidad_plantas_ha IS NULL) OR (densidad_plantas_ha > 0))); ck_campania_est: (((estado) = ANY ((ARRAY['Planificada', 'En curso', 'Cerrada', 'Perdida']))))); ck_campania_fech: ((fecha_fin_estimada > fecha_siembra)); ck_campania_rend: ((rendimiento_kg_ha >= (0)))

Sistema_Riego

Infraestructura de riego instalada en una parcela.

Columna	Tipo	Nulo	Clave	Descripción
id_sistema	SERIAL	No	PK	Identificador del sistema.
id_parcela	INTEGER	No	FK → Parcela	Parcela que abastece.
tipo	VARCHAR(25)	No		Goteo, aspersión, microaspersión o gravedad tecnificada.
caudal_nominal_lph	NUMERIC(8,2)	No		Caudal de diseño en litros por hora.
fuentes_agua	VARCHAR(20)	No		Origen del agua.
fecha_instalacion	DATE	Sí		Fecha de instalación.
estado	VARCHAR(15)	No		Operativo, Averiado o Fuera de servicio. Por defecto: 'Operativo'.

Restricciones CHECK: ck_sistema_caudal: ((caudal_nominal_lph > (0))); ck_sistema_estado: (((estado) = ANY ((ARRAY['Operativo', 'Averiado', 'Fuera de servicio']))))); ck_sistema_fuentes: (((fuentes_agua) = ANY ((ARRAY['Pozo', 'Canal', 'Reservorio', 'Rio']))))); ck_sistema_tipo: (((tipo) = ANY ((ARRAY['Goteo', 'Aspersión', 'Microaspersión', 'Gravedad tecnificada']))))

Dispositivo

Superclase de todo equipo IoT instalado en campo.

Columna	Tipo	Nulo	Clave	Descripción
id_dispositivo	SERIAL	No	PK	Identificador del dispositivo (heredado por las subclases).
codigo_serie	VARCHAR(20)	No	UQ	Número de serie del fabricante (único).
modelo	VARCHAR(30)	No		Modelo comercial.
fabricante	VARCHAR(30)	Sí		Fabricante.
fecha_instalacion	DATE	No		Fecha de instalación en campo.
estado	VARCHAR(15)	No		Activo, Inactivo, Mantenimiento o Baja. Por defecto: 'Activo'.
id_parcela	INTEGER	No	FK → Zona_Manejo	Parcela de la zona donde está instalado.
num_zona	SMALLINT	No	FK → Zona_Manejo	Zona de manejo donde está instalado.

Restricciones CHECK: ck_dispositivo_est: (((estado) = ANY ((ARRAY['Activo', 'Inactivo', 'Mantenimiento', 'Baja']))))

Sensor

Subclase de Dispositivo que mide variables del suelo.

Columna	Tipo	Nulo	Clave	Descripción
id_dispositivo	INTEGER	No	PK, FK → Dispositivo	Clave heredada de Dispositivo.

Columna	Tipo	Nulo	Clave	Descripción
tipo_sensor	VARCHAR(20)	No		Humedad, Multiparametro, Conductividad, pH o Temperatura.
profundidad_cm	SMALLINT	No		Profundidad de instalación en centímetros.
frecuencia_min	SMALLINT	No		Intervalo de muestreo configurado (minutos). Por defecto: 15.
unidad_medida	VARCHAR(10)	Sí		Unidad principal que reporta.
rango_min	NUMERIC(7,2)	Sí		Límite inferior de medición declarado por el fabricante.
rango_max	NUMERIC(7,2)	Sí		Límite superior de medición declarado por el fabricante.

Restricciones CHECK: ck_sensor_frec: ((frecuencia_min > 0)); ck_sensor_prof: (((profundidad_cm >= 0) AND (profundidad_cm <= 200))); ck_sensor_rango: (((rango_min IS NULL) OR (rango_max IS NULL) OR (rango_min < rango_max))); ck_sensor_tipo: (((tipo_sensor) = ANY ((ARRAY['Humedad', 'Multiparametro', 'pH', 'Conductividad', 'Temperatura']))))

Controlador

Subclase de Dispositivo que abre y cierra válvulas de un sistema de riego.

Columna	Tipo	Nulo	Clave	Descripción
id_dispositivo	INTEGER	No	PK, FK → Dispositivo	Clave heredada de Dispositivo.
id_sistema	INTEGER	No	FK → Sistema_Riego	Sistema de riego que acciona.
caudal_max_lps	NUMERIC(6,2)	No		Caudal máximo que admite la válvula (L/s).
tipo_valvula	VARCHAR(20)	No		Solenoide, hidráulica o manual motorizada.

Restricciones CHECK: ck_controlador_cau: ((caudal_max_lps > (0))); ck_controlador_val: (((tipo_valvula) = ANY ((ARRAY['Solenoide', 'Hidraulica', 'Manual motorizada']))))

Lectura_Suelo

Medición enviada por un sensor en un instante (entidad débil).

Columna	Tipo	Nulo	Clave	Descripción
id_dispositivo	INTEGER	No	PK, FK → Sensor	Sensor que reporta (parte de la clave).
fecha_hora	TIMESTAMP	No	PK	Instante de la medición, truncado al minuto (clave parcial).
humedad_pct	NUMERIC(5,2)	Sí		Humedad volumétrica del suelo (%). Nulo si el sensor no la mide.
temperatura_c	NUMERIC(5,2)	Sí		Temperatura del suelo (°C).
ph	NUMERIC(4,2)	Sí		pH del suelo. Nulo si el sensor no lo mide.
conductividad_ds_m	NUMERIC(5,2)	Sí		Conductividad eléctrica (dS/m). Nulo si el sensor no la mide.
bateria_pct	SMALLINT	Sí		Carga de batería del equipo al enviar (%).

Restricciones CHECK: ck_lectura_bat: (((bateria_pct IS NULL) OR ((bateria_pct >= 0) AND (bateria_pct <= 100)))); ck_lectura_ce: (((conductividad_ds_m IS NULL) OR (conductividad_ds_m >= (0)))); ck_lectura_hum: (((humedad_pct IS NULL) OR ((humedad_pct >= (0)) AND (humedad_pct <= (100)))); ck_lectura_ph: (((ph IS NULL) OR ((ph >= (0)) AND (ph <= (14)))); ck_lectura_temp: (((temperatura_c IS NULL) OR ((temperatura_c >= ('-10')) AND (temperatura_c <= (60)))))

Evento_Riego

Riego ejecutado sobre una zona por un sistema.

Columna	Tipo	Nulo	Clave	Descripción
id_evento	BIGSERIAL	No	PK	Identificador del evento.
id_sistema	INTEGER	No	FK → Sistema_Riego	Sistema de riego utilizado.
id_parcela	INTEGER	No	FK → Zona_Manejo, UQ	Parcela de la zona regada.
num_zona	SMALLINT	No	FK → Zona_Manejo, UQ	Zona regada.
id_operario	INTEGER	Sí	FK → Operario	Operario que lo ejecutó o supervisó (nulo si fue automático sin supervisión).
fecha_hora_inicio	TIMESTAMP	No	UQ	Inicio del riego.
duracion_min	INTEGER	No		Duración en minutos (máximo 720).
volumen_litros	NUMERIC(10,2)	No		Volumen aplicado; si no se reporta, lo calcula un trigger.
modo	VARCHAR(12)	No		Automatico, Manual o Programado.

Restricciones CHECK: ck_evento_dur: (((duracion_min > 0) AND (duracion_min <= 720))); ck_evento_modo: (((modo) = ANY ((ARRAY['Automatico', 'Manual', 'Programado'])))); ck_evento_vol: ((volumen_litros >= (0)))

Alerta

Aviso generado a partir de una lectura fuera de rango.

Columna	Tipo	Nulo	Clave	Descripción
id_alerta	BIGSERIAL	No	PK	Identificador de la alerta.
id_dispositivo	INTEGER	No	FK → Lectura_Suelo, UQ	Sensor de la lectura que originó la alerta.
fecha_hora	TIMESTAMP	No	FK → Lectura_Suelo, UQ	Instante de la lectura que originó la alerta.
tipo	VARCHAR(25)	No	UQ	Humedad baja, Humedad alta, pH fuera de rango, Salinidad alta o Bateria baja.
severidad	VARCHAR(10)	No		Baja, Media, Alta o Critica.
valor_registrado	NUMERIC(7,2)	Sí		Valor que disparó la alerta.
atendida	BOOLEAN	No		Indica si la alerta fue atendida. Por defecto: false .
fecha_atencion	TIMESTAMP	Sí		Momento de atención; obligatorio si atendida es verdadero.

Columna	Tipo	Nulo	Clave	Descripción
id_operario	INTEGER	Sí	FK → Operario	Operario que la atendió.

Restricciones CHECK: ck_alerta_aten: (((atendida = false) AND (fecha_atencion IS NULL)) OR ((atendida = true) AND (fecha_atencion IS NOT NULL))); ck_alerta_sev: (((severidad) = ANY ((ARRAY['Baja', 'Media', 'Alta', 'Critica']))))); ck_alerta_tipo: (((tipo) = ANY ((ARRAY['Humedad baja', 'Humedad alta', 'pH fuera de rango', 'Salinidad alta', 'Bateria baja']))))

Cosecha

Registro de un pase de cosecha de una campaña.

Columna	Tipo	Nulo	Clave	Descripción
id_cosecha	SERIAL	No	PK	Identificador del registro.
id_campania	INTEGER	No	FK → Campania	Campania cosechada.
id_operario	INTEGER	Sí	FK → Operario	Responsable del registro en campo.
fecha	DATE	No		Fecha de cosecha.
cantidad_kg	NUMERIC(10,2)	No		Peso cosechado en kilogramos.
categoria	VARCHAR(12)	No		Calidad comercial: Primera, Segunda, Tercera o Descarte.
humedad_producto_pct	NUMERIC(5,2)	Sí		Humedad del producto cosechado (%).
precio_kg_soles	NUMERIC(7,2)	Sí		Precio de venta estimado por kilogramo (S/).

Restricciones CHECK: ck_cosecha_cat: (((categoria) = ANY ((ARRAY['Primera', 'Segunda', 'Tercera', 'Descarte']))))); ck_cosecha_hum: (((humedad_producto_pct IS NULL) OR ((humedad_producto_pct >= (0)) AND (humedad_producto_pct <= (100)))))); ck_cosecha_kg: ((cantidad_kg > (0))); ck_cosecha_prec: (((precio_kg_soles IS NULL) OR (precio_kg_soles >= (0))))

Insumo

Catálogo de fertilizantes, enmiendas y productos fitosanitarios.

Columna	Tipo	Nulo	Clave	Descripción
id_insumo	SERIAL	No	PK	Identificador del insumo.
nombre	VARCHAR(50)	No	UQ	Nombre comercial y concentración.
tipo	VARCHAR(20)	No		Fertilizante, Fungicida, Insecticida, Enmienda o Bioestimulante.
unidad	VARCHAR(10)	No		Unidad de dosificación (kg o L).
costo_unitario	NUMERIC(8,2)	No		Costo por unidad (S/).
periodo_carencia_dias	SMALLINT	Sí		Días mínimos entre la aplicación y la cosecha.

Restricciones CHECK: ck_insumo_car: (((periodo_carencia_dias IS NULL) OR (periodo_carencia_dias >= (0))))); ck_insumo_cost: ((costo_unitario >= (0))); ck_insumo_tipo: (((tipo) = ANY ((ARRAY['Fertilizante', 'Fungicida', 'Insecticida', 'Enmienda', 'Bioestimulante']))))

Aplicacion_Insumo

Relación ternaria: un operario aplica un insumo en una zona de manejo.

Columna	Tipo	Nulo	Clave	Descripción
id_parcela	INTEGER	No	PK, FK → Zona_Manejo	Parcela de la zona tratada.
num_zona	SMALLINT	No	PK, FK → Zona_Manejo	Zona tratada.
id_insumo	INTEGER	No	PK, FK → Insumo	Insumo aplicado.
id_operario	INTEGER	No	PK, FK → Operario	Operario que aplicó.
fecha_hora	TIMESTAMP	No	PK	Momento de la aplicación.
dosis	NUMERIC(8,2)	No		Cantidad aplicada en la unidad del insumo.
metodo	VARCHAR(20)	No		Fertirriego, Foliar, Al suelo o Drench.

Restricciones CHECK: ck_aplicacion_dos: (((dosis > (0)))); ck_aplicacion_met: (((metodo) = ANY ((ARRAY['Fertirriego', 'Foliar', 'Al suelo', 'Drench']))))

Mantenimiento

Relación N:M entre operarios y dispositivos que atienden.

Columna	Tipo	Nulo	Clave	Descripción
id_dispositivo	INTEGER	No	PK, FK → Dispositivo	Equipo intervenido.
id_operario	INTEGER	No	PK, FK → Operario	Técnico que realizó el trabajo.
fecha	DATE	No	PK	Fecha de la intervención.
tipo	VARCHAR(15)	No		Preventivo, Correctivo o Calibracion.
observacion	VARCHAR(120)	Sí		Detalle del trabajo realizado.
costo	NUMERIC(8,2)	Sí		Costo de la intervención (S/).

Restricciones CHECK: ck_mantenimiento_cost: (((costo IS NULL) OR (costo >= (0)))); ck_mantenimiento_tipo: (((tipo) = ANY ((ARRAY['Preventivo', 'Correctivo', 'Calibracion']))))

Auditoria_Campania

Bitácora técnica de cambios de estado de campañas (no forma parte del modelo conceptual).

Columna	Tipo	Nulo	Clave	Descripción
id_auditoria	BIGSERIAL	No	PK	Identificador del registro de auditoría.
id_campania	INTEGER	No		Campaña modificada.
estado_anterior	VARCHAR(15)	Sí		Estado previo al cambio.
estado_nuevo	VARCHAR(15)	Sí		Estado posterior al cambio.
usuario_bd	VARCHAR(40)	No		Usuario de base de datos que hizo el cambio. Por defecto: CURRENT_USER.

Columna	Tipo	Nulo	Clave	Descripción
fecha_cambio	TIMESTAMP	No		Momento del cambio. Por defecto: now().

4. Implementación de la base de datos

4.1. Creación de Tablas en PostgreSQL

El script `01_ddl.sql` crea las 18 tablas en orden de dependencia: primero los catálogos y al final las tablas de hechos. Todas las restricciones llevan nombre propio con un prefijo que indica su tipo (`pk_`, `fk_`, `uq_`, `ck_`), así el mensaje de error dice qué regla se violó. Algunas decisiones sobre los tipos de datos:

- **NUMERIC** con precisión fija para las mediciones y los montos, porque es exacto y evita errores de redondeo al sumar litros o soles.
- **SMALLINT** para valores pequeños como el número de zona o la profundidad del sensor.
- **TIMESTAMP** sin zona horaria, porque todos los fundos operan en la hora de Lima (UTC−5) y el Perú no aplica horario de verano.
- Listas de valores con **CHECK** en lugar de tipos **ENUM**. Un **ENUM** no permite eliminar valores, mientras que un **CHECK** se reemplaza con dos sentencias **ALTER TABLE**.

01_ddl.sql

```

1  -- =====
2  -- Proyecto: Agricultura de Precision y Sensores IoT Agropecuarios
3  -- Curso: Base de Datos I (CS2041) - Laboratorio 14 - 2026-2
4  -- Archivo: 01_ddl.sql (creacion de esquema)
5  -- Motor: PostgreSQL 16
6  -- =====
7
8  DROP TABLE IF EXISTS Auditoria_Campania CASCADE;
9  DROP TABLE IF EXISTS Mantenimiento CASCADE;
10 DROP TABLE IF EXISTS Aplicacion_Insumo CASCADE;
11 DROP TABLE IF EXISTS Insumo CASCADE;
12 DROP TABLE IF EXISTS Cosecha CASCADE;
13 DROP TABLE IF EXISTS Alerta CASCADE;
14 DROP TABLE IF EXISTS Evento_Riego CASCADE;
15 DROP TABLE IF EXISTS Lectura_Suelo CASCADE;
16 DROP TABLE IF EXISTS Controlador CASCADE;
17 DROP TABLE IF EXISTS Sensor CASCADE;
18 DROP TABLE IF EXISTS Dispositivo CASCADE;
19 DROP TABLE IF EXISTS Sistema_Riego CASCADE;
20 DROP TABLE IF EXISTS Campania CASCADE;
21 DROP TABLE IF EXISTS Cultivo CASCADE;
22 DROP TABLE IF EXISTS Operario CASCADE;
23 DROP TABLE IF EXISTS Zona_Manejo CASCADE;
24 DROP TABLE IF EXISTS Parcela CASCADE;
25 DROP TABLE IF EXISTS Fundo CASCADE;
26
27 -----
28 -- 1. FUNDO (entidad fuerte)
29 -----
30 CREATE TABLE Fundo (
31     id_fundo SERIAL,
32     nombre VARCHAR(60) NOT NULL,
33     departamento VARCHAR(40) NOT NULL,
34     provincia VARCHAR(40) NOT NULL,

```

```

35     distrito      VARCHAR(40),
36     area_total_ha  NUMERIC(8,2) NOT NULL,
37     latitud        NUMERIC(9,6),
38     longitud       NUMERIC(9,6),
39     fecha_registro DATE          NOT NULL DEFAULT CURRENT_DATE,
40     CONSTRAINT pk_fundo PRIMARY KEY (id_fundo),
41     CONSTRAINT uq_fundo_nombre UNIQUE (nombre, departamento),
42     CONSTRAINT ck_fundo_area CHECK (area_total_ha > 0),
43     CONSTRAINT ck_fundo_lat CHECK (latitud BETWEEN -19 AND 1),
44     CONSTRAINT ck_fundo_lon CHECK (longitud BETWEEN -82 AND -68)
45 );
46
47 -----
48 -- 2. PARCELA (entidad fuerte, dependiente de Fundo por FK)
49 -----
50 CREATE TABLE Parcela (
51     id_parcela      SERIAL,
52     id_fundo        INTEGER      NOT NULL,
53     codigo_parcela  VARCHAR(15) NOT NULL,
54     area_ha         NUMERIC(7,2) NOT NULL,
55     tipo_suelo      VARCHAR(20) NOT NULL,
56     pendiente_pct   NUMERIC(4,1),
57     fecha_habilitacion DATE,
58     CONSTRAINT pk_parcela PRIMARY KEY (id_parcela),
59     CONSTRAINT fk_parcela_fundo FOREIGN KEY (id_fundo) REFERENCES Fundo(id_fundo)
60     ON DELETE RESTRICT ON UPDATE CASCADE,
61     CONSTRAINT uq_parcela_codigo UNIQUE (id_fundo, codigo_parcela),
62     CONSTRAINT ck_parcela_area CHECK (area_ha > 0),
63     CONSTRAINT ck_parcela_pend CHECK (pendiente_pct BETWEEN 0 AND 100),
64     CONSTRAINT ck_parcela_suelo CHECK (tipo_suelo IN
65     ('Arenoso','Franco','Franco arenoso','Franco arcilloso','Arcilloso'))
66 );
67
68 -----
69 -- 3. ZONA_MANEJO (entidad debil: depende de Parcela)
70 -- Clave parcial: num_zona. Clave primaria: (id_parcela, num_zona)
71 -----
72 CREATE TABLE Zona_Manejo (
73     id_parcela      INTEGER      NOT NULL,
74     num_zona        SMALLINT     NOT NULL,
75     area_ha         NUMERIC(6,2) NOT NULL,
76     descripcion     VARCHAR(80),
77     CONSTRAINT pk_zona PRIMARY KEY (id_parcela, num_zona),
78     CONSTRAINT fk_zona_parcel FOREIGN KEY (id_parcela) REFERENCES Parcela(id_parcela)
79     ON DELETE CASCADE ON UPDATE CASCADE,
80     CONSTRAINT ck_zona_num CHECK (num_zona > 0),
81     CONSTRAINT ck_zona_area CHECK (area_ha > 0)
82 );
83
84 -----
85 -- 4. CULTIVO (catalogo)
86 -----
87 CREATE TABLE Cultivo (
88     id_cultivo      SERIAL,
89     nombre_comun     VARCHAR(40) NOT NULL,
90     nombre_cientifico VARCHAR(60),
91     ciclo_dias       INTEGER      NOT NULL,
92     humedad_optima_min NUMERIC(5,2) NOT NULL,
93     humedad_optima_max NUMERIC(5,2) NOT NULL,
94     ph_optimo_min     NUMERIC(4,2) NOT NULL,
95     ph_optimo_max     NUMERIC(4,2) NOT NULL,
96     ce_maxima_ds_m    NUMERIC(5,2),
97     CONSTRAINT pk_cultivo PRIMARY KEY (id_cultivo),
98     CONSTRAINT uq_cultivo_nom UNIQUE (nombre_comun),
99     CONSTRAINT ck_cultivo_hum CHECK (humedad_optima_min < humedad_optima_max),
100    CONSTRAINT ck_cultivo_ph CHECK (ph_optimo_min < ph_optimo_max
101    AND ph_optimo_min >= 0 AND ph_optimo_max <= 14),
102    CONSTRAINT ck_cultivo_cic CHECK (ciclo_dias > 0)
103 );

```

```

104
105 -- -----
106 -- 5. OPERARIO
107 -- -----
108 CREATE TABLE Operario (
109     id_operario    SERIAL,
110     dni            CHAR(8)      NOT NULL,
111     nombre         VARCHAR(40) NOT NULL,
112     apellido       VARCHAR(40) NOT NULL,
113     rol            VARCHAR(25) NOT NULL,
114     telefono       VARCHAR(9),
115     id_fundo       INTEGER,
116     fecha_ingreso  DATE,
117     CONSTRAINT pk_operario    PRIMARY KEY (id_operario),
118     CONSTRAINT uq_operario_dni UNIQUE (dni),
119     CONSTRAINT fk_operario_fun FOREIGN KEY (id_fundo) REFERENCES Fundo(id_fundo)
120     ON DELETE SET NULL ON UPDATE CASCADE,
121     CONSTRAINT ck_operario_dni CHECK (dni ~ '[0-9]{8}$'),
122     CONSTRAINT ck_operario_tel CHECK (telefono IS NULL OR telefono ~ '[0-9]{9}$'),
123     CONSTRAINT ck_operario_rol CHECK (rol IN
124     ('Tecnico de campo','Ingeniero agronomo','Operador de riego','Supervisor'))
125 );
126
127 -- -----
128 -- 6. CAMPANIA (entidad asociativa Parcela x Cultivo con clave propia)
129 -- -----
130 CREATE TABLE Campania (
131     id_campania    SERIAL,
132     id_parcela     INTEGER      NOT NULL,
133     id_cultivo     INTEGER      NOT NULL,
134     fecha_siembra  DATE         NOT NULL,
135     fecha_fin_estimada DATE     NOT NULL,
136     densidad_plantas_ha INTEGER,
137     estado         VARCHAR(15) NOT NULL DEFAULT 'En curso',
138     rendimiento_kg_ha NUMERIC(10,2) NOT NULL DEFAULT 0,
139     CONSTRAINT pk_campania    PRIMARY KEY (id_campania),
140     CONSTRAINT fk_campania_parcel FOREIGN KEY (id_parcela) REFERENCES Parcela(id_parcela)
141     ON DELETE RESTRICT ON UPDATE CASCADE,
142     CONSTRAINT fk_campania_cult FOREIGN KEY (id_cultivo) REFERENCES Cultivo(id_cultivo)
143     ON DELETE RESTRICT ON UPDATE CASCADE,
144     CONSTRAINT uq_campania    UNIQUE (id_parcela, fecha_siembra),
145     CONSTRAINT ck_campania_fecha CHECK (fecha_fin_estimada > fecha_siembra),
146     CONSTRAINT ck_campania_dens CHECK (densidad_plantas_ha IS NULL OR densidad_plantas_ha > 0),
147     CONSTRAINT ck_campania_estado CHECK (estado IN ('Planificada','En curso','Cerrada','Perdida')),
148     CONSTRAINT ck_campania_rend CHECK (rendimiento_kg_ha >= 0)
149 );
150
151 -- -----
152 -- 7. SISTEMA_RIEGO
153 -- -----
154 CREATE TABLE Sistema_Riego (
155     id_sistema     SERIAL,
156     id_parcela     INTEGER      NOT NULL,
157     tipo           VARCHAR(25) NOT NULL,
158     caudal_nominal_lph NUMERIC(8,2) NOT NULL,
159     fuente_agua     VARCHAR(20) NOT NULL,
160     fecha_instalacion DATE,
161     estado         VARCHAR(15) NOT NULL DEFAULT 'Operativo',
162     CONSTRAINT pk_sistema    PRIMARY KEY (id_sistema),
163     CONSTRAINT fk_sistema_parcel FOREIGN KEY (id_parcela) REFERENCES Parcela(id_parcela)
164     ON DELETE CASCADE ON UPDATE CASCADE,
165     CONSTRAINT ck_sistema_caudal CHECK (caudal_nominal_lph > 0),
166     CONSTRAINT ck_sistema_tipo CHECK (tipo IN
167     ('Goteo','Aspersión','Microaspersión','Gravedad tecnificada')),
168     CONSTRAINT ck_sistema_fuente CHECK (fuente_agua IN ('Pozo','Canal','Reservorio','Rio')),
169     CONSTRAINT ck_sistema_estado CHECK (estado IN ('Operativo','Averiado','Fuera de servicio'))
170 );
171
172 -- -----

```

```

173 -- 8. DISPOSITIVO (superclase de la jerarquia Is-A)
174 -----
175 CREATE TABLE Dispositivo (
176     id_dispositivo SERIAL,
177     codigo_serie VARCHAR(20) NOT NULL,
178     modelo VARCHAR(30) NOT NULL,
179     fabricante VARCHAR(30),
180     fecha_instalacion DATE NOT NULL,
181     estado VARCHAR(15) NOT NULL DEFAULT 'Activo',
182     id_parcela INTEGER NOT NULL,
183     num_zona SMALLINT NOT NULL,
184     CONSTRAINT pk_dispositivo PRIMARY KEY (id_dispositivo),
185     CONSTRAINT uq_dispositivo_ser UNIQUE (codigo_serie),
186     CONSTRAINT fk_dispositivo_zona FOREIGN KEY (id_parcela, num_zona)
187         REFERENCES Zona_Manejo(id_parcela, num_zona)
188         ON DELETE CASCADE ON UPDATE CASCADE,
189     CONSTRAINT ck_dispositivo_est CHECK (estado IN ('Activo','Inactivo','Mantenimiento','Baja'))
190 );
191
192 -----
193 -- 9. SENSOR (subclase de Dispositivo)
194 -----
195 CREATE TABLE Sensor (
196     id_dispositivo INTEGER NOT NULL,
197     tipo_sensor VARCHAR(20) NOT NULL,
198     profundidad_cm SMALLINT NOT NULL,
199     frecuencia_min SMALLINT NOT NULL DEFAULT 15,
200     unidad_medida VARCHAR(10),
201     rango_min NUMERIC(7,2),
202     rango_max NUMERIC(7,2),
203     CONSTRAINT pk_sensor PRIMARY KEY (id_dispositivo),
204     CONSTRAINT fk_sensor_disp FOREIGN KEY (id_dispositivo) REFERENCES Dispositivo(id_dispositivo)
205         ON DELETE CASCADE ON UPDATE CASCADE,
206     CONSTRAINT ck_sensor_tipo CHECK (tipo_sensor IN
207         ('Humedad','Multiparametro','pH','Conductividad','Temperatura')),
208     CONSTRAINT ck_sensor_prof CHECK (profundidad_cm BETWEEN 0 AND 200),
209     CONSTRAINT ck_sensor_frec CHECK (frecuencia_min > 0),
210     CONSTRAINT ck_sensor_rango CHECK (rango_min IS NULL OR rango_max IS NULL OR rango_min < rango_max)
211 );
212
213 -----
214 -- 10. CONTROLADOR (subclase de Dispositivo)
215 -----
216 CREATE TABLE Controlador (
217     id_dispositivo INTEGER NOT NULL,
218     id_sistema INTEGER NOT NULL,
219     caudal_max_lps NUMERIC(6,2) NOT NULL,
220     tipo_valvula VARCHAR(20) NOT NULL,
221     CONSTRAINT pk_controlador PRIMARY KEY (id_dispositivo),
222     CONSTRAINT fk_controlador_disp FOREIGN KEY (id_dispositivo) REFERENCES Dispositivo(id_dispositivo)
223         ON DELETE CASCADE ON UPDATE CASCADE,
224     CONSTRAINT fk_controlador_sis FOREIGN KEY (id_sistema) REFERENCES Sistema_Riego(id_sistema)
225         ON DELETE CASCADE ON UPDATE CASCADE,
226     CONSTRAINT ck_controlador_cau CHECK (caudal_max_lps > 0),
227     CONSTRAINT ck_controlador_val CHECK (tipo_valvula IN ('Solenoide','Hidraulica','Manual motorizada'))
228 );
229
230 -----
231 -- 11. LECTURA_SUELO (entidad debil: depende del Sensor)
232 -- Clave parcial: fecha_hora. PK: (id_dispositivo, fecha_hora)
233 -----
234 CREATE TABLE Lectura_Suelo (
235     id_dispositivo INTEGER NOT NULL,
236     fecha_hora TIMESTAMP NOT NULL,
237     humedad_pct NUMERIC(5,2),
238     temperatura_c NUMERIC(5,2),
239     ph NUMERIC(4,2),
240     conductividad_ds_m NUMERIC(5,2),
241     bateria_pct SMALLINT,

```

```

242     CONSTRAINT pk_lectura PRIMARY KEY (id_dispositivo, fecha_hora),
243     CONSTRAINT fk_lectura_sens FOREIGN KEY (id_dispositivo) REFERENCES Sensor(id_dispositivo)
244         ON DELETE CASCADE ON UPDATE CASCADE,
245     CONSTRAINT ck_lectura_hum CHECK (humedad_pct IS NULL OR humedad_pct BETWEEN 0 AND 100),
246     CONSTRAINT ck_lectura_temp CHECK (temperatura_c IS NULL OR temperatura_c BETWEEN -10 AND 60),
247     CONSTRAINT ck_lectura_ph CHECK (ph IS NULL OR ph BETWEEN 0 AND 14),
248     CONSTRAINT ck_lectura_ce CHECK (conductividad_ds_m IS NULL OR conductividad_ds_m >= 0),
249     CONSTRAINT ck_lectura_bat CHECK (bateria_pct IS NULL OR bateria_pct BETWEEN 0 AND 100)
250 );
251
252 -----
253 -- 12. EVENTO_RIEGO
254 -----
255 CREATE TABLE Evento_Riego (
256     id_evento BIGSERIAL,
257     id_sistema INTEGER NOT NULL,
258     id_parcela INTEGER NOT NULL,
259     num_zona SMALLINT NOT NULL,
260     id_operario INTEGER,
261     fecha_hora_inicio TIMESTAMP NOT NULL,
262     duracion_min INTEGER NOT NULL,
263     volumen_litros NUMERIC(10,2) NOT NULL,
264     modo VARCHAR(12) NOT NULL,
265     CONSTRAINT pk_evento PRIMARY KEY (id_evento),
266     CONSTRAINT fk_evento_sis FOREIGN KEY (id_sistema) REFERENCES Sistema_Riego(id_sistema)
267         ON DELETE CASCADE ON UPDATE CASCADE,
268     CONSTRAINT fk_evento_zona FOREIGN KEY (id_parcela, num_zona)
269         REFERENCES Zona_Manejo(id_parcela, num_zona)
270         ON DELETE CASCADE ON UPDATE CASCADE,
271     CONSTRAINT fk_evento_oper FOREIGN KEY (id_operario) REFERENCES Operario(id_operario)
272         ON DELETE SET NULL ON UPDATE CASCADE,
273     CONSTRAINT uq_evento UNIQUE (id_parcela, num_zona, fecha_hora_inicio),
274     CONSTRAINT ck_evento_dur CHECK (duracion_min > 0 AND duracion_min <= 720),
275     CONSTRAINT ck_evento_vol CHECK (volumen_litros >= 0),
276     CONSTRAINT ck_evento_modos CHECK (modo IN ('Automatico', 'Manual', 'Programado'))
277 );
278
279 -----
280 -- 13. ALERTA (se origina en una lectura concreta)
281 -----
282 CREATE TABLE Alerta (
283     id_alerta BIGSERIAL,
284     id_dispositivo INTEGER NOT NULL,
285     fecha_hora TIMESTAMP NOT NULL,
286     tipo VARCHAR(25) NOT NULL,
287     severidad VARCHAR(10) NOT NULL,
288     valor_registrado NUMERIC(7,2),
289     atendida BOOLEAN NOT NULL DEFAULT FALSE,
290     fecha_atencion TIMESTAMP,
291     id_operario INTEGER,
292     CONSTRAINT pk_alerta PRIMARY KEY (id_alerta),
293     CONSTRAINT fk_alerta_lect FOREIGN KEY (id_dispositivo, fecha_hora)
294         REFERENCES Lectura_Suelo(id_dispositivo, fecha_hora)
295         ON DELETE CASCADE ON UPDATE CASCADE,
296     CONSTRAINT fk_alerta_oper FOREIGN KEY (id_operario) REFERENCES Operario(id_operario)
297         ON DELETE SET NULL ON UPDATE CASCADE,
298     CONSTRAINT uq_alerta UNIQUE (id_dispositivo, fecha_hora, tipo),
299     CONSTRAINT ck_alerta_tipo CHECK (tipo IN
300         ('Humedad baja', 'Humedad alta', 'pH fuera de rango', 'Salinidad alta', 'Bateria baja')),
301     CONSTRAINT ck_alerta_sev CHECK (severidad IN ('Baja', 'Media', 'Alta', 'Critica')),
302     CONSTRAINT ck_alerta_aten CHECK ((atendida = FALSE AND fecha_atencion IS NULL)
303         OR (atendida = TRUE AND fecha_atencion IS NOT NULL))
304 );
305
306 -----
307 -- 14. COSECHA
308 -----
309 CREATE TABLE Cosecha (
310     id_cosecha SERIAL,

```

```

311     id_campania      INTEGER      NOT NULL,
312     id_operario      INTEGER,
313     fecha            DATE          NOT NULL,
314     cantidad_kg       NUMERIC(10,2) NOT NULL,
315     categoria        VARCHAR(12)  NOT NULL,
316     humedad_producto_pct NUMERIC(5,2),
317     precio_kg_soles   NUMERIC(7,2),
318     CONSTRAINT pk_cosecha PRIMARY KEY (id_cosecha),
319     CONSTRAINT fk_cosecha_camp FOREIGN KEY (id_campania) REFERENCES Campania(id_campania)
320                             ON DELETE CASCADE ON UPDATE CASCADE,
321     CONSTRAINT fk_cosecha_oper FOREIGN KEY (id_operario) REFERENCES Operario(id_operario)
322                             ON DELETE SET NULL ON UPDATE CASCADE,
323     CONSTRAINT ck_cosecha_kg CHECK (cantidad_kg > 0),
324     CONSTRAINT ck_cosecha_cat CHECK (categoria IN ('Primera','Segunda','Tercera','Descarte')),
325     CONSTRAINT ck_cosecha_hum CHECK (humedad_producto_pct IS NULL
326                                     OR humedad_producto_pct BETWEEN 0 AND 100),
327     CONSTRAINT ck_cosecha_prec CHECK (precio_kg_soles IS NULL OR precio_kg_soles >= 0)
328 );
329
330 -----
331 -- 15. INSUMO
332 -----
333 CREATE TABLE Insumo (
334     id_insumo          SERIAL,
335     nombre             VARCHAR(50) NOT NULL,
336     tipo               VARCHAR(20) NOT NULL,
337     unidad             VARCHAR(10) NOT NULL,
338     costo_unitario     NUMERIC(8,2) NOT NULL,
339     periodo_carencia_dias SMALLINT,
340     CONSTRAINT pk_insumo PRIMARY KEY (id_insumo),
341     CONSTRAINT uq_insumo_nom UNIQUE (nombre),
342     CONSTRAINT ck_insumo_tipo CHECK (tipo IN
343                                     ('Fertilizante','Fungicida','Insecticida','Enmienda','Bioestimulante')),
344     CONSTRAINT ck_insumo_cost CHECK (costo_unitario >= 0),
345     CONSTRAINT ck_insumo_car CHECK (periodo_carencia_dias IS NULL OR periodo_carencia_dias >= 0)
346 );
347
348 -----
349 -- 16. APLICACION_INSUMO (relacion ternaria Zona x Insumo x Operario)
350 -----
351 CREATE TABLE Aplicacion_Insumo (
352     id_parcela    INTEGER      NOT NULL,
353     num_zona      SMALLINT     NOT NULL,
354     id_insumo     INTEGER      NOT NULL,
355     id_operario   INTEGER      NOT NULL,
356     fecha_hora    TIMESTAMP    NOT NULL,
357     dosis         NUMERIC(8,2) NOT NULL,
358     metodo        VARCHAR(20)  NOT NULL,
359     CONSTRAINT pk_aplicacion PRIMARY KEY (id_parcela, num_zona, id_insumo, id_operario, fecha_hora),
360     CONSTRAINT fk_aplicacion_zona FOREIGN KEY (id_parcela, num_zona)
361                             REFERENCES Zona_Manejo(id_parcela, num_zona)
362                             ON DELETE CASCADE ON UPDATE CASCADE,
363     CONSTRAINT fk_aplicacion_ins FOREIGN KEY (id_insumo) REFERENCES Insumo(id_insumo)
364                             ON DELETE RESTRICT ON UPDATE CASCADE,
365     CONSTRAINT fk_aplicacion_ope FOREIGN KEY (id_operario) REFERENCES Operario(id_operario)
366                             ON DELETE RESTRICT ON UPDATE CASCADE,
367     CONSTRAINT ck_aplicacion_dos CHECK (dosis > 0),
368     CONSTRAINT ck_aplicacion_met CHECK (metodo IN ('Fertirriego','Foliar','Al suelo','Drench'))
369 );
370
371 -----
372 -- 17. MANTENIMIENTO (relacion N:M Operario x Dispositivo)
373 -----
374 CREATE TABLE Mantenimiento (
375     id_dispositivo INTEGER      NOT NULL,
376     id_operario    INTEGER      NOT NULL,
377     fecha          DATE          NOT NULL,
378     tipo           VARCHAR(15)  NOT NULL,
379     observacion    VARCHAR(120),

```

```

380     costo          NUMERIC(8,2),
381     CONSTRAINT pk_mantenimiento PRIMARY KEY (id_dispositivo, id_operario, fecha),
382     CONSTRAINT fk_mantenimiento_disp FOREIGN KEY (id_dispositivo) REFERENCES Dispositivo(id_dispositivo)
383         ON DELETE CASCADE ON UPDATE CASCADE,
384     CONSTRAINT fk_mantenimiento_oper FOREIGN KEY (id_operario) REFERENCES Operario(id_operario)
385         ON DELETE RESTRICT ON UPDATE CASCADE,
386     CONSTRAINT ck_mantenimiento_tipo CHECK (tipo IN ('Preventivo','Correctivo','Calibracion')),
387     CONSTRAINT ck_mantenimiento_cost CHECK (costo IS NULL OR costo >= 0)
388 );
389
390 -----
391 -- 18. AUDITORIA_CAMPANIA (bitacora alimentada por trigger)
392 -----
393 CREATE TABLE Auditoria_Campania (
394     id_auditoria BIGSERIAL,
395     id_campania INTEGER NOT NULL,
396     estado_anterior VARCHAR(15),
397     estado_nuevo VARCHAR(15),
398     usuario_bd VARCHAR(40) NOT NULL DEFAULT CURRENT_USER,
399     fecha_cambio TIMESTAMP NOT NULL DEFAULT NOW(),
400     CONSTRAINT pk_auditoria PRIMARY KEY (id_auditoria)
401 );

```

4.2. Carga de datos

La carga la hace `ejecutar_todo.py`, que funciona igual en Windows y en Linux. Para cada escenario genera los archivos CSV, crea la base, ejecuta el DDL y los comentarios, y luego copia las tablas con la instrucción `COPY` de PostgreSQL a través de `psycopg2`. Usamos `COPY` y no `INSERT` porque procesa el archivo completo en un solo comando, sin un viaje de red por fila. El millón de registros se cargó en unos diez segundos.

El orden de las operaciones importa:

1. Se cargan las tablas respetando las claves foráneas: catálogos, parcelas y zonas, dispositivos y subclases, y al final lecturas, alertas, riegos, cosechas, aplicaciones y mantenimientos.
2. Se ajustan las secuencias con `setval` para que los próximos `SERIAL` no choquen con los identificadores cargados.
3. Se calcula una sola vez el rendimiento de todas las campañas con un `UPDATE`.
4. Recién entonces se crean vistas, procedimientos y triggers. Si los triggers existieran durante la carga, `trg_alerta_lectura` se ejecutaría 765 861 veces y duplicaría las alertas ya generadas.
5. Se crean los índices de optimización y se ejecuta `VACUUM ANALYZE` para actualizar las estadísticas del planificador.

La Tabla 24 muestra las filas de cada tabla por escenario y los tiempos medidos.

Tabla	1K	10K	100K	1M
Fundo	3	5	8	12
Cultivo	8	10	12	12
Insumo	10	12	15	15
Operario	12	40	150	400
Parcela	15	50	200	600
Zona_Manejo	40	150	700	2 400
Sistema_Riego	15	50	200	600
Campania	25	90	400	1 500
Dispositivo	45	170	800	2 800
Sensor	35	130	620	2 200
Controlador	10	40	180	600
Lectura_Suelo	532	6 853	73 515	765 861
Alerta	30	300	3 000	30 000
Evento_Riego	120	1 200	12 000	120 000
Cosecha	40	350	3 000	25 000
Aplicacion_Insumo	40	400	4 000	40 000
Mantenimiento	20	150	1 200	8 000
Total	1 000	10 000	100 000	1 000 000
Generación de CSV (s)	0.0	0.1	1.1	13.0
Carga con COPY (s)	0.0	0.1	1.1	10.7
Creación de índices (s)	0.0	0.0	0.1	1.2
Tamaño de la base (MB)	9.3	10.8	26.4	183.9

Tabla 24: Filas por tabla y tiempos de carga por escenario.

Como los triggers no existían durante la carga, las reglas que ellos hacen cumplir tuvieron que verificarse después con el script `07_verificacion.sql`. Cada columna cuenta violaciones de una regla y el resultado esperado es cero. Durante el desarrollo estas verificaciones detectaron errores del propio generador, como campañas superpuestas o riegos con el sistema de otra parcela, que corregimos antes de medir. El resultado final aparece en la Tabla 25.

07_verificacion.sql

```

1  -- =====
2  -- Verificacion de consistencia posterior a la carga masiva.
3  -- Cada columna cuenta violaciones; el resultado esperado es 0.
4  -- =====
5  SELECT
6  (SELECT count(*) FROM Evento_Riego e JOIN Sistema_Riego s USING (id_sistema)
7   WHERE s.id_parcela <> e.id_parcela) AS riego_otra_parcela,
8  (SELECT count(*) FROM Controlador c JOIN Dispositivo d USING (id_dispositivo)
9   JOIN Sistema_Riego s USING (id_sistema)
10  WHERE s.id_parcela <> d.id_parcela) AS controlador_otra_parcela,
11  (SELECT count(*) FROM (SELECT id_parcela FROM Campania WHERE estado = 'En curso'
12   GROUP BY id_parcela HAVING count(*) > 1) x) AS parcelas_dos_vigentes,
13  (SELECT count(*) FROM Campania a JOIN Campania b
14   ON a.id_parcela = b.id_parcela AND a.id_campania < b.id_campania
15   AND a.fecha_siembra < b.fecha_fin_estimada
16   AND a.fecha_fin_estimada > b.fecha_siembra) AS campanias_superpuestas,
17  (SELECT count(*) FROM Cosecha co JOIN Campania ca USING (id_campania)
18   WHERE co.fecha < ca.fecha_siembra) AS cosechas_antes_siembra,
19  (SELECT count(*) FROM Cosecha co JOIN Campania ca USING (id_campania)
20   WHERE ca.estado = 'Perdida') AS cosechas_campania_perdida,
21  (SELECT count(*) FROM Dispositivo d

```

```

22 WHERE (EXISTS (SELECT 1 FROM Sensor s WHERE s.id_dispositivo = d.id_dispositivo))::int
23 + (EXISTS (SELECT 1 FROM Controlador c WHERE c.id_dispositivo = d.id_dispositivo))::int
24 <> 1) AS fuera_jerarquia_isa,
25 (SELECT count(*) FROM Campania ca
26 WHERE ca.rendimiento_kg_ha <> COALESCE((SELECT ROUND(SUM(co.cantidad_kg) / MAX(p.area_ha), 2)
27 FROM Cosecha co JOIN Parcela p ON p.id_parcela = ca.id_parcela
28 WHERE co.id_campania = ca.id_campania), 0)) AS rendimiento_desactualizado;

```

Regla verificada (violaciones)	1K	10K	100K	1M
Riegos con el sistema de otra parcela	0	0	0	0
Controladores de otra parcela	0	0	0	0
Parcelas con dos campañas vigentes	0	0	0	0
Campañas superpuestas	0	0	0	0
Cosechas anteriores a la siembra	0	0	0	0
Cosechas en campañas perdidas	0	0	0	0
Dispositivos fuera de la jerarquía Is-A	0	0	0	0
Rendimientos desactualizados	0	0	0	0

Tabla 25: Resultado de la verificación de consistencia.

4.3. Simulación de Datos Faltantes

Datos que no existen en fuentes públicas. Ninguna empresa publica las lecturas de sus sensores, así que generamos los datos con el script `generar_datos.py` (Anexo 7.4), que solo usa la biblioteca estándar de Python y una semilla fija, por lo que cada ejecución produce exactamente los mismos datos. Para que las consultas devuelvan resultados con sentido, el generador aplica estas reglas:

- **Geografía y clima.** Doce departamentos con distritos agrícolas reales y una temperatura media de suelo propia de cada zona.
- **Cultivos aptos.** Cada departamento solo siembra cultivos compatibles con su clima: no hay cacao en Ica ni espárrago en la selva.
- **Campañas.** Se encadenan hacia atrás desde la fecha de corte (20 de setiembre de 2026) sin superponerse. La primera de cada parcela es la vigente; el 10 % de las vigentes y el 8 % de las históricas se marcan como perdidas.
- **Sensores.** Cinco tipos, cada uno con las variables que realmente mide y con modelos comerciales compatibles con ese tipo.
- **Lecturas.** La humedad sigue una distribución normal centrada en el punto medio del rango óptimo del cultivo. En el 22 % de las parcelas la media queda 8 puntos por debajo del mínimo, para simular estrés hídrico. La batería se descarga en diente de sierra y se renueva cada 120 días. Las lecturas se guardan en orden cronológico, como llegarían desde el campo.
- **Alertas.** Se derivan de lecturas reales por debajo del umbral (85 %) o con batería menor a 15 % (15 %), y su severidad depende del déficit.
- **Riegos.** El 55 % de los eventos responde a una alerta de humedad entre 15 y 355 minutos después. El volumen es el caudal del sistema por la duración, afectado por un factor entre 0.55 y 1.
- **Cosechas.** Solo se cosechan campañas cerradas o con más del 70 % de avance. Los kilos salen del rendimiento de referencia del cultivo por el área de la parcela, con una variación de $\pm 25\%$.

Estas reglas tienen una consecuencia que conviene declarar: como el generador programa al menos un riego de respuesta para cada alerta de humedad, la consulta 5 devuelve 100 % de respuesta en todos los fondos. Con datos reales esa métrica sí mostraría diferencias. Para el experimento lo que importa es el costo de evaluar la subconsulta, que se analiza en la Sección 5.7.

Valores faltantes dentro de las series. En las lecturas hay dos tipos de nulos. Los estructurales aparecen cuando el sensor no mide esa variable: un sensor de pH no reporta humedad. Los de transmisión simulan paquetes perdidos: el 2 % de las humedades, el 1 % de las temperaturas, el 4 % de los pH, el 3 % de las conductividades y el 5 % de las baterías. También faltan el 8 % de los teléfonos de operarios, el 10 % de las humedades de producto y el 6 % de los precios de cosecha. La Tabla 26 muestra los porcentajes resultantes en el escenario de un millón.

Tipo de sensor	Lecturas	Humedad	Temp.	pH	CE	Batería
Multiparametro	297 981	2.0	1.0	4.0	3.1	5.0
Humedad	281 288	2.0	1.0	100.0	100.0	5.0
pH	75 542	100.0	1.1	4.0	100.0	5.1
Conductividad	74 844	2.0	1.0	100.0	3.1	5.0
Temperatura	36 206	100.0	1.0	100.0	100.0	5.0

Tabla 26: Porcentaje de valores nulos por variable y tipo de sensor (1M). Un 100 % indica un nulo estructural.

Nunca imputamos valores en la tabla base, porque el dato crudo debe conservarse tal como llegó. Las consultas filtran con `IS NOT NULL` o con `COUNT(*) FILTER`. Cuando un análisis necesita series continuas, la siguiente consulta rellena cada hueco con la última observación válida del mismo sensor. El conteo acumulado de valores no nulos forma grupos que empiezan en cada dato real, y `FIRST_VALUE` propaga ese dato dentro del grupo. Solo se aplica a sensores que miden humedad, para no inventar valores donde el nulo es estructural.

```

1 SELECT id_dispositivo, fecha_hora, humedad_pct,
2        FIRST_VALUE(humedad_pct) OVER (PARTITION BY id_dispositivo, grupo
3                                       ORDER BY fecha_hora) AS humedad_imputada
4 FROM (SELECT l.*,
5            COUNT(l.humedad_pct) OVER (PARTITION BY l.id_dispositivo
6                                       ORDER BY l.fecha_hora) AS grupo
7      FROM Lectura_Suelo l
8     JOIN Sensor s ON s.id_dispositivo = l.id_dispositivo
9    WHERE s.tipo_sensor IN ('Humedad', 'Multiparametro', 'Conductividad')) t;
```

4.4. Views

Implementamos tres vistas, pensadas para los perfiles que no deben escribir SQL complejo:

- **vw_estado_hidrico_zona:** última lectura de humedad de cada sensor con su zona, su cultivo vigente y un semáforo (*Deficit*, *Optimo*, *Exceso*). Usa `DISTINCT ON`, una extensión de PostgreSQL que devuelve la primera fila de cada grupo según el orden indicado, más eficiente que una subconsulta con `MAX`.
- **vw_produccion_campania:** kilos totales, kilos por hectárea e ingreso estimado por campaña. El `LEFT JOIN` hacia las cosechas conserva las campañas que aún no cosechan.
- **vw_alertas_pendientes:** bandeja de trabajo del operador de riego, con las alertas sin atender ordenadas por severidad y antigüedad.

```

1  -- =====
2  -- A. VISTAS
3  -- =====
4
5  -- A.1 Estado hidrico vigente de cada zona de manejo.
6  -- Compara la ultima lectura de humedad disponible con el rango optimo del
7  -- cultivo sembrado y clasifica la zona en deficit, optimo o exceso.
8  CREATE OR REPLACE VIEW vw_estado_hidrico_zona AS
9  WITH ultima AS (
10     SELECT DISTINCT ON (l.id_dispositivo)
11         l.id_dispositivo, l.fecha_hora, l.humedad_pct, l.conductividad_ds_m
12     FROM Lectura_Suelo l
13     WHERE l.humedad_pct IS NOT NULL
14     ORDER BY l.id_dispositivo, l.fecha_hora DESC
15 )
16 SELECT f.nombre           AS fundo,
17        p.codigo_parcela,
18        z.num_zona,
19        cu.nombre_comun     AS cultivo,
20        u.fecha_hora        AS ultima_medicion,
21        u.humedad_pct,
22        cu.humedad_optima_min,
23        cu.humedad_optima_max,
24        CASE WHEN u.humedad_pct < cu.humedad_optima_min THEN 'Deficit'
25              WHEN u.humedad_pct > cu.humedad_optima_max THEN 'Exceso'
26              ELSE 'Optimo' END AS diagnostico
27 FROM ultima u
28 JOIN Dispositivo d ON d.id_dispositivo = u.id_dispositivo
29 JOIN Zona_Manejo z ON z.id_parcela = d.id_parcela AND z.num_zona = d.num_zona
30 JOIN Parcela p ON p.id_parcela = z.id_parcela
31 JOIN Fundo f ON f.id_fundo = p.id_fundo
32 JOIN Campania ca ON ca.id_parcela = p.id_parcela AND ca.estado = 'En curso'
33 JOIN Cultivo cu ON cu.id_cultivo = ca.id_cultivo;
34
35 -- A.2 Resumen productivo por campania (kg totales, kg/ha e ingreso estimado).
36 CREATE OR REPLACE VIEW vw_produccion_campania AS
37 SELECT ca.id_campania,
38        f.nombre           AS fundo,
39        p.codigo_parcela,
40        cu.nombre_comun     AS cultivo,
41        ca.estado,
42        p.area_ha,
43        COALESCE(SUM(co.cantidad_kg), 0) AS kilos_totales,
44        ROUND(COALESCE(SUM(co.cantidad_kg), 0) / p.area_ha, 2) AS kilos_por_ha,
45        ROUND(COALESCE(SUM(co.cantidad_kg * co.precio_kg_soles), 0), 2) AS ingreso_soles
46 FROM Campania ca
47 JOIN Parcela p ON p.id_parcela = ca.id_parcela
48 JOIN Fundo f ON f.id_fundo = p.id_fundo
49 JOIN Cultivo cu ON cu.id_cultivo = ca.id_cultivo
50 LEFT JOIN Cosecha co ON co.id_campania = ca.id_campania
51 GROUP BY ca.id_campania, f.nombre, p.codigo_parcela, cu.nombre_comun, ca.estado, p.area_ha;
52
53 -- A.3 Bandeja de alertas sin atender, ordenada por severidad y antigüedad.
54 CREATE OR REPLACE VIEW vw_alertas_pendientes AS
55 SELECT a.id_alerta,
56        f.nombre           AS fundo,
57        p.codigo_parcela,
58        d.num_zona,
59        d.codigo_serie     AS sensor,
60        a.tipo,
61        a.severidad,
62        a.valor_registrado,
63        a.fecha_hora,
64        ROUND(EXTRACT(EPOCH FROM (TIMESTAMP '2026-09-20 00:00:00' - a.fecha_hora)) / 3600, 1)
65            AS horas_sin_atender
66 FROM Alerta a
67 JOIN Dispositivo d ON d.id_dispositivo = a.id_dispositivo
68 JOIN Parcela p ON p.id_parcela = d.id_parcela
69 JOIN Fundo f ON f.id_fundo = p.id_fundo

```

```

70 WHERE a.atendida = FALSE
71 ORDER BY CASE a.severidad WHEN 'Critica' THEN 1 WHEN 'Alta' THEN 2
72                WHEN 'Media' THEN 3 ELSE 4 END,
73        a.fecha_hora;

```

Las primeras filas de cada vista en el escenario de un millón:

fundo	codigo_parcela	num_zona	cultivo	ultima_medicion	humedad_pct	humedad_optima_min	humedad_optima_max	diagnostico
Las Lomas	P-001	1	Maiz amarillo	2026-09-15 04:21:00	63.29	55.00	75.00	Optimo
Valle Alto	P-001	1	Cafe Caturra	2026-09-15 22:34:00	52.12	60.00	80.00	Deficit
Los Olivares	P-001	1	Mango Kent	2026-09-14 03:50:00	46.06	50.00	70.00	Deficit
El Mirador	P-001	1	Maiz amarillo	2026-09-15 21:12:00	77.44	55.00	75.00	Exceso
Las Lomas	P-002	1	Cacao	2026-09-14 09:49:00	71.53	65.00	85.00	Optimo

Tabla 27: Primeras filas de `vw_estado_hidrico_zona`.

id_campania	fundo	codigo_parcela	cultivo	estado	area_ha	kilos_totales	kilos_por_ha	ingreso_soles
1	Las Lomas	P-001	Maiz amarillo	En curso	1.60	0	0.00	0.00
2	Valle Alto	P-001	Cafe Caturra	En curso	20.02	10857.79	542.35	143945.94
3	Los Olivares	P-001	Mango Kent	En curso	10.00	42704.18	4270.42	90240.19
4	El Mirador	P-001	Maiz amarillo	En curso	11.88	0	0.00	0.00
5	Las Lomas	P-002	Cacao	En curso	10.59	0	0.00	0.00

Tabla 28: Primeras filas de `vw_produccion_campania`.

id_alerta	fundo	codigo_parcela	num_zona	sensor	tipo	severidad	valor_registrado	fecha_hora	horas_sin_atender
116	La Huaca	P-009	1	SN-0400186	Humedad baja	Critica	33.71	2026-01-01 21:22:00	6266.6
239	San Jacinto	P-037	3	SN-0401088	Humedad baja	Critica	19.32	2026-01-02 23:36:00	6240.4
387	Los Olivares	P-001	2	SN-0400868	Humedad baja	Critica	26.45	2026-01-04 08:18:00	6207.7
537	El Mirador	P-009	3	SN-0401379	Humedad baja	Critica	44.00	2026-01-05 15:59:00	6176.0
733	Buenaventura	P-010	1	SN-0400093	Humedad baja	Critica	44.46	2026-01-07 10:46:00	6133.2

Tabla 29: Primeras filas de `vw_alertas_pendientes`.

4.5. Procedimientos Almacenados

- `sp_registrar_lectura`: único punto de entrada para la plataforma IoT. Verifica que el sensor exista y esté activo; si llega una lectura repetida en el mismo minuto, la ignora con `ON CONFLICT DO NOTHING` en lugar de abortar el lote.
- `sp_cerrar_campania`: calcula el rendimiento final, cambia el estado a *Cerrada* y deja el cambio en la bitácora a través del trigger de auditoría.
- `fn_balance_hidrico`: función que devuelve una tabla con el agua aplicada, el número de riegos y la humedad media por zona de una parcela en un intervalo.
- `fn_costo_insumos`: costo total de los insumos aplicados en una parcela entre dos fechas.
- `sp_registrar_dispositivo`: alta atómica de un equipo en la superclase y en su subclase (cobertura total de la jerarquía).

```

1  -- =====
2  -- B. PROCEDIMIENTOS ALMACENADOS Y FUNCIONES
3  -- =====
4
5  -- B.1 Registra una lectura validando que el sensor exista y este activo.

```

```

6 CREATE OR REPLACE PROCEDURE sp_registrar_lectura(
7     p_sensor INTEGER,
8     p_fecha  TIMESTAMP,
9     p_humedad NUMERIC,
10    p_temp    NUMERIC,
11    p_ph      NUMERIC DEFAULT NULL,
12    p_ce      NUMERIC DEFAULT NULL,
13    p_bateria INTEGER DEFAULT NULL)
14 LANGUAGE plpgsql AS $$
15 DECLARE
16     v_estado VARCHAR(15);
17 BEGIN
18     SELECT d.estado INTO v_estado
19     FROM Dispositivo d JOIN Sensor s ON s.id_dispositivo = d.id_dispositivo
20     WHERE d.id_dispositivo = p_sensor;
21
22     IF NOT FOUND THEN
23         RAISE EXCEPTION 'El dispositivo % no existe o no es un sensor', p_sensor;
24     END IF;
25     IF v_estado <> 'Activo' THEN
26         RAISE EXCEPTION 'El sensor % esta en estado % y no puede reportar', p_sensor, v_estado;
27     END IF;
28
29     INSERT INTO Lectura_Suelo(id_dispositivo, fecha_hora, humedad_pct, temperatura_c,
30                             ph, conductividad_ds_m, bateria_pct)
31     VALUES (p_sensor, date_trunc('minute', p_fecha), p_humedad, p_temp, p_ph, p_ce, p_bateria)
32     ON CONFLICT (id_dispositivo, fecha_hora) DO NOTHING;
33 END;
34 $$;
35
36 -- B.2 Cierra una campania, calcula su rendimiento y deja registro de auditoria.
37 CREATE OR REPLACE PROCEDURE sp_cerrar_campania(p_campania INTEGER)
38 LANGUAGE plpgsql AS $$
39 DECLARE
40     v_area NUMERIC;
41     v_kilos NUMERIC;
42 BEGIN
43     SELECT p.area_ha INTO v_area
44     FROM Campania ca JOIN Parcela p ON p.id_parcela = ca.id_parcela
45     WHERE ca.id_campania = p_campania;
46
47     IF v_area IS NULL THEN
48         RAISE EXCEPTION 'La campania % no existe', p_campania;
49     END IF;
50
51     SELECT COALESCE(SUM(cantidad_kg), 0) INTO v_kilos
52     FROM Cosecha WHERE id_campania = p_campania;
53
54     UPDATE Campania
55     SET estado = 'Cerrada',
56         rendimiento_kg_ha = ROUND(v_kilos / v_area, 2)
57     WHERE id_campania = p_campania;
58 END;
59 $$;
60
61 -- B.3 Balance hidrico de una parcela en un intervalo: agua aplicada por zona
62 -- y humedad media registrada en el mismo periodo.
63 CREATE OR REPLACE FUNCTION fn_balance_hidrico(
64     p_parcela INTEGER, p_desde TIMESTAMP, p_hasta TIMESTAMP)
65 RETURNS TABLE (num_zona SMALLINT, litros NUMERIC, eventos BIGINT, humedad_media NUMERIC)
66 LANGUAGE sql AS $$
67     SELECT z.num_zona,
68            COALESCE(SUM(er.volumen_litros), 0) AS litros,
69            COUNT(er.id_evento) AS eventos,
70            (SELECT ROUND(AVG(l.humedad_pct), 2)
71             FROM Lectura_Suelo l
72             JOIN Dispositivo d2 ON d2.id_dispositivo = l.id_dispositivo
73             WHERE d2.id_parcela = z.id_parcela
74             AND d2.num_zona = z.num_zona

```

```

75         AND l.fecha_hora BETWEEN p_desde AND p_hasta) AS humedad_media
76 FROM Zona_Manejo z
77 LEFT JOIN Evento_Riego er
78     ON er.id_parcela = z.id_parcela AND er.num_zona = z.num_zona
79     AND er.fecha_hora_inicio BETWEEN p_desde AND p_hasta
80 WHERE z.id_parcela = p_parcela
81 GROUP BY z.id_parcela, z.num_zona
82 ORDER BY z.num_zona;
83 $$;
84
85 -- B.4 Costo de insumos aplicados a una parcela en un rango de fechas.
86 CREATE OR REPLACE FUNCTION fn_costo_insumos(
87     p_parcela INTEGER, p_desde DATE, p_hasta DATE)
88 RETURNS NUMERIC
89 LANGUAGE sql AS $$
90     SELECT COALESCE(ROUND(SUM(ap.dosis * i.costo_unitario), 2), 0)
91     FROM Aplicacion_Insumo ap
92     JOIN Insumo i ON i.id_insumo = ap.id_insumo
93     WHERE ap.id_parcela = p_parcela
94           AND ap.fecha_hora::date BETWEEN p_desde AND p_hasta;
95 $$;

```

```

1  -- D.2 Cobertura total: el alta de un equipo crea la superclase y su subclase
2  --     dentro de la misma transaccion; si una de las dos falla, no queda nada.
3  CREATE OR REPLACE PROCEDURE sp_registrar_dispositivo(
4      p_clase      VARCHAR,
5      p_serie      VARCHAR,
6      p_modelo     VARCHAR,
7      p_fabricante VARCHAR,
8      p_parcela    INTEGER,
9      p_zona       INTEGER,
10     p_tipo_sensor VARCHAR DEFAULT NULL,
11     p_profundidad INTEGER DEFAULT NULL,
12     p_sistema     INTEGER DEFAULT NULL,
13     p_caudal_lps  NUMERIC DEFAULT NULL,
14     p_valvula     VARCHAR DEFAULT NULL)
15 LANGUAGE plpgsql AS $$
16 DECLARE
17     v_id INTEGER;
18 BEGIN
19     IF p_clase NOT IN ('Sensor', 'Controlador') THEN
20         RAISE EXCEPTION 'Clase de dispositivo no valida: %', p_clase;
21     END IF;
22
23     INSERT INTO Dispositivo(codigo_serie, modelo, fabricante, fecha_instalacion,
24                             id_parcela, num_zona)
25     VALUES (p_serie, p_modelo, p_fabricante, CURRENT_DATE, p_parcela, p_zona)
26     RETURNING id_dispositivo INTO v_id;
27
28     IF p_clase = 'Sensor' THEN
29         INSERT INTO Sensor(id_dispositivo, tipo_sensor, profundidad_cm)
30         VALUES (v_id, p_tipo_sensor, p_profundidad);
31     ELSE
32         INSERT INTO Controlador(id_dispositivo, id_sistema, caudal_max_lps, tipo_valvula)
33         VALUES (v_id, p_sistema, p_caudal_lps, p_valvula);
34     END IF;
35     RAISE NOTICE 'Dispositivo % registrado como %', v_id, p_clase;
36 END;
37 $$;

```

4.6. Triggers

Las validaciones y los cálculos sobre la propia fila se hacen en triggers **BEFORE**, porque pueden modificar **NEW** o abortar antes de escribir. Los efectos sobre otras tablas se hacen en triggers **AFTER**, cuando la fila ya pasó todas las restricciones.

Trigger	Momento	Qué hace
trg_alerta_lectura	AFTER INSERT en lecturas	Crea alertas de humedad baja (con severidad según el déficit) y de batería baja.
trg_evento_riego	BEFORE INSERT/UPDATE en riegos	Rechaza riegos con el sistema de otra parcela y calcula el volumen si no se informó.
trg_rendimiento	AFTER INSERT/UPDATE/DELETE en cosechas	Recalcula el rendimiento por hectárea de la campaña.
trg_campania_sin_solape	BEFORE INSERT/UPDATE en campañas	Impide dos campañas superpuestas en la misma parcela.
trg_auditoria_campania	AFTER UPDATE en campañas	Registra cada cambio de estado con usuario y fecha.
trg_sensor_exclusivo, trg_controlador_exclusivo	BEFORE INSERT/UPDATE en subclases	Garantizan que un dispositivo pertenezca a una sola sub-clase.

Tabla 30: Triggers implementados.

```

1  -- =====
2  -- C. TRIGGERS
3  -- =====
4
5  -- C.1 Genera alertas a partir de cada lectura nueva: humedad por debajo del
6  --     minimo del cultivo vigente o bateria por debajo de 15 %.
7  CREATE OR REPLACE FUNCTION trg_fn_alerta_lectura() RETURNS TRIGGER
8  LANGUAGE plpgsql AS $$
9  DECLARE
10     v_min NUMERIC;
11  BEGIN
12     IF NEW.humedad_pct IS NOT NULL THEN
13         SELECT cu.humedad_optima_min INTO v_min
14         FROM Dispositivo d
15         JOIN Campania ca ON ca.id_parcela = d.id_parcela AND ca.estado = 'En curso'
16         JOIN Cultivo cu ON cu.id_cultivo = ca.id_cultivo
17         WHERE d.id_dispositivo = NEW.id_dispositivo;
18
19         IF v_min IS NOT NULL AND NEW.humedad_pct < v_min THEN
20             INSERT INTO Alerta(id_dispositivo, fecha_hora, tipo, severidad, valor_registrado)
21             VALUES (NEW.id_dispositivo, NEW.fecha_hora, 'Humedad baja',
22                     CASE WHEN NEW.humedad_pct < v_min - 20 THEN 'Critica'
23                          WHEN NEW.humedad_pct < v_min - 10 THEN 'Alta'
24                          ELSE 'Media' END,
25                     NEW.humedad_pct)
26             ON CONFLICT (id_dispositivo, fecha_hora, tipo) DO NOTHING;
27         END IF;
28     END IF;
29
30     IF NEW.bateria_pct IS NOT NULL AND NEW.bateria_pct < 15 THEN
31         INSERT INTO Alerta(id_dispositivo, fecha_hora, tipo, severidad, valor_registrado)
32         VALUES (NEW.id_dispositivo, NEW.fecha_hora, 'Bateria baja',
33                 CASE WHEN NEW.bateria_pct < 8 THEN 'Alta' ELSE 'Media' END,
34                 NEW.bateria_pct)
35         ON CONFLICT (id_dispositivo, fecha_hora, tipo) DO NOTHING;
36     END IF;
37     RETURN NEW;
38 END;

```

```

39 $$;
40
41 DROP TRIGGER IF EXISTS trg_alerta_lectura ON Lectura_Suelo;
42 CREATE TRIGGER trg_alerta_lectura
43 AFTER INSERT ON Lectura_Suelo
44 FOR EACH ROW EXECUTE FUNCTION trg_fn_alerta_lectura();
45
46 -- C.2 Verifica que la zona regada pertenezca a la parcela que abastece el
47 -- sistema y, si el controlador no reporta volumen, lo calcula con el caudal.
48 CREATE OR REPLACE FUNCTION trg_fn_evento_riego() RETURNS TRIGGER
49 LANGUAGE plpgsql AS $$
50 DECLARE
51     v_parcela INTEGER;
52     v_caudal NUMERIC;
53 BEGIN
54     SELECT id_parcela, caudal_nominal_lph INTO v_parcela, v_caudal
55     FROM Sistema_Riego WHERE id_sistema = NEW.id_sistema;
56
57     IF v_parcela IS DISTINCT FROM NEW.id_parcela THEN
58         RAISE EXCEPTION 'El sistema % no abastece a la parcela %',
59             NEW.id_sistema, NEW.id_parcela;
60     END IF;
61     IF NEW.volumen_litros IS NULL OR NEW.volumen_litros = 0 THEN
62         NEW.volumen_litros := ROUND(v_caudal * NEW.duracion_min / 60.0, 2);
63     END IF;
64     RETURN NEW;
65 END;
66 $$;
67
68 DROP TRIGGER IF EXISTS trg_evento_riego ON Evento_Riego;
69 CREATE TRIGGER trg_evento_riego
70 BEFORE INSERT OR UPDATE ON Evento_Riego
71 FOR EACH ROW EXECUTE FUNCTION trg_fn_evento_riego();
72
73 -- C.3 Mantiene actualizado el atributo derivado rendimiento_kg_ha.
74 CREATE OR REPLACE FUNCTION trg_fn_rendimiento() RETURNS TRIGGER
75 LANGUAGE plpgsql AS $$
76 DECLARE
77     v_camp INTEGER := COALESCE(NEW.id_campania, OLD.id_campania);
78     v_area NUMERIC;
79 BEGIN
80     SELECT p.area_ha INTO v_area
81     FROM Campania ca JOIN Parcela p ON p.id_parcela = ca.id_parcela
82     WHERE ca.id_campania = v_camp;
83
84     UPDATE Campania
85     SET rendimiento_kg_ha = ROUND(
86         (SELECT COALESCE(SUM(cantidad_kg), 0) FROM Cosecha WHERE id_campania = v_camp)
87         / NULLIF(v_area, 0), 2)
88     WHERE id_campania = v_camp;
89     RETURN NULL;
90 END;
91 $$;
92
93 DROP TRIGGER IF EXISTS trg_rendimiento ON Cosecha;
94 CREATE TRIGGER trg_rendimiento
95 AFTER INSERT OR UPDATE OR DELETE ON Cosecha
96 FOR EACH ROW EXECUTE FUNCTION trg_fn_rendimiento();
97
98 -- C.4 Impide que una parcela tenga dos campanias con periodos superpuestos.
99 CREATE OR REPLACE FUNCTION trg_fn_campania_sin_solape() RETURNS TRIGGER
100 LANGUAGE plpgsql AS $$
101 BEGIN
102     IF EXISTS (
103         SELECT 1 FROM Campania c
104         WHERE c.id_parcela = NEW.id_parcela
105             AND c.id_campania <> COALESCE(NEW.id_campania, -1)
106             AND NEW.fecha_siembra < c.fecha_fin_estimada
107             AND NEW.fecha_fin_estimada > c.fecha_siembra

```

```

108     ) THEN
109         RAISE EXCEPTION 'La parcela % ya tiene una campania activa en ese periodo',
110             NEW.id_parcela;
111     END IF;
112     RETURN NEW;
113 END;
114 $$;
115
116 DROP TRIGGER IF EXISTS trg_campania_sin_solape ON Campania;
117 CREATE TRIGGER trg_campania_sin_solape
118 BEFORE INSERT OR UPDATE ON Campania
119 FOR EACH ROW EXECUTE FUNCTION trg_fn_campania_sin_solape();
120
121 -- C.5 Bitacora de cambios de estado de una campania.
122 CREATE OR REPLACE FUNCTION trg_fn_auditoria_campania() RETURNS TRIGGER
123 LANGUAGE plpgsql AS $$
124 BEGIN
125     IF NEW.estado IS DISTINCT FROM OLD.estado THEN
126         INSERT INTO Auditoria_Campania(id_campania, estado_anterior, estado_nuevo)
127         VALUES (OLD.id_campania, OLD.estado, NEW.estado);
128     END IF;
129     RETURN NULL;
130 END;
131 $$;
132
133 DROP TRIGGER IF EXISTS trg_auditoria_campania ON Campania;
134 CREATE TRIGGER trg_auditoria_campania
135 AFTER UPDATE ON Campania
136 FOR EACH ROW EXECUTE FUNCTION trg_fn_auditoria_campania();

```

```

1  -- =====
2  -- D. INTEGRIDAD DE LA JERARQUIA IS-A (cobertura total y disjunta)
3  -- =====
4
5  -- D.1 Disjuncion: un dispositivo no puede ser sensor y controlador a la vez.
6  CREATE OR REPLACE FUNCTION trg_fn_subclase_exclusiva() RETURNS TRIGGER
7  LANGUAGE plpgsql AS $$
8  BEGIN
9      IF TG_TABLE_NAME = 'sensor' AND EXISTS
10         (SELECT 1 FROM Controlador WHERE id_dispositivo = NEW.id_dispositivo) THEN
11         RAISE EXCEPTION 'El dispositivo % ya figura como controlador', NEW.id_dispositivo;
12     ELSIF TG_TABLE_NAME = 'controlador' AND EXISTS
13         (SELECT 1 FROM Sensor WHERE id_dispositivo = NEW.id_dispositivo) THEN
14         RAISE EXCEPTION 'El dispositivo % ya figura como sensor', NEW.id_dispositivo;
15     END IF;
16     RETURN NEW;
17 END;
18 $$;
19
20 DROP TRIGGER IF EXISTS trg_sensor_exclusivo ON Sensor;
21 CREATE TRIGGER trg_sensor_exclusivo
22 BEFORE INSERT OR UPDATE ON Sensor
23 FOR EACH ROW EXECUTE FUNCTION trg_fn_subclase_exclusiva();
24
25 DROP TRIGGER IF EXISTS trg_controlador_exclusivo ON Controlador;
26 CREATE TRIGGER trg_controlador_exclusivo
27 BEFORE INSERT OR UPDATE ON Controlador
28 FOR EACH ROW EXECUTE FUNCTION trg_fn_subclase_exclusiva();

```

4.6.1. Pruebas de funcionamiento

El script `06_pruebas.sql` ejercita cada procedimiento, función y trigger dentro de una transacción que se revierte al final, de modo que la base queda intacta. La salida siguiente se obtuvo sobre la base de un millón de registros. Los errores que aparecen son los esperados: cada uno demuestra que una regla bloqueó una operación inválida.

```

1  --- P1. sp_registrar_dispositivo crea la superclase y la subclase
2  NOTICE: Dispositivo 2834 registrado como Sensor
3  id_dispositivo | modelo | tipo_sensor | profundidad_cm
4  -----+-----+-----+-----
5  2834 | TEROS 12 | Conductividad | 30
6
7  --- P2. Si falla la subclase no queda un dispositivo huérfano
8  ERROR: null value in column "id_sistema" of relation "controlador" violates not-null constraint
9  DETAIL: Failing row contains (2835, null, null, null).
10 CONTEXT: SQL statement "INSERT INTO Controlador(id_dispositivo, id_sistema, caudal_max_lps, tipo_valvula)
11 VALUES (v_id, p_sistema, p_caudal_lps, p_valvula)"
12 PL/pgSQL function sp_registrar_dispositivo(character varying,character varying,character varying,integer,integer,character varying,
13 integer,integer,numeric,character varying) line 18 at SQL statement
14 filas_con_serie_prueba_02
15 -----+-----+-----+-----
16 0
17
18 --- P3. Disjunción: el sensor no puede registrarse además como controlador
19 ERROR: El dispositivo 2834 ya figura como sensor
20 CONTEXT: PL/pgSQL function trg_fn_subclase_exclusiva() line 8 at RAISE
21
22 --- P4. Una lectura con humedad crítica y batería baja genera dos alertas
23 tipo | severidad | valor_registrado
24 -----+-----+-----
25 Bateria baja | Alta | 6.00
26 Humedad baja | Crítica | 10.00
27
28 --- P5. Una lectura repetida en el mismo minuto se ignora
29 lecturas_del_sensor | humedad_guardada
30 -----+-----+-----
31 1 | 10.00
32
33 --- P6. Un riego con el sistema de otra parcela se rechaza
34 ERROR: El sistema 2 no abastece a la parcela 1
35 CONTEXT: PL/pgSQL function trg_fn_evento_riego() line 10 at RAISE
36
37 --- P7. Si el volumen llega en cero se calcula con el caudal nominal
38 duracion_min | caudal_nominal_lph | volumen_litros
39 -----+-----+-----
40 30 | 163858.65 | 81929.33
41
42 --- P8. No se permite una campaña superpuesta con la vigente
43 ERROR: La parcela 1 ya tiene una campaña activa en ese periodo
44 CONTEXT: PL/pgSQL function trg_fn_campania_sin_solape() line 10 at RAISE
45
46 --- P9. Una cosecha nueva actualiza el rendimiento derivado
47 rendimiento_antes
48 -----+-----+-----
49 0.00
50
51 rendimiento_despues | area_ha
52 -----+-----+-----
53 3125.00 | 1.60
54
55 --- P10. sp_cerrar_campania cierra la campaña y deja rastro en la auditoria
56 estado | rendimiento_kg_ha
57 -----+-----+-----
58 Cerrada | 3125.00
59
60 estado_anterior | estado_nuevo | usuario_bd
61 -----+-----+-----
62 En curso | Cerrada | postgres
63
64 --- P11. Una alerta no puede marcarse como atendida sin fecha de atención
65 ERROR: new row for relation "alerta" violates check constraint "ck_alerta_aten"
66 DETAIL: Failing row contains (2, 2016, 2026-01-01 00:17:00, Humedad baja, Media, 53.83, t, null, null).
67
68 --- P12. Funciones de consulta
69 num_zona | litros | eventos | humedad_media
70 -----+-----+-----+-----
71 1 | 2108178.73 | 5 | 65.97
72 2 | 1112000.75 | 5 | 64.61
73 3 | 4667226.56 | 12 | 64.38
74 4 | 2319403.94 | 5 | 64.18
75 5 | 1105978.77 | 2 | 65.47
76 6 | 682443.27 | 2 |
77
78 costo_insumos_soles
79 -----+-----+-----
80 73579.21

```

5. Optimización y Experimentación

5.1. Consultas SQL para el experimento

5.1.1. Descripción de las 5 consultas intermedias seleccionadas (no triviales)

Consulta 1. Parcelas con estrés hídrico. Para el agrónomo. Calcula la humedad promedio del último mes en cada parcela y devuelve las que están por debajo del mínimo de su cultivo vigente. Une ocho tablas, filtra un rango de fechas sobre la tabla más grande y compara el agregado contra un umbral en el HAVING.

Consulta 2. Eficiencia hídrica por campaña. Para la gerencia. Suma los litros aplicados durante cada campaña y los divide entre los kilos cosechados, y devuelve solo las campañas que consumen más agua que el promedio. Usa dos CTE, una unión por rango temporal (un riego pertenece a la campaña si ocurrió entre su siembra y su fin) y una subconsulta escalar sobre la propia CTE. En el escenario de mil registros no devuelve filas, porque hay muy pocas campañas cosechadas que coincidan con la ventana de riegos.

Consulta 3. Sensores con lecturas anómalas. Para el técnico de mantenimiento. Calcula, por sensor, qué proporción de sus lecturas cae fuera del rango óptimo y devuelve los que superan la proporción de toda la red. Un sensor con muchas anomalías suele estar descalibrado. Usa una CTE, conteo condicional con FILTER y una subconsulta sobre la CTE.

Consulta 4. Ranking de rendimiento por fundo. Para la gerencia. Calcula los kilos por hectárea de cada campaña (solo primera y segunda categoría), les asigna un puesto dentro de su fundo con RANK() OVER (PARTITION BY ...) y muestra el promedio del fundo con otra función de ventana.

Consulta 5. Respuesta de riego ante alertas. Para la jefatura de operaciones. Mide qué porcentaje de las alertas de humedad de severidad alta o crítica recibió un riego en su zona dentro de las seis horas siguientes. Usa una subconsulta correlacionada EXISTS con una ventana temporal que depende de cada fila externa.

5.1.2. Implementación de consultas en SQL

Consulta 1

```

1  -- Parcelas con estres hidrico: la humedad media del ultimo mes esta por
2  -- debajo del minimo optimo del cultivo que tienen sembrado.
3  SELECT f.nombre                               AS fundo,
4         p.codigo_parcela,
5         cu.nombre_comun                        AS cultivo,
6         ROUND(AVG(l.humedad_pct), 2)          AS humedad_promedio,
7         MIN(cu.humedad_optima_min)            AS umbral_minimo,
8         COUNT(*)                              AS lecturas_evaluadas
9  FROM Lectura_Suelo l
10 JOIN Sensor      s ON s.id_dispositivo = l.id_dispositivo
11 JOIN Dispositivo d ON d.id_dispositivo = s.id_dispositivo
12 JOIN Zona_Manejo z ON z.id_parcela = d.id_parcela AND z.num_zona = d.num_zona
13 JOIN Parcela     p ON p.id_parcela = z.id_parcela
14 JOIN Fundo       f ON f.id_fundo = p.id_fundo
15 JOIN Campaña     ca ON ca.id_parcela = p.id_parcela AND ca.estado = 'En curso'
16 JOIN Cultivo     cu ON cu.id_cultivo = ca.id_cultivo
17 WHERE l.fecha_hora >= TIMESTAMP '2026-08-16 00:00:00'
18       AND l.fecha_hora < TIMESTAMP '2026-09-16 00:00:00'
19       AND l.humedad_pct IS NOT NULL
20 GROUP BY f.nombre, p.codigo_parcela, cu.nombre_comun
21 HAVING AVG(l.humedad_pct) < MIN(cu.humedad_optima_min)
22 ORDER BY humedad_promedio ASC;

```

Consulta 2

```

1  -- Eficiencia hidrica por campania (litros aplicados por kilogramo cosechado),
2  -- restringida a las campanias que consumen mas agua que el promedio general.
3  WITH riego AS (
4      SELECT ca.id_campania,
5             SUM(er.volumen_litros) AS litros_aplicados
6      FROM Evento_Riego er
7      JOIN Campania ca
8           ON ca.id_parcela = er.id_parcela
9           AND er.fecha_hora_inicio >= ca.fecha_siembra
10          AND er.fecha_hora_inicio < ca.fecha_fin_estimada
11      GROUP BY ca.id_campania
12  ),
13  produccion AS (
14      SELECT id_campania, SUM(cantidad_kg) AS kilos
15      FROM Cosecha
16      GROUP BY id_campania
17  )
18  SELECT f.nombre                                AS fundo,
19         cu.nombre_comun                         AS cultivo,
20         r.id_campania,
21         ROUND(r.litros_aplicados, 2)            AS litros,
22         ROUND(pr.kilos, 2)                     AS kilos,
23         ROUND(r.litros_aplicados / pr.kilos, 2) AS litros_por_kilo
24  FROM riego r
25  JOIN produccion pr ON pr.id_campania = r.id_campania
26  JOIN Campania ca ON ca.id_campania = r.id_campania
27  JOIN Parcela p ON p.id_parcela = ca.id_parcela
28  JOIN Fundo f ON f.id_fundo = p.id_fundo
29  JOIN Cultivo cu ON cu.id_cultivo = ca.id_cultivo
30  WHERE r.litros_aplicados > (SELECT AVG(litros_aplicados) FROM riego)
31  ORDER BY litros_por_kilo DESC;

```

Consulta 3

```

1  -- Sensores cuya proporcion de lecturas fuera del rango optimo del cultivo
2  -- supera la proporcion promedio de toda la red.
3  WITH medidas AS (
4      SELECT l.id_dispositivo,
5             COUNT(*) AS total,
6             COUNT(*) FILTER (
7                 WHERE l.humedad_pct < cu.humedad_optima_min
8                 OR l.humedad_pct > cu.humedad_optima_max) AS anomalas
9      FROM Lectura_Suelo l
10     JOIN Dispositivo d ON d.id_dispositivo = l.id_dispositivo
11     JOIN Campania ca ON ca.id_parcela = d.id_parcela AND ca.estado = 'En curso'
12     JOIN Cultivo cu ON cu.id_cultivo = ca.id_cultivo
13     WHERE l.humedad_pct IS NOT NULL
14     GROUP BY l.id_dispositivo
15  )
16  SELECT d.codigo_serie,
17         s.tipo_sensor,
18         p.codigo_parcela,
19         m.total,
20         m.anomalas,
21         ROUND(100.0 * m.anomalas / m.total, 2) AS pct_anomalas
22  FROM medidas m
23  JOIN Sensor s ON s.id_dispositivo = m.id_dispositivo
24  JOIN Dispositivo d ON d.id_dispositivo = m.id_dispositivo
25  JOIN Parcela p ON p.id_parcela = d.id_parcela
26  WHERE m.total >= 5
27         AND 1.0 * m.anomalas / m.total >
28         (SELECT SUM(anomalas)::numeric / NULLIF(SUM(total), 0) FROM medidas)
29  ORDER BY pct_anomalas DESC, m.total DESC;

```

Consulta 4

```

1  -- Ranking de rendimiento (kg/ha) por fundo usando funcion de ventana,

```

```

2  -- considerando solo la produccion de primera y segunda categoria.
3  WITH rendimiento AS (
4      SELECT f.id_fundo,
5             f.nombre          AS fundo,
6             cu.nombre_comun   AS cultivo,
7             ca.id_campania,
8             p.area_ha,
9             SUM(co.cantidad_kg) AS kilos
10     FROM Cosecha co
11     JOIN Campania ca ON ca.id_campania = co.id_campania
12     JOIN Parcela p  ON p.id_parcela   = ca.id_parcela
13     JOIN Fondo f    ON f.id_fundo     = p.id_fundo
14     JOIN Cultivo cu  ON cu.id_cultivo  = ca.id_cultivo
15     WHERE co.categoria IN ('Primera','Segunda')
16     GROUP BY f.id_fundo, f.nombre, cu.nombre_comun, ca.id_campania, p.area_ha
17 )
18 SELECT fundo,
19        cultivo,
20        id_campania,
21        ROUND(kilos / area_ha, 2) AS kg_por_ha,
22        RANK() OVER (PARTITION BY id_fundo ORDER BY kilos / area_ha DESC) AS puesto,
23        ROUND(AVG(kilos / area_ha) OVER (PARTITION BY id_fundo), 2) AS promedio_fundo
24 FROM rendimiento
25 ORDER BY fundo, puesto;

```

Consulta 5

```

1  -- Tiempo de respuesta operativa: porcentaje de alertas de humedad baja que
2  -- recibieron riego en la zona afectada dentro de las 6 horas siguientes.
3  SELECT f.nombre AS fundo,
4         COUNT(*) AS alertas,
5         COUNT(*) FILTER (WHERE EXISTS (
6             SELECT 1
7             FROM Evento_Riego er
8             WHERE er.id_parcela = d.id_parcela
9                   AND er.num_zona = d.num_zona
10                  AND er.fecha_hora_inicio > a.fecha_hora
11                  AND er.fecha_hora_inicio <= a.fecha_hora + INTERVAL '6 hours'
12         )) AS atendidas_con_riego,
13         ROUND(100.0 * COUNT(*) FILTER (WHERE EXISTS (
14             SELECT 1
15             FROM Evento_Riego er
16             WHERE er.id_parcela = d.id_parcela
17                   AND er.num_zona = d.num_zona
18                  AND er.fecha_hora_inicio > a.fecha_hora
19                  AND er.fecha_hora_inicio <= a.fecha_hora + INTERVAL '6 hours'
20         )) / COUNT(*), 2) AS pct_respuesta
21 FROM Alerta a
22 JOIN Dispositivo d ON d.id_dispositivo = a.id_dispositivo
23 JOIN Parcela p ON p.id_parcela = d.id_parcela
24 JOIN Fondo f ON f.id_fundo = p.id_fundo
25 WHERE a.tipo = 'Humedad baja'
26        AND a.severidad IN ('Alta','Critica')
27 GROUP BY f.nombre
28 HAVING COUNT(*) > 3
29 ORDER BY pct_respuesta ASC;

```

5.2. Metodología del experimento

Escenarios. Cuatro bases independientes con 1 000, 10 000, 100 000 y 1 000 000 de registros en total, generadas con la misma lógica y una semilla fija por escenario.

Condiciones. En la condición *sin índices* se eliminan los nueve índices de optimización y además se desactivan en la sesión los accesos por índice (`enable_indexscan`, `enable_bitmapscan` y `enable_indexonlyscan` en OFF). Así el planificador tampoco puede usar los índices de las claves primarias y

únicas, que PostgreSQL crea siempre. En la condición *con índices* se crean los nueve índices y se dejan activos todos los métodos de acceso para que el planificador elija.

Preparación. Antes de cada bloque (escenario y condición) se ejecuta `VACUUM (FULL, ANALYZE)`, que reescribe las tablas sin tuplas muertas y actualiza las estadísticas. Antes de cada iteración se ejecuta `VACUUM ANALYZE`.

Medición. Cada consulta se ejecuta cinco veces con `EXPLAIN (ANALYZE, BUFFERS)` y se registra el *Execution Time*, que es el tiempo de ejecución en el servidor, sin la planificación ni la transferencia al cliente. Se reportan el promedio y la desviación estándar muestral. No se vacía la caché del sistema operativo entre iteraciones; si la primera fuera más lenta por lecturas en disco, se vería en la desviación.

Experimentos adicionales en 1M. Para responder qué pasa “cuando uno usa un índice u otro”, comparamos cuatro alternativas de índice para la consulta 1 y cuatro para la consulta 5, midiendo tiempo de consulta, tamaño y tiempo de construcción. Para medir el costo de mantener índices, ejecutamos tres operaciones de escritura con y sin los índices de optimización, cada una dentro de una transacción revertida y con los triggers de usuario desactivados para aislar el costo de los índices. Por último, comparamos la consulta 5 original con dos reescrituras.

Todo el procedimiento está automatizado en `experimento.py`, que guarda los resultados en JSON y CSV y los planes de ejecución en texto.

5.3. Optimización de consultas

Antes de proponer índices revisamos los que ya existen. PostgreSQL crea un índice B+ tree por cada clave primaria y cada restricción `UNIQUE`, pero no por las claves foráneas. Entre los existentes, tres resultan relevantes para las consultas: `pk_lectura` sobre (`id_dispositivo`, `fecha_hora`), `uq_evento` sobre (`id_parcela`, `num_zona`, `fecha_hora_inicio`) y `uq_campania` sobre (`id_parcela`, `fecha_siembra`). Crear otro índice con las mismas columnas solo duplicaría el costo de escritura, así que ninguno de los nueve índices propuestos repite uno existente.

5.3.1. Plan de índices para Consulta 1

El filtro por fecha selecciona alrededor de la décima parte de las lecturas. `pk_lectura` no sirve para ese filtro porque su primera columna es el sensor, no la fecha. Proponemos un B+ tree sobre `fecha_hora` que incluye (`INCLUDE`) el sensor y la humedad: así la consulta obtiene todo lo que necesita del índice sin visitar la tabla (*index only scan*). Agregamos un índice parcial sobre las campañas en curso y uno sobre la zona de cada dispositivo para las uniones.

```
1  -- ---- Consulta 1: filtro por rango de fechas sobre la tabla de lecturas
2  CREATE INDEX IF NOT EXISTS idx_lectura_fecha
3      ON Lectura_Suelo USING btree (fecha_hora) INCLUDE (id_dispositivo, humedad_pct);
4  CREATE INDEX IF NOT EXISTS idx_campania_vigente
5      ON Campania USING btree (id_parcela) WHERE estado = 'En curso';
6  CREATE INDEX IF NOT EXISTS idx_dispositivo_zona
7      ON Dispositivo USING btree (id_parcela, num_zona);
```

5.3.2. Plan de índices para Consulta 2

La unión entre riegos y campañas busca los riegos de una parcela dentro de un rango de fechas. Un B+ tree sobre (`id_parcela`, `fecha_hora_inicio`) permite ubicar esos riegos sin recorrer la tabla. Para la agregación de cosechas por campaña, que solo usa igualdad, probamos un índice hash.

```

1  -- ---- Consulta 2: union por rango entre eventos de riego y campanias
2  CREATE INDEX IF NOT EXISTS idx_evento_parcela_fecha
3      ON Evento_Riego USING btree (id_parcela, fecha_hora_inicio);
4  CREATE INDEX IF NOT EXISTS idx_cosecha_campania
5      ON Cosecha USING hash (id_campania);

```

5.3.3. Plan de índices para Consulta 3

La consulta agrupa todas las lecturas por sensor. Un B+ tree sobre el sensor que incluye la humedad permitiría recorrer las lecturas ya ordenadas por sensor sin tocar la tabla. Advertimos desde el diseño que el beneficio es dudoso: cuando una consulta necesita casi todas las filas, leer la tabla en orden físico suele ser más barato que recorrer un índice.

```

1  -- ---- Consulta 3: agregacion por sensor sobre toda la serie
2  CREATE INDEX IF NOT EXISTS idx_lectura_disp_humedad
3      ON Lectura_Suelo USING btree (id_dispositivo) INCLUDE (humedad_pct);

```

5.3.4. Plan de índices para Consulta 4

Un índice sobre (categoria, id_campania) que incluye los kilos cubre el filtro por categoría y la suma. Otro sobre el cultivo de cada campaña apoya la unión con Cultivo. Las parcelas por fundo ya están cubiertas por uq_parcela_codigo, cuya primera columna es id_fundo.

```

1  -- ---- Consulta 4: ranking de rendimiento por fundo
2  CREATE INDEX IF NOT EXISTS idx_cosecha_categoria
3      ON Cosecha USING btree (categoria, id_campania) INCLUDE (cantidad_kg);
4  CREATE INDEX IF NOT EXISTS idx_campania_cultivo
5      ON Campania USING btree (id_cultivo);

```

5.3.5. Plan de índices para Consulta 5

Por cada alerta, la subconsulta busca riegos de la misma zona dentro de un rango de seis horas. El índice ideal es un B+ tree sobre (id_parcela, num_zona, fecha_hora_inicio): las dos igualdades fijan la zona y el rango se recorre en orden. En la primera versión del proyecto propusimos ese índice, hasta notar que uq_evento ya lo proporciona con las mismas columnas y en el mismo orden. El único índice nuevo es uno parcial sobre las alertas de humedad baja, que cubre el filtro externo por tipo y severidad.

```

1  -- ---- Consulta 5: alertas criticas de humedad (la subconsulta correlacionada
2  --      sobre Evento_Riego usa el indice de la restricción uq_evento)
3  CREATE INDEX IF NOT EXISTS idx_alerta_humedad_baja
4      ON Alerta USING btree (severidad, id_dispositivo, fecha_hora)
5      WHERE tipo = 'Humedad baja';

```

5.4. Plataforma de Pruebas

Componente	Especificación
Procesador	Intel(R) Xeon(R) Processor @ 2.10GHz
Núcleos disponibles	1
Memoria RAM	3.9 GB
Sistema operativo	Ubuntu 24.04.4 LTS
Gestor de base de datos	PostgreSQL 16.15 (Ubuntu 16.15-0ubuntu0.24.04.1)
shared_buffers / work_mem	128MB / 4MB
effective_cache_size	4GB
max_parallel_workers_per_gather	2
random_page_cost	4
Cliente	Python 3.12.3, psycogp2 2.9.13

Tabla 31: Plataforma de pruebas.

La máquina tiene un solo núcleo virtual. PostgreSQL igual lanza procesos paralelos (*Parallel Seq Scan* con dos trabajadores), pero comparten el mismo núcleo, así que el paralelismo no aporta aceleración real. En un equipo con varios núcleos los recorridos secuenciales serían más rápidos, y la ventaja relativa de los índices en las consultas 1 y 2 probablemente sería menor. Los tiempos absolutos dependen de este equipo; lo comparable son las proporciones.

5.5. Medición de tiempos

5.5.1. Sin índices

Escenario	It. 1	It. 2	It. 3	It. 4	It. 5	Promedio	Desv. est.
1K	0.485	0.294	0.280	0.268	0.296	0.325	0.090
10K	1.385	1.134	1.163	1.243	1.155	1.216	0.103
100K	9.567	9.513	9.542	9.651	10.23	9.701	0.300
1M	122.57	121.73	121.86	122.13	121.71	122.00	0.360

Tabla 32: Consulta 1 sin índices: tiempo de ejecución por iteración (ms).

Escenario	It. 1	It. 2	It. 3	It. 4	It. 5	Promedio	Desv. est.
1K	0.306	0.224	0.223	0.229	0.231	0.243	0.036
10K	0.987	0.985	1.022	1.233	1.009	1.047	0.105
100K	8.244	8.720	9.240	8.921	8.131	8.651	0.464
1M	89.13	89.80	89.70	90.20	89.70	89.71	0.382

Tabla 33: Consulta 2 sin índices: tiempo de ejecución por iteración (ms).

Escenario	It. 1	It. 2	It. 3	It. 4	It. 5	Promedio	Desv. est.
1K	0.437	0.446	0.430	0.395	0.412	0.424	0.021
10K	3.035	3.055	2.921	2.932	2.892	2.967	0.073
100K	30.32	30.11	30.74	29.84	30.25	30.25	0.330
1M	344.83	341.69	343.57	335.61	338.41	340.82	3.790

Tabla 34: Consulta 3 sin índices: tiempo de ejecución por iteración (ms).

Escenario	It. 1	It. 2	It. 3	It. 4	It. 5	Promedio	Desv. est.
1K	0.214	0.202	0.201	0.212	0.206	0.207	0.006
10K	0.640	0.658	0.638	0.692	0.615	0.649	0.029
100K	3.792	3.769	3.802	3.990	3.783	3.827	0.092
1M	27.47	27.33	28.10	27.50	27.26	27.53	0.331

Tabla 35: Consulta 4 sin índices: tiempo de ejecución por iteración (ms).

Escenario	It. 1	It. 2	It. 3	It. 4	It. 5	Promedio	Desv. est.
1K	0.154	0.181	0.165	0.147	0.152	0.160	0.014
10K	5.732	5.310	5.317	5.380	5.317	5.411	0.182
100K	793.72	619.45	613.83	623.10	614.67	652.95	78.78
1M	29 154.01	29 149.10	29 797.96	29 508.77	29 041.98	29 330.36	315.28

Tabla 36: Consulta 5 sin índices: tiempo de ejecución por iteración (ms).

5.5.2. Con índices

Escenario	It. 1	It. 2	It. 3	It. 4	It. 5	Promedio	Desv. est.
1K	0.423	0.273	0.275	0.300	0.257	0.306	0.067
10K	1.042	0.829	0.874	0.835	0.855	0.887	0.088
100K	5.784	5.653	5.685	5.657	5.744	5.705	0.057
1M	53.29	51.37	51.16	51.32	51.85	51.80	0.871

Tabla 37: Consulta 1 con índices: tiempo de ejecución por iteración (ms).

Escenario	It. 1	It. 2	It. 3	It. 4	It. 5	Promedio	Desv. est.
1K	0.220	0.214	0.271	0.220	0.222	0.229	0.023
10K	1.041	1.004	1.025	1.020	1.008	1.020	0.015
100K	8.141	8.000	8.057	8.064	8.045	8.061	0.051
1M	114.66	115.68	110.75	112.37	110.60	112.81	2.289

Tabla 38: Consulta 2 con índices: tiempo de ejecución por iteración (ms).

Escenario	It. 1	It. 2	It. 3	It. 4	It. 5	Promedio	Desv. est.
1K	0.374	0.382	0.388	0.389	0.410	0.389	0.013
10K	3.024	2.917	2.883	2.872	2.876	2.914	0.064
100K	29.88	29.89	35.39	30.52	30.14	31.16	2.377
1M	335.59	337.34	337.71	341.95	333.90	337.30	3.012

Tabla 39: Consulta 3 con índices: tiempo de ejecución por iteración (ms).

Escenario	It. 1	It. 2	It. 3	It. 4	It. 5	Promedio	Desv. est.
1K	0.242	0.198	0.213	0.190	0.189	0.206	0.022
10K	0.648	0.622	0.642	0.636	0.589	0.627	0.024
100K	3.808	3.790	3.741	3.838	3.779	3.791	0.036
1M	27.19	27.70	27.17	27.14	27.30	27.30	0.229

Tabla 40: Consulta 4 con índices: tiempo de ejecución por iteración (ms).

Escenario	It. 1	It. 2	It. 3	It. 4	It. 5	Promedio	Desv. est.
1K	0.174	0.119	0.192	0.123	0.125	0.147	0.034
10K	0.457	0.500	0.474	0.446	0.419	0.459	0.030
100K	3.110	3.009	2.891	2.887	2.942	2.968	0.094
1M	31.96	28.91	29.66	28.57	29.37	29.69	1.333

Tabla 41: Consulta 5 con índices: tiempo de ejecución por iteración (ms).

5.5.3. Comparación entre tipos de índice

Esta prueba responde qué pasa si en lugar del índice propuesto se usa otro. Para la consulta 1 comparamos el B+ tree con INCLUDE, un B+ tree simple, un índice BRIN y la ausencia de índice sobre la fecha. Para la consulta 5 retiramos temporalmente `uq_evento` y comparamos un B+ tree compuesto, un hash sobre la parcela, un B+ tree sobre la fecha y la ausencia de índice.

Consulta	Variante	Construcción (s)	Tamaño (KB)	Promedio (ms)	Desv. (ms)	Mejora
C1	B+ tree con INCLUDE (propuesto)	0.46	30 336	51.80	1.002	2.3
C1	B+ tree simple	0.31	12 488	57.46	2.935	2.1
C1	BRIN (32 paginas por rango)	0.09	24	66.98	15.18	1.8
C1	Sin indice sobre fecha_hora	–	–	118.07	1.156	1.0
C5	B+ tree compuesto (zona, fecha)	0.09	3 720	28.83	0.673	1 038.9
C5	Hash sobre id_parcela	0.11	5 232	2 075.46	98.27	14.4
C5	B+ tree sobre fecha_hora_inicio	0.07	2 568	334.76	26.12	89.5
C5	Sin indice sobre Evento_Riego	–	–	29 949.36	259.06	1.0

Tabla 42: Comparación entre tipos de índice con un millón de registros. La mejora se calcula contra la variante sin índice.

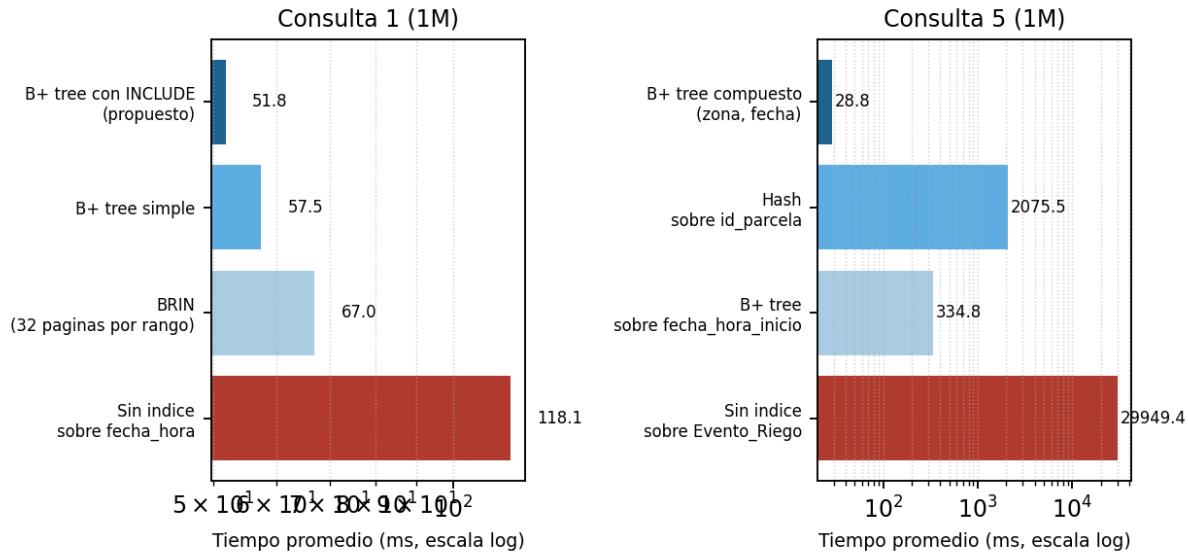


Figura 4: Tiempo promedio según el tipo de índice (1M, escala logarítmica).

El resultado más útil para este dominio es el del BRIN. Guarda solo el valor mínimo y máximo de la fecha por cada bloque de 32 páginas, por eso ocupa 24 KB frente a los casi 30 MB del B+ tree con `INCLUDE`, y se construye cinco veces más rápido. Aun así logra el 77 % de la mejora del B+ tree (67 ms frente a 52 ms, partiendo de 118 ms). Funciona porque las lecturas están guardadas en orden cronológico: cada bloque contiene un intervalo de tiempo estrecho y el índice descarta la mayoría de bloques sin leerlos. Si las lecturas llegaran desordenadas, el BRIN no serviría.

En la consulta 5, el hash sobre la parcela solo resuelve la primera igualdad: encuentra todos los riegos de la parcela y debe filtrar zona y fecha fila por fila, por eso tarda 72 veces más que el B+ tree compuesto. El B+ tree sobre la fecha recorre el rango de seis horas de toda la empresa y filtra la zona, lo que lo deja en un punto intermedio. Un índice solo es tan bueno como su coincidencia con el patrón de búsqueda: el orden y la combinación de columnas pesan más que el tipo de estructura.

5.5.4. Costo de los índices en las operaciones de escritura

Operación	Filas	Sin (ms)	Desv.	Con (ms)	Desv.	Sobrecosto
Insercion de 20 000 lecturas	20 000	167.63	3.853	238.34	9.828	+42 %
Actualizacion de humedad (sensores 1 a 60)	17 083	84.62	19.02	148.26	25.75	+75 %
Insercion de 5 000 eventos de riego	5 000	66.60	2.339	74.31	4.604	+12 %

Tabla 43: Costo de escritura con y sin los índices de optimización (1M, 5 iteraciones).

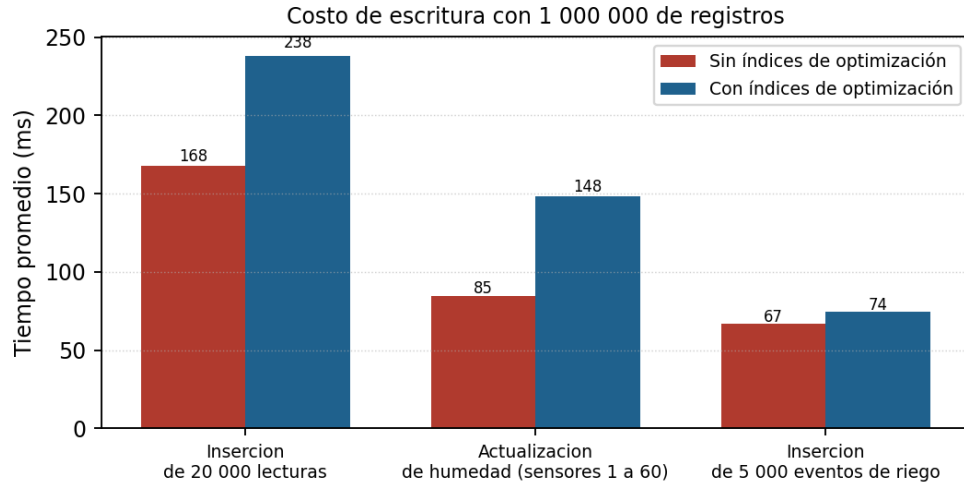


Figura 5: Tiempo de las operaciones de escritura con y sin índices de optimización (1M).

Los índices tienen un precio. Insertar 20 000 lecturas cuesta 42 % más con los índices de optimización, porque cada fila debe insertarse en tres árboles en lugar de uno. La actualización de humedad es la más afectada (75 %): la columna `humedad_pct` está incluida en dos índices, así que PostgreSQL no puede hacer una actualización HOT (en la misma página, sin tocar índices) y debe escribir una entrada nueva en cada índice. En `Evento_Riego`, con un solo índice adicional, el sobre costo es de 12 %. La Tabla 44 muestra el espacio que ocupa cada índice.

Tabla	Índice	Tipo	Tamaño (KB)	Origen
Alerta	<code>uq_alerta</code>	B+ tree	1 464	UNIQUE
Alerta	<code>idx_alerta_humedad_baja</code>	B+ tree	1 024	Optimización
Alerta	<code>pk_alerta</code>	B+ tree	672	Clave primaria
Campania	<code>pk_campania</code>	B+ tree	56	Clave primaria
Campania	<code>uq_campania</code>	B+ tree	56	UNIQUE
Campania	<code>idx_campania_vigente</code>	B+ tree	32	Optimización
Campania	<code>idx_campania_cultivo</code>	B+ tree	32	Optimización
Cosecha	<code>idx_cosecha_campania</code>	Hash	1 040	Optimización
Cosecha	<code>idx_cosecha_categoria</code>	B+ tree	1 008	Optimización
Cosecha	<code>pk_cosecha</code>	B+ tree	568	Clave primaria
Dispositivo	<code>uq_dispositivo_ser</code>	B+ tree	104	UNIQUE
Dispositivo	<code>pk_dispositivo</code>	B+ tree	80	Clave primaria
Dispositivo	<code>idx_dispositivo_zona</code>	B+ tree	80	Optimización
Evento_Riego	<code>idx_evento_parcela_fecha</code>	B+ tree	3 720	Optimización
Evento_Riego	<code>uq_evento</code>	B+ tree	3 720	UNIQUE
Evento_Riego	<code>pk_evento</code>	B+ tree	2 656	Clave primaria
Lectura_Suelo	<code>idx_lectura_fecha</code>	B+ tree	30 336	Optimización
Lectura_Suelo	<code>idx_lectura_disp_humedad</code>	B+ tree	23 608	Optimización
Lectura_Suelo	<code>pk_lectura</code>	B+ tree	23 608	Clave primaria

Tabla 44: Índices de las tablas involucradas (1M).

5.6. Resultados

5.6.1. Consulta 1

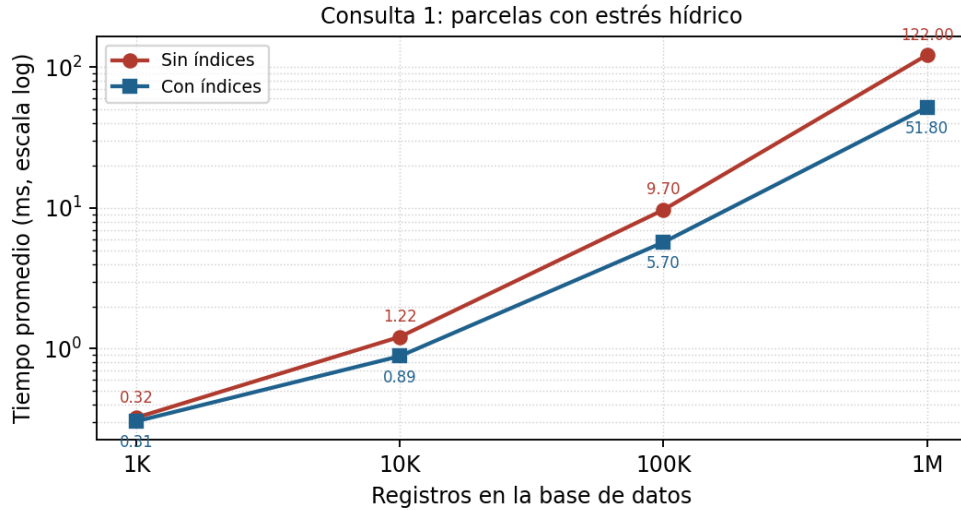


Figura 6: Consulta 1: tiempo promedio por escenario.

Escenario	Sin índices (ms)	Con índices (ms)	Relación sin/con	Filas devueltas
1K	0.325	0.306	1.06	2
10K	1.216	0.887	1.37	10
100K	9.701	5.705	1.70	39
1M	122.00	51.80	2.36	116

Tabla 45: Consulta 1: promedios y relación de mejora.

El índice mejora la consulta a partir de 10 000 registros y con un millón la acelera 2.4 veces. Sin índice, PostgreSQL recorre las 765 861 lecturas con un *Parallel Seq Scan* y descarta nueve de cada diez. Con el índice, el *Index Only Scan* sobre `idx_lectura_fecha` lee solo las 77 020 lecturas del último mes. Los índices sobre campañas y dispositivos no se usan: esas tablas tienen pocos miles de filas y el planificador las recorre completas porque le resulta más barato.

Plan de ejecución sin índices (1M)

```

1 Sort (actual time=118.423..118.500 rows=116 loops=1)
2   Sort Key: (round(avg(l.humedad_pct), 2))
3   Sort Method: quicksort Memory: 33kB
4   -> Finalize HashAggregate (actual time=118.132..118.447 rows=116 loops=1)
5     Group Key: f.nombre, p.codigo_parcela, cu.nombre_comun
6     Filter: (avg(l.humedad_pct) < min(cu.humedad_optima_min))
7     Batches: 1 Memory Usage: 913kB
8     Rows Removed by Filter: 418
9     -> Gather (actual time=112.133..117.039 rows=1602 loops=1)
10       Workers Planned: 2
11       Workers Launched: 2
12       -> Partial HashAggregate (actual time=104.522..104.728 rows=534 loops=3)
13         Group Key: f.nombre, p.codigo_parcela, cu.nombre_comun
14         Batches: 1 Memory Usage: 657kB
15         Worker 0: Batches: 1 Memory Usage: 657kB
16         Worker 1: Batches: 1 Memory Usage: 657kB
17         -> Hash Join (actual time=55.064..73.298 rows=23118 loops=3)
18           Hash Cond: (l.id_dispositivo = s.id_dispositivo)
19           -> Parallel Seq Scan on lectura_suelo l (actual time=43.527..55.417 rows=25673 loops=3)
20             Filter: ((humedad_pct IS NOT NULL) AND (fecha_hora >= '2026-08-16 00:00:00':timestamp without time zone) AND (fecha_hora < '2026-09-16 00:00:00':timestamp without time zone))

```

```

21 Rows Removed by Filter: 229614
22 -> Hash (actual time=11.493..11.502 rows=1971 loops=3)
23 Buckets: 2048 Batches: 1 Memory Usage: 159kB
24 -> Hash Join (actual time=1.614..1.831 rows=1971 loops=3)
25 Hash Cond: ((d.id_parcela = z.id_parcela) AND (d.num_zona = z.num_zona))
26 -> Hash Join (actual time=1.110..1.754 rows=1971 loops=3)
27 Hash Cond: (d.id_parcela = p.id_parcela)
28 -> Hash Join (actual time=0.417..0.377 rows=2200 loops=3)
29 Hash Cond: (d.id_dispositivo = s.id_dispositivo)
30 -> Seq Scan on dispositivo d (actual time=0.008..0.193 rows=2800 loops=3)
31 -> Hash (actual time=0.386..0.387 rows=2200 loops=3)
32 Buckets: 4096 Batches: 1 Memory Usage: 110kB
33 -> Seq Scan on sensor s (actual time=0.005..0.174 rows=2200 loops=3)
34 -> Hash (actual time=0.682..0.687 rows=539 loops=3)
35 Buckets: 1024 Batches: 1 Memory Usage: 49kB
36 -> Hash Join (actual time=0.204..0.570 rows=539 loops=3)
37 Hash Cond: (ca.id_cultivo = cu.id_cultivo)
38 -> Hash Join (actual time=0.180..0.471 rows=539 loops=3)
39 Hash Cond: (p.id_fundo = f.id_fundo)
40 -> Hash Join (actual time=0.155..0.372 rows=539 loops=3)
41 Hash Cond: (ca.id_parcela = p.id_parcela)
42 -> Seq Scan on campania ca (actual time=0.004..0.139 rows=539 loops=3)
43 Filter: ((estado)::text = 'En curso':text)
44 Rows Removed by Filter: 961
45 -> Hash (actual time=0.135..0.135 rows=600 loops=3)
46 Buckets: 1024 Batches: 1 Memory Usage: 37kB
47 -> Seq Scan on parcela p (actual time=0.006..0.057 rows=600 loops=3)
48 -> Hash (actual time=0.016..0.017 rows=12 loops=3)
49 Buckets: 1024 Batches: 1 Memory Usage: 9kB
50 -> Seq Scan on fundo f (actual time=0.010..0.011 rows=12 loops=3)
51 -> Hash (actual time=0.013..0.014 rows=12 loops=3)
52 Buckets: 1024 Batches: 1 Memory Usage: 9kB
53 -> Seq Scan on cultivo cu (actual time=0.006..0.007 rows=12 loops=3)
54 -> Hash (actual time=0.475..0.475 rows=2400 loops=3)
55 Buckets: 4096 Batches: 1 Memory Usage: 122kB
56 -> Seq Scan on zona_manejo z (actual time=0.013..0.217 rows=2400 loops=3)
57 Planning Time: 3.552 ms
58 Execution Time: 118.710 ms

```

Plan de ejecución con índices (1M)

```

1 Sort (actual time=49.632..49.647 rows=116 loops=1)
2 Sort Key: (round(avg(l.humedad_pct), 2))
3 Sort Method: quicksort Memory: 33kB
4 -> HashAggregate (actual time=49.378..49.594 rows=116 loops=1)
5 Group Key: f.nombre, p.codigo_parcela, cu.nombre_comun
6 Filter: (avg(l.humedad_pct) < min(cu.humedad_optima_min))
7 Batches: 1 Memory Usage: 657kB
8 Rows Removed by Filter: 418
9 -> Hash Join (actual time=3.176..27.939 rows=69353 loops=1)
10 Hash Cond: (l.id_dispositivo = s.id_dispositivo)
11 -> Index Only Scan using idx_lectura_fecha on lectura_suelo l (actual time=0.012..10.757 rows=77020 loops=1)
12 Index Cond: ((fecha_hora >= '2026-08-16 00:00:00':timestamp without time zone) AND (fecha_hora < '2026-09-16 00:00:00':timestamp without time zone))
13 Filter: (humedad_pct IS NOT NULL)
14 Rows Removed by Filter: 14931
15 Heap Fetches: 0
16 -> Hash (actual time=3.160..3.167 rows=1971 loops=1)
17 Buckets: 2048 Batches: 1 Memory Usage: 159kB
18 -> Hash Join (actual time=1.450..2.776 rows=1971 loops=1)
19 Hash Cond: ((d.id_parcela = z.id_parcela) AND (d.num_zona = z.num_zona))
20 -> Hash Join (actual time=0.970..1.982 rows=1971 loops=1)
21 Hash Cond: (d.id_parcela = p.id_parcela)
22 -> Hash Join (actual time=0.356..0.995 rows=2200 loops=1)
23 Hash Cond: (d.id_dispositivo = s.id_dispositivo)
24 -> Seq Scan on dispositivo d (actual time=0.002..0.182 rows=2800 loops=1)
25 -> Hash (actual time=0.349..0.350 rows=2200 loops=1)
26 Buckets: 4096 Batches: 1 Memory Usage: 110kB
27 -> Seq Scan on sensor s (actual time=0.003..0.165 rows=2200 loops=1)
28 -> Hash (actual time=0.610..0.614 rows=539 loops=1)
29 Buckets: 1024 Batches: 1 Memory Usage: 49kB
30 -> Hash Join (actual time=0.146..0.511 rows=539 loops=1)
31 Hash Cond: (ca.id_cultivo = cu.id_cultivo)
32 -> Hash Join (actual time=0.136..0.427 rows=539 loops=1)
33 Hash Cond: (p.id_fundo = f.id_fundo)
34 -> Hash Join (actual time=0.128..0.347 rows=539 loops=1)
35 Hash Cond: (ca.id_parcela = p.id_parcela)
36 -> Seq Scan on campania ca (actual time=0.003..0.140 rows=539 loops=1)
37 Filter: ((estado)::text = 'En curso':text)
38 Rows Removed by Filter: 961
39 -> Hash (actual time=0.121..0.121 rows=600 loops=1)
40 Buckets: 1024 Batches: 1 Memory Usage: 37kB
41 -> Seq Scan on parcela p (actual time=0.003..0.059 rows=600 loops=1)
42 -> Hash (actual time=0.005..0.006 rows=12 loops=1)
43 Buckets: 1024 Batches: 1 Memory Usage: 9kB
44 -> Seq Scan on fundo f (actual time=0.002..0.003 rows=12 loops=1)
45 -> Hash (actual time=0.007..0.007 rows=12 loops=1)
46 Buckets: 1024 Batches: 1 Memory Usage: 9kB
47 -> Seq Scan on cultivo cu (actual time=0.003..0.004 rows=12 loops=1)
48 -> Hash (actual time=0.475..0.475 rows=2400 loops=1)
49 Buckets: 4096 Batches: 1 Memory Usage: 122kB
50 -> Seq Scan on zona_manejo z (actual time=0.006..0.200 rows=2400 loops=1)

```

```

51 Planning Time: 3.962 ms
52 Execution Time: 49.742 ms

```

5.6.2. Consulta 2

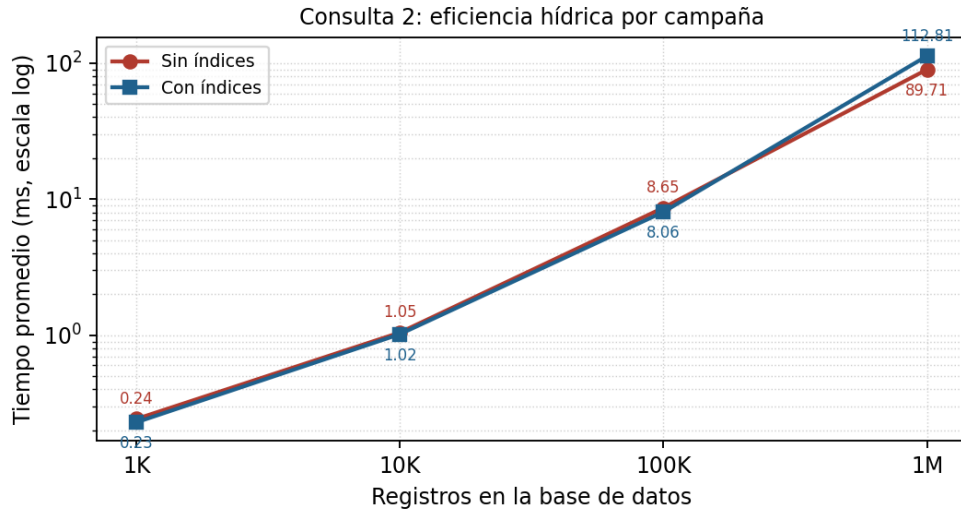


Figura 7: Consulta 2: tiempo promedio por escenario.

Escenario	Sin índices (ms)	Con índices (ms)	Relación sin/con	Filas devueltas
1K	0.243	0.229	1.06	0
10K	1.047	1.020	1.03	7
100K	8.651	8.061	1.07	58
1M	89.71	112.81	0.80 (más lenta)	159

Tabla 46: Consulta 2: promedios y relación de mejora.

Es el caso en que el índice perjudica. Hasta 100 000 registros la versión con índices es ligeramente más rápida, pero con un millón es 26 % más lenta. Sin índices, el plan lee una sola vez los 120 000 riegos y los une con las campañas mediante un *Hash Join*. Con índices, el planificador cambia a un *Nested Loop*: por cada una de las 1 500 campañas desciende por `idx_evento_parcela_fecha` y agrega un nodo *Memoize* para reutilizar búsquedas repetidas. Ese caché no acierta ni una vez (0 aciertos, 1 500 fallos) porque cada campaña tiene un rango de fechas distinto. Mil quinientos descensos por el índice más el costo del caché inútil superan a un único recorrido secuencial. El planificador estimó mal y solo la medición lo revela.

Plan de ejecución sin índices (1M)

```

1 Sort (actual time=85.733..85.816 rows=159 loops=1)
2 Sort Key: (round((r.litros_aplicados / pr.kilos), 2)) DESC
3 Sort Method: quicksort Memory: 37kB
4 CTE riego
5   -> Finalize HashAggregate (actual time=76.102..76.432 rows=1095 loops=1)
6     Group Key: ca_1.id_campania
7     Batches: 1 Memory Usage: 577kB
8     -> Gather (actual time=73.451..75.096 rows=2165 loops=1)
9       Workers Planned: 1
10      Workers Launched: 1
11      -> Partial HashAggregate (actual time=69.902..70.184 rows=1082 loops=2)

```

```

12      Group Key: ca_1.id_campania
13      Batches: 1 Memory Usage: 577kB
14      Worker 0: Batches: 1 Memory Usage: 577kB
15      -> Hash Join (actual time=0.340..46.863 rows=52198 loops=2)
16          Hash Cond: (er.id_parcela = ca_1.id_parcela)
17          Join Filter: ((er.fecha_hora_inicio >= ca_1.fecha_siembra) AND (er.fecha_hora_inicio < ca_1.fecha_fin_estimada))
18          Rows Removed by Join Filter: 98268
19          -> Parallel Seq Scan on evento_riego er (actual time=0.003..4.063 rows=60000 loops=2)
20              -> Hash (actual time=0.317..0.318 rows=1500 loops=2)
21                  Buckets: 2048 Batches: 1 Memory Usage: 87kB
22              -> Seq Scan on campania ca_1 (actual time=0.008..0.135 rows=1500 loops=2)
23  InitPlan 2 (returns $2)
24      -> Aggregate (actual time=0.595..0.596 rows=1 loops=1)
25          -> CTE Scan on riego (actual time=0.001..0.443 rows=1095 loops=1)
26  -> Hash Join (actual time=85.120..85.673 rows=159 loops=1)
27      Hash Cond: (r.id_campania = pr.id_campania)
28      -> Hash Join (actual time=77.144..77.578 rows=323 loops=1)
29          Hash Cond: (ca.id_cultivo = cu.id_cultivo)
30          -> Hash Join (actual time=77.132..77.519 rows=323 loops=1)
31              Hash Cond: (p.id_fundo = f.id_fundo)
32              -> Hash Join (actual time=77.123..77.462 rows=323 loops=1)
33                  Hash Cond: (ca.id_parcela = p.id_parcela)
34                  -> Hash Join (actual time=77.008..77.280 rows=323 loops=1)
35                      Hash Cond: (r.id_campania = ca.id_campania)
36                      -> CTE Scan on riego r (actual time=76.706..76.907 rows=323 loops=1)
37                          Filter: (litros_aplicados > $2)
38                          Rows Removed by Filter: 772
39                      -> Hash (actual time=0.293..0.293 rows=1500 loops=1)
40                          Buckets: 2048 Batches: 1 Memory Usage: 81kB
41                      -> Seq Scan on campania ca (actual time=0.009..0.141 rows=1500 loops=1)
42                  -> Hash (actual time=0.111..0.112 rows=600 loops=1)
43                      Buckets: 1024 Batches: 1 Memory Usage: 32kB
44                      -> Seq Scan on parcela p (actual time=0.002..0.054 rows=600 loops=1)
45                  -> Hash (actual time=0.006..0.006 rows=12 loops=1)
46                      Buckets: 1024 Batches: 1 Memory Usage: 9kB
47                      -> Seq Scan on fundo f (actual time=0.003..0.004 rows=12 loops=1)
48          -> Hash (actual time=0.009..0.010 rows=12 loops=1)
49              Buckets: 1024 Batches: 1 Memory Usage: 9kB
50              -> Seq Scan on cultivo cu (actual time=0.004..0.006 rows=12 loops=1)
51  -> Hash (actual time=7.957..7.959 rows=902 loops=1)
52      Buckets: 1024 Batches: 1 Memory Usage: 50kB
53      -> Subquery Scan on pr (actual time=7.563..7.842 rows=902 loops=1)
54          -> HashAggregate (actual time=7.561..7.763 rows=902 loops=1)
55              Group Key: cosecha.id_campania
56              Batches: 1 Memory Usage: 553kB
57              -> Seq Scan on cosecha (actual time=0.004..2.737 rows=25000 loops=1)
58  Planning Time: 0.800 ms
59  Execution Time: 85.911 ms

```

Plan de ejecución con índices (1M)

```

1  Sort (actual time=105.264..105.280 rows=159 loops=1)
2  Sort Key: (round((r.litros_aplicados / pr.kilos), 2)) DESC
3  Sort Method: quicksort Memory: 37kB
4  CTE riego
5  -> GroupAggregate (actual time=0.082..96.307 rows=1095 loops=1)
6      Group Key: ca_1.id_campania
7      -> Nested Loop (actual time=0.016..79.549 rows=104397 loops=1)
8          -> Index Scan using pk_campania on campania ca_1 (actual time=0.006..0.364 rows=1500 loops=1)
9          -> Memoize (actual time=0.004..0.046 rows=70 loops=1500)
10             Cache Key: ca_1.fecha_siembra, ca_1.fecha_fin_estimada, ca_1.id_parcela
11             Cache Mode: binary
12             Hits: 0 Misses: 1500 Evictions: 0 Overflows: 0 Memory Usage: 5821kB
13             -> Index Scan using idx_evento_parcela_fecha on evento_riego er (actual time=0.003..0.026 rows=70 loops=1500)
14                 Index Cond: ((id_parcela = ca_1.id_parcela) AND (fecha_hora_inicio >= ca_1.fecha_siembra) AND (fecha_hora_inicio < ca_1.fecha_fin_estimada))
15  )
16  InitPlan 2 (returns $4)
17      -> Aggregate (actual time=96.801..96.802 rows=1 loops=1)
18          -> CTE Scan on riego (actual time=0.000..96.537 rows=1095 loops=1)
19  -> Hash Join (actual time=104.606..105.201 rows=159 loops=1)
20      Hash Cond: (r.id_campania = pr.id_campania)
21      -> Hash Join (actual time=97.308..97.770 rows=323 loops=1)
22          Hash Cond: (ca.id_cultivo = cu.id_cultivo)
23          -> Hash Join (actual time=97.296..97.706 rows=323 loops=1)
24              Hash Cond: (p.id_fundo = f.id_fundo)
25              -> Hash Join (actual time=97.287..97.648 rows=323 loops=1)
26                  Hash Cond: (ca.id_parcela = p.id_parcela)
27                  -> Hash Join (actual time=97.172..97.455 rows=323 loops=1)
28                      Hash Cond: (r.id_campania = ca.id_campania)
29                      -> CTE Scan on riego r (actual time=96.888..97.095 rows=323 loops=1)
30                          Filter: (litros_aplicados > $4)
31                          Rows Removed by Filter: 772
32                      -> Hash (actual time=0.275..0.276 rows=1500 loops=1)
33                          Buckets: 2048 Batches: 1 Memory Usage: 81kB
34                      -> Seq Scan on campania ca (actual time=0.009..0.133 rows=1500 loops=1)
35                  -> Hash (actual time=0.111..0.112 rows=600 loops=1)
36                      Buckets: 1024 Batches: 1 Memory Usage: 32kB
37                      -> Seq Scan on parcela p (actual time=0.002..0.053 rows=600 loops=1)
38              -> Hash (actual time=0.005..0.006 rows=12 loops=1)
39                  Buckets: 1024 Batches: 1 Memory Usage: 9kB
40                  -> Seq Scan on fundo f (actual time=0.001..0.003 rows=12 loops=1)
41          -> Hash (actual time=0.009..0.009 rows=12 loops=1)

```

```

41 Buckets: 1024 Batches: 1 Memory Usage: 9kB
42 -> Seq Scan on cultivo cu (actual time=0.003..0.005 rows=12 loops=1)
43 -> Hash (actual time=7.290..7.291 rows=902 loops=1)
44 Buckets: 1024 Batches: 1 Memory Usage: 50kB
45 -> Subquery Scan on pr (actual time=6.854..7.176 rows=902 loops=1)
46 -> HashAggregate (actual time=6.852..7.076 rows=902 loops=1)
47 Group Key: cosecha.id_campania
48 Batches: 1 Memory Usage: 553kB
49 -> Seq Scan on cosecha (actual time=0.003..1.765 rows=25000 loops=1)
50 Planning Time: 0.879 ms
51 Execution Time: 105.374 ms

```

5.6.3. Consulta 3

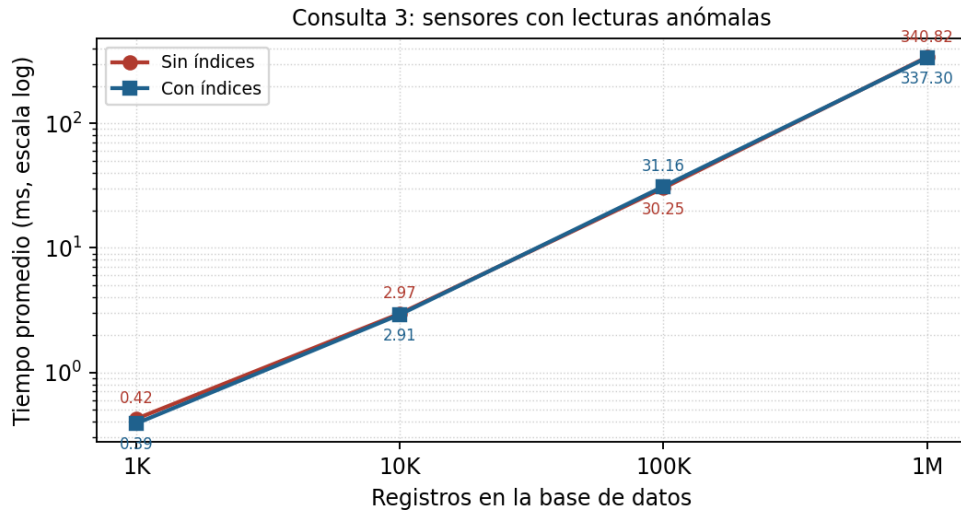


Figura 8: Consulta 3: tiempo promedio por escenario.

Escenario	Sin índices (ms)	Con índices (ms)	Relación sin/con	Filas devueltas
1K	0.424	0.389	1.09	8
10K	2.967	2.914	1.02	25
100K	30.25	31.16	0.97 (más lenta)	110
1M	340.82	337.30	1.01	359

Tabla 47: Consulta 3: promedios y relación de mejora.

Los tiempos son prácticamente iguales en las dos condiciones y el plan es el mismo: un recorrido secuencial paralelo de toda la tabla de lecturas. La consulta necesita el 84 % de las filas, y para ese volumen leer la tabla en orden físico es más barato que recorrer un índice. El índice `idx_lectura_disp_humedad` ocupa 24 MB y nunca se usa.

Plan de ejecución sin índices (1M)

```

1 Sort (actual time=324.220..324.318 rows=359 loops=1)
2 Sort Key: (round(((100.0 * (m.anomalas)::numeric) / (m.total)::numeric), 2)) DESC, m.total DESC
3 Sort Method: quicksort Memory: 53kB
4 CTE medidas
5 -> Finalize HashAggregate (actual time=319.637..319.899 rows=1693 loops=1)
6 Group Key: l.id_dispositivo
7 Batches: 1 Memory Usage: 369kB
8 -> Gather (actual time=315.349..318.795 rows=5079 loops=1)
9 Workers Planned: 2
10 Workers Launched: 2

```

```

11  -> Partial HashAggregate (actual time=309.122..309.317 rows=1693 loops=3)
12  Group Key: l.id_dispositivo
13  Batches: 1 Memory Usage: 369kB
14  Worker 0: Batches: 1 Memory Usage: 369kB
15  Worker 1: Batches: 1 Memory Usage: 369kB
16  -> Hash Join (actual time=3.992..161.661 rows=192566 loops=3)
17  Hash Cond: (l.id_dispositivo = d_1.id_dispositivo)
18  -> Parallel Seq Scan on lectura_suelo l (actual time=0.015..70.872 rows=213710 loops=3)
19  Filter: (humedad_pct IS NOT NULL)
20  Rows Removed by Filter: 41577
21  -> Hash (actual time=3.951..3.955 rows=2509 loops=3)
22  Buckets: 4096 Batches: 1 Memory Usage: 145kB
23  -> Hash Join (actual time=0.302..3.679 rows=2509 loops=3)
24  Hash Cond: (d_1.id_parcela = ca.id_parcela)
25  -> Seq Scan on dispositivo d_1 (actual time=0.005..0.180 rows=2800 loops=3)
26  -> Hash (actual time=0.287..0.290 rows=539 loops=3)
27  Buckets: 1024 Batches: 1 Memory Usage: 33kB
28  -> Hash Join (actual time=0.032..0.223 rows=539 loops=3)
29  Hash Cond: (ca.id_cultivo = cu.id_cultivo)
30  -> Seq Scan on campania ca (actual time=0.008..0.121 rows=539 loops=3)
31  Filter: ((estado)::text = 'En curso':text)
32  Rows Removed by Filter: 961
33  -> Hash (actual time=0.014..0.015 rows=12 loops=3)
34  Buckets: 1024 Batches: 1 Memory Usage: 9kB
35  -> Seq Scan on cultivo cu (actual time=0.007..0.009 rows=12 loops=3)
36  InitPlan 2 (returns $2)
37  -> Aggregate (actual time=0.652..0.652 rows=1 loops=1)
38  -> CTE Scan on medidas (actual time=0.001..0.523 rows=1693 loops=1)
39  -> Hash Join (actual time=323.425..324.057 rows=359 loops=1)
40  Hash Cond: (d.id_parcela = p.id_parcela)
41  -> Hash Join (actual time=323.312..323.765 rows=359 loops=1)
42  Hash Cond: (d.id_dispositivo = s.id_dispositivo)
43  -> Seq Scan on dispositivo d (actual time=0.001..0.180 rows=2800 loops=1)
44  -> Hash (actual time=323.304..323.305 rows=359 loops=1)
45  Buckets: 1024 Batches: 1 Memory Usage: 33kB
46  -> Hash Join (actual time=320.790..323.230 rows=359 loops=1)
47  Hash Cond: (m.id_dispositivo = s.id_dispositivo)
48  -> CTE Scan on medidas m (actual time=320.300..322.663 rows=359 loops=1)
49  Filter: ((total >= 5) AND (((1.0 * (anomalas)::numeric) / (total)::numeric) > $2))
50  Rows Removed by Filter: 1334
51  -> Hash (actual time=0.477..0.477 rows=2200 loops=1)
52  Buckets: 4096 Batches: 1 Memory Usage: 135kB
53  -> Seq Scan on sensor s (actual time=0.009..0.181 rows=2200 loops=1)
54  -> Hash (actual time=0.108..0.108 rows=600 loops=1)
55  Buckets: 1024 Batches: 1 Memory Usage: 34kB
56  -> Seq Scan on parcela p (actual time=0.004..0.054 rows=600 loops=1)
57  Planning Time: 0.649 ms
58  Execution Time: 324.423 ms

```

Plan de ejecución con índices (1M)

```

1  Sort (actual time=322.806..322.894 rows=359 loops=1)
2  Sort Key: (round(((100.0 * (m.anomalas)::numeric) / (m.total)::numeric), 2)) DESC, m.total DESC
3  Sort Method: quicksort Memory: 53kB
4  CTE medidas
5  -> Finalize HashAggregate (actual time=318.307..318.572 rows=1693 loops=1)
6  Group Key: l.id_dispositivo
7  Batches: 1 Memory Usage: 369kB
8  -> Gather (actual time=313.694..317.475 rows=5079 loops=1)
9  Workers Planned: 2
10 Workers Launched: 2
11 -> Partial HashAggregate (actual time=308.832..309.029 rows=1693 loops=3)
12 Group Key: l.id_dispositivo
13 Batches: 1 Memory Usage: 369kB
14 Worker 0: Batches: 1 Memory Usage: 369kB
15 Worker 1: Batches: 1 Memory Usage: 369kB
16 -> Hash Join (actual time=3.991..184.920 rows=192566 loops=3)
17 Hash Cond: (l.id_dispositivo = d_1.id_dispositivo)
18 -> Parallel Seq Scan on lectura_suelo l (actual time=0.014..63.854 rows=213710 loops=3)
19 Filter: (humedad_pct IS NOT NULL)
20 Rows Removed by Filter: 41577
21 -> Hash (actual time=3.954..3.959 rows=2509 loops=3)
22 Buckets: 4096 Batches: 1 Memory Usage: 145kB
23 -> Hash Join (actual time=0.297..0.993 rows=2509 loops=3)
24 Hash Cond: (d_1.id_parcela = ca.id_parcela)
25 -> Seq Scan on dispositivo d_1 (actual time=0.003..0.186 rows=2800 loops=3)
26 -> Hash (actual time=0.285..0.287 rows=539 loops=3)
27 Buckets: 1024 Batches: 1 Memory Usage: 33kB
28 -> Hash Join (actual time=0.030..0.229 rows=539 loops=3)
29 Hash Cond: (ca.id_cultivo = cu.id_cultivo)
30 -> Seq Scan on campania ca (actual time=0.007..0.127 rows=539 loops=3)
31 Filter: ((estado)::text = 'En curso':text)
32 Rows Removed by Filter: 961
33 -> Hash (actual time=0.014..0.015 rows=12 loops=3)
34 Buckets: 1024 Batches: 1 Memory Usage: 9kB
35 -> Seq Scan on cultivo cu (actual time=0.007..0.009 rows=12 loops=3)
36 InitPlan 2 (returns $2)
37 -> Aggregate (actual time=0.632..0.633 rows=1 loops=1)
38 -> CTE Scan on medidas (actual time=0.001..0.517 rows=1693 loops=1)
39 -> Hash Join (actual time=320.296..322.637 rows=359 loops=1)
40 Hash Cond: (d.id_parcela = p.id_parcela)
41 -> Hash Join (actual time=320.184..322.336 rows=359 loops=1)

```

```

42 Hash Cond: (d.id_dispositivo = s.id_dispositivo)
43 -> Seq Scan on dispositivo d (actual time=0.001..1.874 rows=2800 loops=1)
44 -> Hash (actual time=320.177..320.178 rows=359 loops=1)
45 Buckets: 1024 Batches: 1 Memory Usage: 33kB
46 -> Hash Join (actual time=319.443..320.108 rows=359 loops=1)
47 Hash Cond: (m.id_dispositivo = s.id_dispositivo)
48 -> CTE Scan on medidas m (actual time=318.952..319.562 rows=359 loops=1)
49 Filter: ((total >= 5) AND (((1.0 * (anomalias)::numeric) / (total)::numeric) > $2))
50 Rows Removed by Filter: 1334
51 -> Hash (actual time=0.479..0.480 rows=2200 loops=1)
52 Buckets: 4096 Batches: 1 Memory Usage: 135kB
53 -> Seq Scan on sensor s (actual time=0.008..0.180 rows=2200 loops=1)
54 -> Hash (actual time=0.107..0.107 rows=600 loops=1)
55 Buckets: 1024 Batches: 1 Memory Usage: 34kB
56 -> Seq Scan on parcela p (actual time=0.004..0.054 rows=600 loops=1)
57 Planning Time: 0.810 ms
58 Execution Time: 323.000 ms

```

5.6.4. Consulta 4

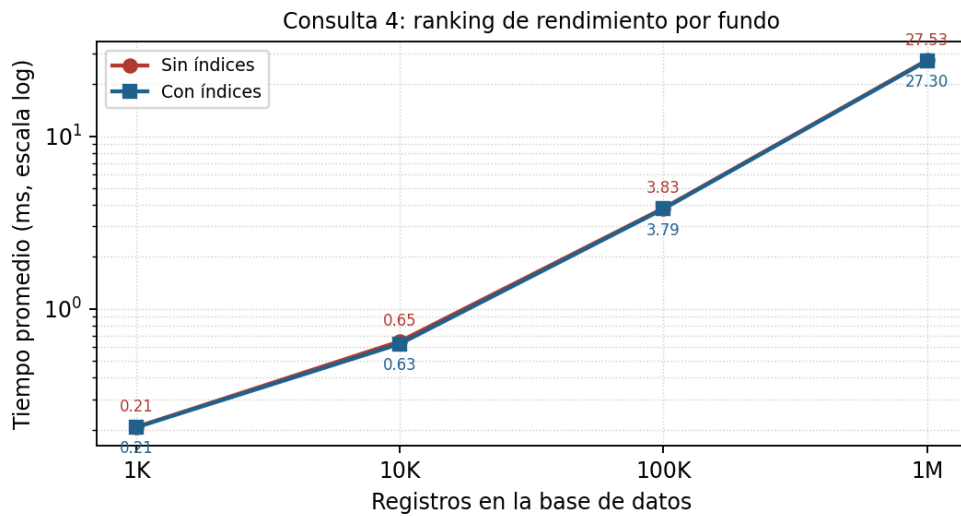


Figura 9: Consulta 4: tiempo promedio por escenario.

Escenario	Sin índices (ms)	Con índices (ms)	Relación sin/con	Filas devueltas
1K	0.207	0.206	1.00	10
10K	0.649	0.627	1.03	43
100K	3.827	3.791	1.01	195
1M	27.53	27.30	1.01	902

Tabla 48: Consulta 4: promedios y relación de mejora.

Tampoco hay diferencia. El filtro por categoría conserva el 82 % de las cosechas (primera y segunda), así que el planificador vuelve a preferir el recorrido secuencial, y la tabla de cosechas es pequeña (25 000 filas). La mayor parte del tiempo se va en las funciones de ventana y en el ordenamiento, que ningún índice acelera en este caso.

Plan de ejecución sin índices (1M)

```

1 Sort (actual time=26.184..26.270 rows=902 loops=1)
2 Sort Key: rendimiento.fundo, (rank() OVER (??))
3 Sort Method: quicksort Memory: 111kB
4 -> WindowAgg (actual time=24.438..25.414 rows=902 loops=1)

```

```

5  -> WindowAgg (actual time=24.359..24.814 rows=902 loops=1)
6  -> Sort (actual time=24.354..24.402 rows=902 loops=1)
7      Sort Key: rendimiento.id_fundo, ((rendimiento.kilos / rendimiento.area_ha)) DESC
8      Sort Method: quicksort Memory: 104kB
9  -> Subquery Scan on rendimiento (actual time=22.940..23.434 rows=902 loops=1)
10     -> HashAggregate (actual time=22.937..23.230 rows=902 loops=1)
11         Group Key: f.id_fundo, cu.nombre_comun, ca.id_campania, p.area_ha
12         Batches: 1 Memory Usage: 1297kB
13     -> Hash Join (actual time=0.435..16.063 rows=20452 loops=1)
14         Hash Cond: (ca.id_cultivo = cu.id_cultivo)
15     -> Hash Join (actual time=0.422..13.190 rows=20452 loops=1)
16         Hash Cond: (p.id_fundo = f.id_fundo)
17     -> Hash Join (actual time=0.414..10.296 rows=20452 loops=1)
18         Hash Cond: (ca.id_parcela = p.id_parcela)
19     -> Hash Join (actual time=0.296..7.078 rows=20452 loops=1)
20         Hash Cond: (co.id_campania = ca.id_campania)
21     -> Seq Scan on cosecha co (actual time=0.003..3.583 rows=20452 loops=1)
22         Filter: ((categoria)::text = ANY ('{Primera,Segunda}'::text[]))
23         Rows Removed by Filter: 4548
24     -> Hash (actual time=0.290..0.291 rows=1500 loops=1)
25         Buckets: 2048 Batches: 1 Memory Usage: 81kB
26     -> Seq Scan on campania ca (actual time=0.002..0.124 rows=1500 loops=1)
27     -> Hash (actual time=0.115..0.115 rows=600 loops=1)
28         Buckets: 1024 Batches: 1 Memory Usage: 37kB
29     -> Seq Scan on parcela p (actual time=0.002..0.054 rows=600 loops=1)
30     -> Hash (actual time=0.006..0.006 rows=12 loops=1)
31         Buckets: 1024 Batches: 1 Memory Usage: 9kB
32     -> Seq Scan on fundo f (actual time=0.002..0.004 rows=12 loops=1)
33     -> Hash (actual time=0.010..0.011 rows=12 loops=1)
34         Buckets: 1024 Batches: 1 Memory Usage: 9kB
35     -> Seq Scan on cultivo cu (actual time=0.004..0.006 rows=12 loops=1)
36 Planning Time: 0.471 ms
37 Execution Time: 26.434 ms

```

Plan de ejecución con índices (1M)

```

1  Sort (actual time=26.507..26.557 rows=902 loops=1)
2  Sort Key: rendimiento.fundo, (rank() OVER (?))
3  Sort Method: quicksort Memory: 111kB
4  -> WindowAgg (actual time=24.669..25.706 rows=902 loops=1)
5  -> WindowAgg (actual time=24.588..25.092 rows=902 loops=1)
6  -> Sort (actual time=24.581..24.632 rows=902 loops=1)
7      Sort Key: rendimiento.id_fundo, ((rendimiento.kilos / rendimiento.area_ha)) DESC
8      Sort Method: quicksort Memory: 104kB
9  -> Subquery Scan on rendimiento (actual time=22.929..23.485 rows=902 loops=1)
10     -> HashAggregate (actual time=22.926..23.267 rows=902 loops=1)
11         Group Key: f.id_fundo, cu.nombre_comun, ca.id_campania, p.area_ha
12         Batches: 1 Memory Usage: 1297kB
13     -> Hash Join (actual time=0.401..16.217 rows=20452 loops=1)
14         Hash Cond: (ca.id_cultivo = cu.id_cultivo)
15     -> Hash Join (actual time=0.387..13.325 rows=20452 loops=1)
16         Hash Cond: (p.id_fundo = f.id_fundo)
17     -> Hash Join (actual time=0.380..10.350 rows=20452 loops=1)
18         Hash Cond: (ca.id_parcela = p.id_parcela)
19     -> Hash Join (actual time=0.260..6.996 rows=20452 loops=1)
20         Hash Cond: (co.id_campania = ca.id_campania)
21     -> Seq Scan on cosecha co (actual time=0.003..3.539 rows=20452 loops=1)
22         Filter: ((categoria)::text = ANY ('{Primera,Segunda}'::text[]))
23         Rows Removed by Filter: 4548
24     -> Hash (actual time=0.253..0.254 rows=1500 loops=1)
25         Buckets: 2048 Batches: 1 Memory Usage: 81kB
26     -> Seq Scan on campania ca (actual time=0.002..0.121 rows=1500 loops=1)
27     -> Hash (actual time=0.117..0.118 rows=600 loops=1)
28         Buckets: 1024 Batches: 1 Memory Usage: 37kB
29     -> Seq Scan on parcela p (actual time=0.002..0.055 rows=600 loops=1)
30     -> Hash (actual time=0.005..0.006 rows=12 loops=1)
31         Buckets: 1024 Batches: 1 Memory Usage: 9kB
32     -> Seq Scan on fundo f (actual time=0.002..0.003 rows=12 loops=1)
33     -> Hash (actual time=0.011..0.011 rows=12 loops=1)
34         Buckets: 1024 Batches: 1 Memory Usage: 9kB
35     -> Seq Scan on cultivo cu (actual time=0.004..0.006 rows=12 loops=1)
36 Planning Time: 0.584 ms
37 Execution Time: 26.721 ms

```

5.6.5. Consulta 5

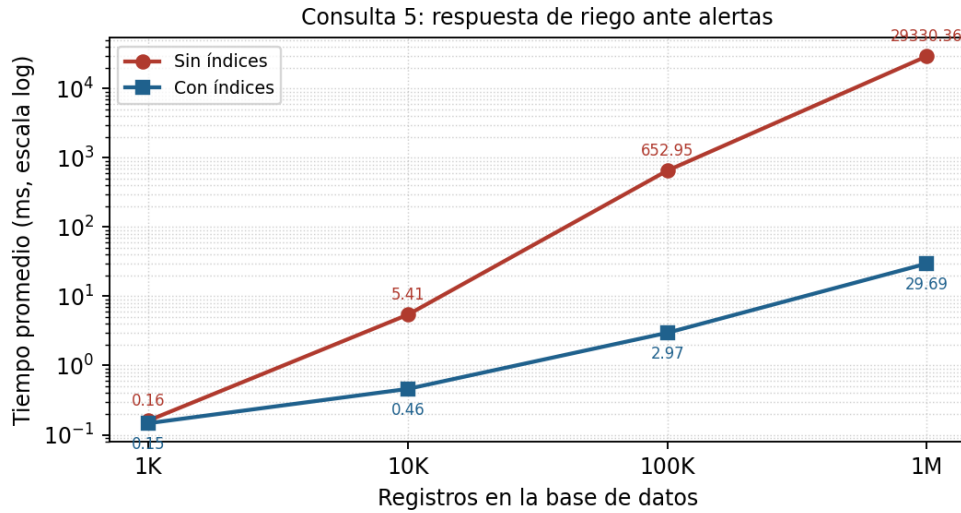


Figura 10: Consulta 5: tiempo promedio por escenario.

Escenario	Sin índices (ms)	Con índices (ms)	Relación sin/con	Filas devueltas
1K	0.160	0.147	1.09	1
10K	5.411	0.459	11.78	4
100K	652.95	2.968	220.01	8
1M	29 330.36	29.69	987.82	12

Tabla 49: Consulta 5: promedios y relación de mejora.

Es la mejora más grande del experimento: con un millón de registros la consulta pasa de 29 segundos a 30 milisegundos, casi mil veces más rápida. Sin índice, la subconsulta correlacionada ejecuta un recorrido secuencial de `Evento_Riego` por cada una de las 7714 alertas y descarta en promedio unas 60 000 filas en cada uno. Con índice, cada búsqueda es un *Index Only Scan* sobre `uq_evento` que tarda alrededor de un microsegundo. La brecha crece con el volumen: 12 veces en 10 000 registros, 220 veces en 100 000 y casi mil en un millón.

Plan de ejecución sin índices (1M)

```

1 Sort (actual time=29572.617..29572.626 rows=12 loops=1)
2 Sort Key: (round(((100.0 * (count(*) FILTER (WHERE (SubPlan 2))))::numeric) / (count(*)::numeric), 2))
3 Sort Method: quicksort Memory: 25kB
4 -> GroupAggregate (actual time=2290.335..29572.571 rows=12 loops=1)
5   Group Key: f.nombre
6   Filter: (count(*) > 3)
7   -> Sort (actual time=350.841..355.686 rows=7714 loops=1)
8     Sort Key: f.nombre
9     Sort Method: quicksort Memory: 614kB
10    -> Hash Join (actual time=343.289..349.062 rows=7714 loops=1)
11      Hash Cond: (p.id_fundo = f.id_fundo)
12      -> Hash Join (actual time=0.871..5.670 rows=7714 loops=1)
13        Hash Cond: (d.id_parcela = p.id_parcela)
14        -> Hash Join (actual time=0.735..4.518 rows=7714 loops=1)
15          Hash Cond: (a.id_dispositivo = d.id_dispositivo)
16          -> Seq Scan on alerta a (actual time=0.009..2.694 rows=7714 loops=1)
17            Filter: (((severidad)::text = ANY ('{Alta,Critica}'::text[])) AND ((tipo)::text = 'Humedad baja'::text))
18            Rows Removed by Filter: 22286
19          -> Hash (actual time=0.715..0.716 rows=2800 loops=1)
20            Buckets: 4096 Batches: 1 Memory Usage: 153kB
21            -> Seq Scan on dispositivo d (actual time=0.005..0.314 rows=2800 loops=1)
22            -> Hash (actual time=0.121..0.122 rows=600 loops=1)

```

```

23      Buckets: 1024 Batches: 1 Memory Usage: 32kB
24      -> Seq Scan on parcela p (actual time=0.013..0.063 rows=600 loops=1)
25      -> Hash (actual time=342.406..342.407 rows=12 loops=1)
26      Buckets: 1024 Batches: 1 Memory Usage: 9kB
27      -> Seq Scan on fundo f (actual time=342.375..342.382 rows=12 loops=1)
28  SubPlan 1
29      -> Seq Scan on evento_riego er (actual time=1.859..1.859 rows=1 loops=7714)
30      Filter: ((fecha_hora_inicio > a.fecha_hora) AND (id_parcela = d.id_parcela) AND (num_zona = d.num_zona) AND (fecha_hora_inicio <= (a.fecha_hora
31      + '06:00:00'::interval)))
32      Rows Removed by Filter: 60186
33  SubPlan 2
34      -> Seq Scan on evento_riego er_1 (actual time=1.921..1.921 rows=1 loops=7714)
35      Filter: ((fecha_hora_inicio > a.fecha_hora) AND (id_parcela = d.id_parcela) AND (num_zona = d.num_zona) AND (fecha_hora_inicio <= (a.fecha_hora
36      + '06:00:00'::interval)))
37      Rows Removed by Filter: 60186
38  Planning Time: 0.457 ms
39  Execution Time: 29574.874 ms

```

Plan de ejecución con índices (1M)

```

1  Sort (actual time=28.026..28.031 rows=12 loops=1)
2  Sort Key: (round(((100.0 * (count(*) FILTER (WHERE (SubPlan 2))::numeric) / (count(*)::numeric), 2))
3  Sort Method: quicksort Memory: 25kB
4  -> GroupAggregate (actual time=7.819..28.018 rows=12 loops=1)
5  Group Key: f.nombre
6  Filter: (count(*) > 3)
7  -> Sort (actual time=6.343..6.802 rows=7714 loops=1)
8  Sort Key: f.nombre
9  Sort Method: quicksort Memory: 614kB
10 -> Hash Join (actual time=0.718..4.859 rows=7714 loops=1)
11 Hash Cond: (p.id_fundo = f.id_fundo)
12 -> Hash Join (actual time=0.703..3.764 rows=7714 loops=1)
13 Hash Cond: (d.id_parcela = p.id_parcela)
14 -> Hash Join (actual time=0.580..2.580 rows=7714 loops=1)
15 Hash Cond: (a.id_dispositivo = d.id_dispositivo)
16 -> Index Only Scan using idx_alerta_humedad_baja on alerta a (actual time=0.018..0.908 rows=7714 loops=1)
17 Index Cond: (severidad = ANY ('{Alta,Critica}'::text[]))
18 Heap Fetches: 0
19 -> Hash (actual time=0.556..0.557 rows=2800 loops=1)
20 Buckets: 4096 Batches: 1 Memory Usage: 153kB
21 -> Seq Scan on dispositivo d (actual time=0.003..0.293 rows=2800 loops=1)
22 -> Hash (actual time=0.121..0.121 rows=600 loops=1)
23 Buckets: 1024 Batches: 1 Memory Usage: 32kB
24 -> Seq Scan on parcela p (actual time=0.002..0.057 rows=600 loops=1)
25 -> Hash (actual time=0.011..0.011 rows=12 loops=1)
26 Buckets: 1024 Batches: 1 Memory Usage: 9kB
27 -> Seq Scan on fundo f (actual time=0.004..0.006 rows=12 loops=1)
28 SubPlan 1
29 -> Index Only Scan using uq_evento on evento_riego er (actual time=0.001..0.001 rows=1 loops=7714)
30 Index Cond: ((id_parcela = d.id_parcela) AND (num_zona = d.num_zona) AND (fecha_hora_inicio > a.fecha_hora) AND (fecha_hora_inicio <= (a.
31 fecha_hora + '06:00:00'::interval)))
32 Heap Fetches: 0
33 SubPlan 2
34 -> Index Only Scan using uq_evento on evento_riego er_1 (actual time=0.001..0.001 rows=1 loops=7714)
35 Index Cond: ((id_parcela = d.id_parcela) AND (num_zona = d.num_zona) AND (fecha_hora_inicio > a.fecha_hora) AND (fecha_hora_inicio <= (a.
36 fecha_hora + '06:00:00'::interval)))
37 Heap Fetches: 0
38 Planning Time: 0.533 ms
39 Execution Time: 28.082 ms

```

5.7. Análisis y Discusión

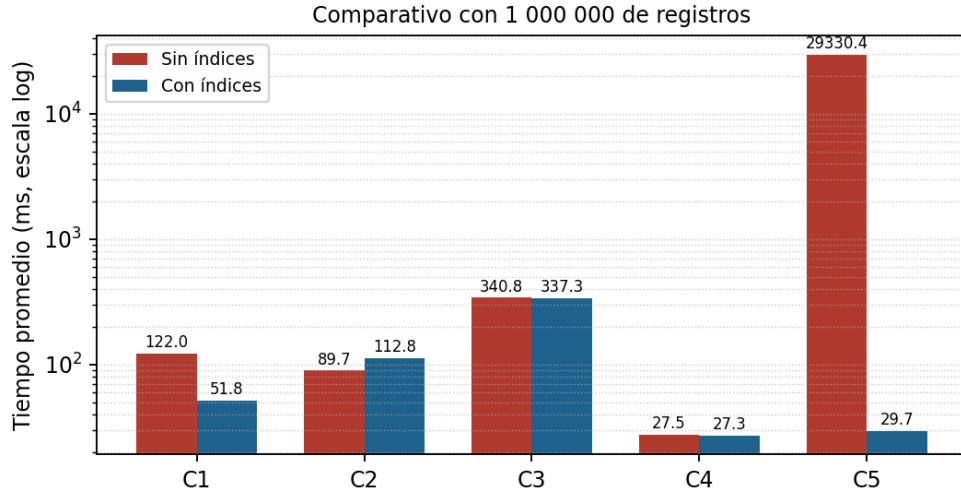


Figura 11: Comparación de las cinco consultas con un millón de registros.

Consulta	1K	10K	100K	1M
C1	1.06	1.37	1.70	2.36
C2	1.06	1.03	1.07	0.80
C3	1.09	1.02	0.97	1.01
C4	1.00	1.03	1.01	1.01
C5	1.09	11.78	220.01	987.82

Tabla 50: Relación entre el tiempo sin índices y con índices. Un valor mayor que 1 indica mejora.

Los índices no mejoran todo. De las cinco consultas, dos mejoran de forma clara, dos no cambian y una empeora. De los nueve índices propuestos, el planificador solo eligió tres en el escenario de un millón (`idx_lectura_fecha`, `idx_alerta_humedad_baja` e `idx_evento_parcela_fecha`), y el tercero hizo más lenta la consulta 2. Los otros seis ocupan espacio y encarecen las escrituras sin aportar nada a estas consultas. La Tabla 51 resume qué método de acceso usó el plan en cada escenario.

Consulta	Índices	1K	10K	100K	1M
C1	sin	Seq	Seq	Seq	Seq paralelo
	con	Index (idx_lectura_fecha)	Only (idx_lectura_fecha)	Index (idx_lectura_fecha)	Only (idx_lectura_fecha)
C2	sin	Seq	Seq	Seq	Seq paralelo
	con	Seq	Seq	Seq	Nested + Loop Index (idx_evento_parcela_fecha)
C3	sin	Seq	Seq	Seq	Seq paralelo
	con	Seq	Seq	Seq	Seq paralelo
C4	sin	Seq	Seq	Seq	Seq
	con	Seq	Seq	Seq	Seq
C5	sin	Seq	Seq	Seq	Seq
	con	Index (uq_evento)	Only (uq_evento)	Index (uq_evento)	Only (uq_evento)

Tabla 51: Método de acceso a la tabla principal de cada consulta (C1 y C3: lecturas; C2 y C5: riegos; C4: cosechas).

Cuándo ayuda un índice. Ayuda cuando la consulta necesita una fracción pequeña de una tabla grande (consulta 1, el último mes) o cuando debe repetir muchas búsquedas puntuales (consulta 5, una por alerta). No ayuda cuando la consulta lee casi toda la tabla (consultas 3 y 4) ni cuando la tabla es tan pequeña que cabe en pocas páginas, como **Campania** con 1 500 filas. En los escenarios de mil y diez mil registros casi no hay diferencias por la misma razón: todo cabe en memoria y cualquier plan termina en fracciones de milisegundo.

El planificador puede equivocarse. La consulta 2 muestra que un índice razonable puede empeorar el tiempo. El planificador elige con estimaciones de costo, y cuando la estimación falla, como con el *Memoize* sin aciertos, el plan resultante es peor. Además, el efecto se invirtió entre escenarios: con 100 000 registros el índice ayudaba un poco. Cada índice debe validarse con **EXPLAIN ANALYZE** en un volumen representativo, no darse por bueno en teoría.

Optimización lógica y física. La consulta 5 repite la misma subconsulta **EXISTS** en dos lugares, así que se evalúa dos veces por alerta. La reescribimos para evaluarla una sola vez dentro de una CTE (Tabla 52).

Versión de la consulta 5	Sin índices (ms)	Con índices (ms)
Original (EXISTS en dos lugares)	29 944.72 (5 it.)	29.08 (5 it.)
CTE en línea	29 723.89 (5 it.)	28.77 (5 it.)
CTE con AS MATERIALIZED	15 333.69 (3 it.)	21.58 (5 it.)

Tabla 52: Efecto de reescribir la consulta 5 (1M). Las tres versiones devuelven exactamente las mismas filas.

La primera reescritura no mejoró nada, y la causa es una característica de PostgreSQL que conviene conocer: desde la versión 12, una CTE que se usa una sola vez se incrusta en la consulta principal, y al hacerlo la expresión **EXISTS** volvió a aparecer en los dos agregados. El plan lo confirma: siguen existiendo dos subconsultas y no aparece ningún *CTE Scan*. Al declarar la CTE como **MATERIALIZED**, la subconsulta se evalúa una sola vez y el tiempo sin índices baja a la mitad. Es una mejora real, pero pequeña frente al índice: reescribir divide el tiempo entre dos y el índice lo divide entre mil. Lo ideal

es combinar ambas cosas.

03c_consulta5_materializada.sql

```

1  -- Consulta 5 reescrita con AS MATERIALIZED: impide que PostgreSQL incruste la
2  -- CTE y garantiza que el EXISTS se evalúe una sola vez por alerta.
3  WITH evaluadas AS MATERIALIZED (
4      SELECT f.nombre AS fundo,
5             EXISTS (
6                 SELECT 1
7                 FROM Evento_Riego er
8                 WHERE er.id_parcela = d.id_parcela
9                       AND er.num_zona = d.num_zona
10                      AND er.fecha_hora_inicio > a.fecha_hora
11                      AND er.fecha_hora_inicio <= a.fecha_hora + INTERVAL '6 hours'
12             ) AS con_riego
13     FROM Alerta a
14     JOIN Dispositivo d ON d.id_dispositivo = a.id_dispositivo
15     JOIN Parcela p ON p.id_parcela = d.id_parcela
16     JOIN Fundo f ON f.id_fundo = p.id_fundo
17     WHERE a.tipo = 'Humedad baja'
18           AND a.severidad IN ('Alta', 'Critica')
19 )
20 SELECT fundo,
21        COUNT(*) AS alertas,
22        COUNT(*) FILTER (WHERE con_riego) AS atendidas_con_riego,
23        ROUND(100.0 * COUNT(*) FILTER (WHERE con_riego) / COUNT(*), 2) AS pct_respuesta
24 FROM evaluadas
25 GROUP BY fundo
26 HAVING COUNT(*) > 3
27 ORDER BY pct_respuesta ASC;

```

Resumen práctico. Para esta base recomendamos conservar `idx_lectura_fecha` (o su versión BRIN si el espacio importa) e `idx_alerta_humedad_baja`, y retirar los otros siete. En particular, `idx_lectura_disp_humedad` ocupa 24 MB, nunca se usa y encarece cada actualización de humedad.

5.8. Complejidad operacional por encima del millón de registros

Cómo crece cada consulta. La Tabla 53 muestra cuántas veces aumenta el tiempo cuando los datos se multiplican por diez. Si el factor es cercano a 10, el crecimiento es lineal; si se acerca a 100, es cuadrático. El exponente b resume el último tramo: el tiempo crece como n^b .

Consulta	Índices	1K→10K	10K→100K	100K→1M	Exponente b
C1	sin	3.7	8.0	12.6	1.10
	con	2.9	6.4	9.1	0.96
C2	sin	4.3	8.3	10.4	1.02
	con	4.4	7.9	14.0	1.15
C3	sin	7.0	10.2	11.3	1.05
	con	7.5	10.7	10.8	1.03
C4	sin	3.1	5.9	7.2	0.86
	con	3.0	6.0	7.2	0.86
C5	sin	33.9	120.7	44.9	1.65
	con	3.1	6.5	10.0	1.00

Tabla 53: Factor de crecimiento del tiempo cuando los datos se multiplican por diez.

Casi todas las consultas crecen de forma lineal ($b \approx 1$), lo que corresponde a recorridos completos

de tabla con uniones hash: el costo es proporcional al número de filas. La excepción es la consulta 5 sin índice, con exponentes entre 1.6 y 2.1. La razón es que el número de alertas y el de riegos crecen a la vez, y por cada alerta se recorre la tabla de riegos: el costo es $O(A \cdot E)$, que en la práctica es cuadrático. Con el índice, cada búsqueda cuesta $O(\log E)$ y el total queda en $O(A \log E)$, casi lineal, como confirma el exponente de 1.0.

Proyección. Si extrapolamos esos exponentes, con diez millones de registros la consulta 5 sin índice pasaría de 30 segundos a entre 20 y 60 minutos, mientras que con índice rondaría los 300 ms. La consulta 1 sin índice subiría a alrededor de un segundo y medio. Son estimaciones gruesas, porque al crecer la base dejaría de caber en memoria y aparecerían lecturas de disco que hoy no existen, pero el orden de magnitud basta para ver que los recorridos completos dejan de ser una opción.

Volumen real esperado. Con los 2 200 sensores reportando cada 15 minutos, la tabla de lecturas recibiría unos 77.1 millones de filas al año (alrededor de 13.2 GB con los índices actuales). El ritmo de escritura no es el problema: equivale a unas 2.4 lecturas por segundo, y en la prueba de escritura insertamos 20 000 lecturas en 238 ms, unas 84 000 por segundo con todos los índices. Aunque registrar fila por fila con el trigger de alertas es mucho más lento que una inserción masiva, el margen es de varios órdenes de magnitud. El problema está en las consultas analíticas sobre años de historia. Incluso la consulta 1 con índice escala con el tamaño de la ventana consultada: con la frecuencia real, un mes tendría unos 6.3 millones de lecturas, y leerlas por índice tomaría varios segundos.

¿Es suficiente la arquitectura cliente-servidor? Para el volumen de esta empresa, sí, siempre que el servidor se organice para ese crecimiento. En orden de prioridad:

1. **Particionamiento por rango de fecha.** Dividir `Lectura_Suelo` en particiones mensuales con el particionamiento declarativo de PostgreSQL. Una consulta del último mes solo toca una partición (*partition pruning*), y las particiones antiguas pueden archivarse o desconectarse sin borrar fila por fila.
2. **BRIN por partición.** Como los datos llegan en orden cronológico, un BRIN sobre la fecha cuesta kilobytes en lugar de megabytes y casi iguala al B+ tree, como medimos.
3. **Agregados materializados.** Una vista materializada con el resumen diario por sensor (promedio, mínimo, máximo y conteo), refrescada cada noche. Las consultas 1 y 3 leerían unas 2 200 filas por día en lugar de 211 000 lecturas.
4. **Réplica de lectura.** Una réplica por *streaming replication* para los reportes de gerencia, de modo que las consultas pesadas no compitan con la ingesta.
5. **Extensiones especializadas.** Si el volumen creciera mucho más, TimescaleDB añade a PostgreSQL particionamiento automático, compresión y agregados continuos para series de tiempo, sin salir del modelo relacional.

El modelo cliente-servidor dejaría de ser suficiente en otro escenario: una plataforma que atienda a miles de fundos de distintas empresas, con miles de millones de filas por año, alta disponibilidad entre regiones o análisis que recorran terabytes. Ahí el lado del servidor tendría que convertirse en un clúster distribuido o apoyarse en un almacén columnar para la analítica. Aun así, los clientes seguirían hablando con un servicio; lo que cambia es que ese servicio ya no es una sola máquina.

6. Conclusiones

1. El modelo cubre el dominio con una especialización total y disjunta, dos entidades débiles y una relación ternaria. El esquema resultante está en tercera forma normal. La única tabla que no cumple FNBC (**Evento_Riego**) se dejó así a propósito para preservar dependencias, y la dependencia se controla con un trigger.
2. Llevar las reglas del negocio a la base de datos funcionó. Restricciones, procedimientos y triggers rechazaron todas las operaciones inválidas de las pruebas, y las verificaciones de consistencia encontraron errores del generador que habrían contaminado el experimento.
3. Los índices no son una mejora universal. Solo dos de las cinco consultas se aceleraron (2.4 y casi mil veces), dos no cambiaron y una se volvió 26 % más lenta. Solo tres de los nueve índices propuestos llegaron a usarse.
4. La mayor ganancia aparece en las subconsultas correlacionadas: el índice convierte un costo cuadrático en uno casi lineal, y la diferencia crece con el volumen.
5. Para series de tiempo guardadas en orden de llegada, un BRIN ofrece casi el mismo beneficio que un B+ tree con una fracción mínima del espacio. En un sistema IoT que crece de forma indefinida, esa diferencia pesa.
6. Todo índice cuesta en escritura, entre 12 y 75 % en nuestras pruebas, y el peor caso aparece al actualizar columnas incluidas en índices. En tablas que reciben datos de forma continua, cada índice debe justificarse con mediciones.
7. La optimización lógica también cuenta, pero hay que verificarla. Reescribir la consulta 5 no sirvió hasta forzar la materialización de la CTE; después, redujo el tiempo a la mitad.
8. La arquitectura cliente-servidor basta para el volumen previsto si se combina con particionamiento por fecha, índices BRIN y agregados materializados. Las alternativas distribuidas solo se justificarían con una escala varios órdenes de magnitud mayor.

7. Anexos

7.1. Video de demostración

Video con la base de un millón de registros, ejecutando las consultas con y sin índices: [\[pegar aquí el enlace al video\]](#)

Los comandos que se muestran en el video son los siguientes:

```

1  -- 1. Restaurar el respaldo (desde la terminal)
2  --   createdb agro1m
3  --   pg_restore -d agro1m agro_1M.dump
4  -- 2. Comprobar el volumen
5  SELECT (SELECT count(*) FROM Lectura_Suelo) AS lecturas,
6         (SELECT count(*) FROM Evento_Riego) AS riegos,
7         (SELECT count(*) FROM Alerta)      AS alertas;
8  -- 3. Consulta 5 sin indices
9  SET enable_indexscan = OFF; SET enable_bitmapscan = OFF; SET enable_indexonlyscan = OFF;
10 EXPLAIN ANALYZE <consulta 5>;
11 -- 4. Consulta 5 con indices
12 SET enable_indexscan = ON; SET enable_bitmapscan = ON; SET enable_indexonlyscan = ON;
13 EXPLAIN ANALYZE <consulta 5>;

```

7.2. Respaldos de la base de datos

Los cuatro respaldos se generaron con `pg_dump` en formato personalizado y compresión nivel 6: [\[pegar aquí el enlace a la carpeta de dumps\]](#)

Archivo	Registros	Tamaño (MB)
agro_1K.dump	1 000	0.13
agro_10K.dump	10 000	0.25
agro_100K.dump	100 000	1.47
agro_1M.dump	1 000 000	13.74

Tabla 54: Respaldos entregados.

Para restaurar uno de ellos: `createdb agro1m` y luego `pg_restore -d agro1m agro_1M.dump`.

7.3. Scripts del proyecto

Carpeta con todos los scripts: [\[pegar aquí el enlace a la carpeta de scripts\]](#)

Archivo	Contenido
sql/01_ddl.sql	Creación de las 18 tablas con sus restricciones.
sql/01b_comentarios.sql	Documentación de tablas y columnas (fuente del diccionario).
sql/02_vistas_procedimientos_triggers.sql	Vistas, procedimientos, funciones y triggers.
sql/03_consultas.sql	Las cinco consultas del experimento.
sql/03b_consulta5_reescrita.sql, 03c...	Reescrituras de la consulta 5.
sql/04_indices.sql	Los nueve índices de optimización.
sql/05_roles.sql	Roles y permisos (opcional).
sql/06_pruebas.sql	Pruebas de procedimientos y triggers.
sql/07_verificacion.sql	Verificación de consistencia posterior a la carga.
scripts/generar_datos.py	Generador de datos sintéticos.
scripts/ejecutar_todo.py	Orquestador: carga, experimento y gráficos.
scripts/experimento.py	Mediciones, reanudables por tramos.
scripts/graficos.py	Figuras del informe.
scripts/construir_informe.py	Genera las tablas y listados de este informe desde los resultados.

Tabla 55: Archivos entregados.

7.4. Generador de datos

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Generador de datos sintéticos - Agricultura de Precisión y Sensores IoT
5  Curso: Base de Datos I (CS2041) - Laboratorio 14 - 2026-2
6
7  Uso:  python generar_datos.py <total_registros> <carpeta_salida>
8  Ej.:  python generar_datos.py 1000000 ../data/1M
9
10 Produce un CSV por tabla. La suma de filas de todas las tablas es exactamente
11 <total_registros>; la tabla Lectura_Suelo absorbe el saldo.

```

```

12 Los parametros agronomicos (rangos optimos, rendimientos, caudales) se tomaron
13 de valores de referencia publicos para la agricultura peruana.
14 """
15 import csv, os, random, sys
16 from datetime import date, datetime, timedelta
17
18 SEED = 20251047
19 INICIO_SERIE = datetime(2026, 1, 1)
20 FIN_SERIE = datetime(2026, 9, 15, 23, 59)
21 VENTANA_SEG = int((FIN_SERIE - INICIO_SERIE).total_seconds())
22 REF = date(2026, 9, 20) # fecha de corte del caso de estudio
23 NULO = "" # NULL para COPY ... NULL ''
24
25 # departamento, provincia, distrito, lat, lon, temperatura media del suelo (C)
26 DEPARTAMENTOS = [
27     ("Ica", "Ica", "Salas Guadalupe", -14.07, -75.73, 23),
28     ("La Libertad", "Viru", "Chao", -8.48, -78.72, 22),
29     ("Lambayeque", "Chiclayo", "Motupe", -6.15, -79.72, 24),
30     ("Piura", "Sullana", "Bellavista", -4.90, -80.69, 26),
31     ("Arequipa", "Caylloma", "Majes", -16.33, -72.19, 18),
32     ("Ancash", "Casma", "Comandante Noel", -9.47, -78.30, 21),
33     ("Lima", "Huaral", "Chancay", -11.57, -77.27, 19),
34     ("Junin", "Chanchamayo", "Perene", -10.95, -75.22, 24),
35     ("San Martin", "Lamas", "Tabalosos", -6.42, -76.63, 26),
36     ("Cusco", "La Convencion", "Echarate", -12.77, -72.57, 24),
37     ("Cajamarca", "Jaen", "Bellavista", -5.67, -78.67, 25),
38     ("Amazonas", "Utcubamba", "Bagua Grande", -5.76, -78.44, 26),
39 ]
40 NOMBRES_FUNDO = ["Santa Rita", "El Algarrobal", "Los Olivares", "Pampa Verde",
41                 "San Jacinto", "La Huaca", "Tierra Nueva", "El Mirador",
42                 "Valle Alto", "Sol de Oro", "Las Lomas", "Buenaventura"]
43
44 # nombre, cientifico, ciclo, hmin, hmax, phmin, phmax, ce_maz, t/ha, S/ por kg
45 CULTIVOS = [
46     ("Esparrago", "Asparagus officinalis", 270, 55, 75, 6.5, 7.5, 3.0, 11, 5.5),
47     ("Arandano", "Vaccinium corymbosum", 365, 60, 80, 4.5, 5.5, 1.5, 14, 12.0),
48     ("Palto Hass", "Persea americana", 420, 55, 70, 5.5, 7.0, 2.0, 12, 6.0),
49     ("Uva de mesa", "Vitis vinifera", 300, 45, 65, 6.0, 7.5, 2.5, 25, 7.0),
50     ("Mango Kent", "Mangifera indica", 330, 50, 70, 5.5, 7.5, 2.0, 16, 2.5),
51     ("Quinua", "Chenopodium quinoa", 150, 40, 60, 6.0, 8.0, 4.0, 2, 6.0),
52     ("Papa Canchan", "Solanum tuberosum", 120, 60, 80, 5.0, 6.5, 2.0, 22, 1.2),
53     ("Maiz amarillo", "Zea mays", 140, 55, 75, 5.8, 7.0, 3.5, 9, 1.1),
54     ("Cacao", "Theobroma cacao", 400, 65, 85, 5.0, 7.5, 1.5, 1, 9.0),
55     ("Cafe Caturra", "Coffea arabica", 390, 60, 80, 4.5, 6.0, 1.5, 1, 14.0),
56     ("Alcachofa", "Cynara scolymus", 180, 60, 75, 6.0, 7.5, 3.0, 18, 2.2),
57     ("Pimiento piquillo", "Capsicum annum", 130, 55, 70, 6.0, 7.0, 2.5, 28, 2.8),
58 ]
59 # cultivos aptos por departamento (se evita, por ejemplo, cacao en Ica)
60 APTOS = {
61     "Ica": ["Esparrago", "Uva de mesa", "Palto Hass", "Arandano", "Pimiento piquillo"],
62     "La Libertad": ["Esparrago", "Arandano", "Palto Hass", "Alcachofa", "Pimiento piquillo"],
63     "Lambayeque": ["Mango Kent", "Palto Hass", "Maiz amarillo", "Uva de mesa"],
64     "Piura": ["Mango Kent", "Uva de mesa", "Arandano", "Maiz amarillo"],
65     "Arequipa": ["Quinua", "Alcachofa", "Papa Canchan", "Palto Hass"],
66     "Ancash": ["Esparrago", "Palto Hass", "Maiz amarillo", "Mango Kent"],
67     "Lima": ["Palto Hass", "Maiz amarillo", "Papa Canchan", "Alcachofa"],
68     "Junin": ["Cafe Caturra", "Cacao", "Maiz amarillo"],
69     "San Martin": ["Cacao", "Cafe Caturra", "Maiz amarillo"],
70     "Cusco": ["Cafe Caturra", "Cacao"],
71     "Cajamarca": ["Cafe Caturra", "Cacao", "Maiz amarillo"],
72     "Amazonas": ["Cafe Caturra", "Cacao", "Maiz amarillo"],
73 }
74
75 INSUMOS = [ # nombre, tipo, unidad, costo, carencia
76     ("Nitrate de amonio 33.5", "Fertilizante", "kg", 3.20, 0),
77     ("Sulfato de potasio", "Fertilizante", "kg", 4.80, 0),
78     ("Fosfato diamonico", "Fertilizante", "kg", 4.10, 0),
79     ("Acido fosforico 85%", "Fertilizante", "L", 6.50, 0),
80     ("Yeso agricola", "Enmienda", "kg", 0.90, 0),
81     ("Azufre micronizado", "Enmienda", "kg", 2.30, 3),
82     ("Mancozeb 80 WP", "Fungicida", "kg", 28.00, 7),
83     ("Azoxistrobina 25 SC", "Fungicida", "L", 95.00, 14),
84     ("Abamectina 1.8 EC", "Insecticida", "L", 78.00, 7),
85     ("Imidacloprid 35 SC", "Insecticida", "L", 64.00, 21),
86     ("Extracto de algas", "Bioestimulante", "L", 42.00, 0),
87     ("Aminoacidos libres 20%", "Bioestimulante", "L", 55.00, 0),
88     ("Quelato de hierro EDDHA", "Fertilizante", "kg", 38.00, 0),
89     ("Cal dolomita", "Enmienda", "kg", 0.65, 0),
90     ("Bacillus subtilis WP", "Fungicida", "kg", 61.00, 0),
91 ]
92
93 DOSIS = {"Fertilizante": (25, 400), "Enmienda": (100, 900), "Fungicida": (0.3, 6),
94         "Insecticida": (0.2, 4), "Bioestimulante": (1, 15)}
95
96 METODOS = {"Fertilizante": ["Fertirriego", "Al suelo"], "Enmienda": ["Al suelo"],
97           "Fungicida": ["Foliar", "Drench"], "Insecticida": ["Foliar", "Drench"],
98           "Bioestimulante": ["Foliar", "Fertirriego"]}
99
100 # tipo de sensor -> variables que mide y modelos comerciales compatibles
101 SENSORES = {
102     "Humedad": ({ "hum", "temp"}, [{"SMT100", "TRUEBNER"}, {"Watermark 200SS", "Irrrometer"}]),
103     "Multiparametro": ({ "hum", "temp", "ph", "ce"}, [{"JXBS-3001", "JXCT"}]),
104     "Conductividad": ({ "hum", "temp", "ce"}, [{"TEROS 12", "METER Group"},
105         {"HydraProbe", "Stevens Water"}]),
106     "pH": ({ "ph", "temp"}, [{"Sonda pH de suelo", "JXCT"}]),
107     "Temperatura": ({ "temp"}, [{"107", "Campbell Scientific"}]),

```

```

106 }
107 PESOS_SENSOR = {"Humedad": 35, "Multiparametro": 40, "Conductividad": 10, "pH": 10,
108                 "Temperatura": 5}
109 UNIDAD = {"Humedad": "%", "Multiparametro": "mixto", "Conductividad": "dS/m",
110           "pH": "pH", "Temperatura": "C"}
111 CONTROLADORES = [{"NMG-Pro", "Netafim"}, {"ACC2", "Hunter"},
112                  {"ESP-LXME2", "Rain Bird"}, {"GSI", "Galcon"}]
113
114 NOMBRES = ["Juan", "Maria", "Jose", "Rosa", "Carlos", "Ana", "Luis", "Carmen", "Miguel",
115            "Elena", "Pedro", "Silvia", "Jorge", "Lucia", "Victor", "Gladys", "Raul", "Martha",
116            "Cesar", "Nancy", "Felipe", "Yolanda", "Hugo", "Teresa", "Marco", "Julia", "Oscar",
117            "Beatriz", "Angel", "Pilar"]
118 APELLIDOS = ["Quispe", "Mamani", "Flores", "Rojas", "Vasquez", "Huaman", "Chavez", "Ramos",
119              "Castillo", "Torres", "Vargas", "Espinoza", "Salazar", "Cordova", "Tapia",
120              "Ccahuana", "Zevallos", "Palomino", "Bustamante", "Aguirre", "Nunez", "Paredes",
121              "Guzman", "Lescano", "Meza"]
122 ROLES = ["Tecnico de campo", "Ingeniero agronomo", "Operador de riego", "Supervisor"]
123 SUELOS = ["Arenoso", "Franco", "Franco arenoso", "Franco arcilloso", "Arcilloso"]
124 TIPOS_RIEGO = ["Goteo", "Aspersión", "Microaspersión", "Gravedad tecnificada"]
125 FUENTES = ["Pozo", "Canal", "Reservorio", "Rio"]
126 VALVULAS = ["Solenoides", "Hidraulica", "Manual motorizada"]
127 CATEGORIAS = ["Primera", "Segunda", "Tercera", "Descarte"]
128
129 PERFILES = {
130     1_000: dict(fundo=3, cultivo=8, insumo=10, operario=12, parcela=15, zona=40,
131                campania=25, sensor=35, controlador=10, evento=120, alerta=30,
132                cosecha=40, aplicacion=40, mantenimiento=20),
133     10_000: dict(fundo=5, cultivo=10, insumo=12, operario=40, parcela=50, zona=150,
134                 cosecha=350, aplicacion=400, mantenimiento=150),
135     100_000: dict(fundo=8, cultivo=12, insumo=15, operario=150, parcela=200, zona=700,
136                  campania=400, sensor=620, controlador=180, evento=12_000, alerta=3_000,
137                  cosecha=3_000, aplicacion=4_000, mantenimiento=1_200),
138     1_000_000: dict(fundo=12, cultivo=12, insumo=15, operario=400, parcela=600, zona=2_400,
139                    campania=1_500, sensor=2_200, controlador=600, evento=120_000,
140                    alerta=30_000, cosecha=25_000, aplicacion=40_000, mantenimiento=8_000),
141 }
142
143
144
145 def w(carpeta, nombre, cabecera, filas):
146     with open(os.path.join(carpeta, nombre + ".csv"), "w", newline="", encoding="utf-8") as fh:
147         esc = csv.writer(fh)
148         esc.writerow(cabecera)
149         esc.writerows(filas)
150     return len(filas)
151
152
153 def ts(dt):
154     return dt.strftime("%Y-%m-%d %H:%M:%S")
155
156
157 def instante():
158     t = INICIO_SERIE + timedelta(seconds=random.randint(0, VENTANA_SEG))
159     return t.replace(second=0, microsecond=0)
160
161
162 def generar(total, carpeta):
163     random.seed(SEED + total)
164     os.makedirs(carpeta, exist_ok=True)
165     p = PERFILES[total]
166     p["sistema"] = p["parcela"] # un sistema de riego por parcela
167     n = {}
168
169     # ----- FUNDO
170     fondos = []
171     for i in range(1, p["fundo"] + 1):
172         dep, prov, dist, lat, lon, _ = DEPARTAMENTOS[i - 1]
173         fondos.append([i, NOMBRES_FUNDO[i - 1], dep, prov, dist,
174                       round(random.uniform(60, 900), 2),
175                       round(lat + random.uniform(-0.2, 0.2), 6),
176                       round(lon + random.uniform(-0.2, 0.2), 6),
177                       (date(2019, 1, 1) + timedelta(days=random.randint(0, 2000))).isoformat())
178     n["Fundo"] = w(carpeta, "fundo", ["id_fundo", "nombre", "departamento", "provincia",
179                                     "distrito", "area_total_ha", "latitud", "longitud", "fecha_registro"], fondos)
180     dep_fundo = {f[0]: f[2] for f in fondos}
181     temp_dep = {d[0]: d[5] for d in DEPARTAMENTOS}
182
183     # ----- CULTIVO
184     cultivos = [[i + list(c[:8]) for i, c in enumerate(CULTIVOS[:p["cultivo"]], start=1)]]
185     n["Cultivo"] = w(carpeta, "cultivo", ["id_cultivo", "nombre_comun", "nombre_cientifico",
186                                         "ciclo_dias", "humedad_optima_min", "humedad_optima_max", "ph_optimo_min",
187                                         "ph_optimo_max", "ce_maxima_ds_m"], cultivos)
188     info = {i: CULTIVOS[i - 1] for i in range(1, p["cultivo"] + 1)}
189     id_por_nombre = {CULTIVOS[i - 1][0]: i for i in info}
190
191     # ----- INSUMO
192     insumos = [[i + list(x) for i, x in enumerate(INSUMOS[:p["insumo"]], start=1)]]
193     n["Insumo"] = w(carpeta, "insumo", ["id_insumo", "nombre", "tipo", "unidad",
194                                         "costo_unitario", "periodo_carencia_dias"], insumos)
195     tipo_insumo = {x[0]: x[2] for x in insumos}
196
197     # ----- OPERARIO
198     dnis, operarios = set(), []
199     for i in range(1, p["operario"] + 1):

```

```

200     while True:
201         d = str(random.randint(10000000, 79999999))
202         if d not in dnis:
203             dnis.add(d)
204             break
205     tel = NULO if random.random() < 0.08 else "9" + str(random.randint(10000000, 99999999))
206     operarios.append([i, d, random.choice(NOMBRES), random.choice(APELLIDOS),
207                     random.choice(ROLES), tel, random.randint(1, p["fundo"]),
208                     (date(2018, 1, 1) + timedelta(days=random.randint(0, 2800))).isoformat()])
209 n["Operario"] = w(carpeta, "operario", ["id_operario", "dni", "nombre", "apellido", "rol",
210                                     "telefono", "id_fundo", "fecha_ingreso"], operarios)
211
212 # ----- PARCELA
213 parcelas, cont = [], {}
214 for i in range(1, p["parcela"] + 1):
215     f = random.randint(1, p["fundo"])
216     cont[f] = cont.get(f, 0) + 1
217     parcelas.append([i, f, "P-%03d" % cont[f], round(random.uniform(1.5, 35.0), 2),
218                    random.choice(SUELOS), round(random.uniform(0, 12), 1),
219                    (date(2020, 1, 1) + timedelta(days=random.randint(0, 1800))).isoformat()])
220 n["Parcela"] = w(carpeta, "parcela", ["id_parcela", "id_fundo", "codigo_parcela", "area_ha",
221                                     "tipo_suelo", "pendiente_pct", "fecha_habilitacion"], parcelas)
222 area = {r[0]: float(r[3]) for r in parcelas}
223 fundo_parcela = {r[0]: r[1] for r in parcelas}
224
225 # ----- ZONA MANEJO
226 zonas, zxp = [], {}
227 for idp in range(1, p["parcela"] + 1):
228     zxp[idp] = [1]
229     zonas.append([idp, 1, round(area[idp] / 2, 2), "Zona de manejo 1"])
230 while len(zonas) < p["zona"]:
231     idp = random.randint(1, p["parcela"])
232     k = len(zxp[idp]) + 1
233     if k > 8:
234         continue
235     zxp[idp].append(k)
236     zonas.append([idp, k, round(max(0.2, area[idp] / (k + 1)), 2), "Zona de manejo %d" % k])
237 n["Zona_Manejo"] = w(carpeta, "zona_manejo", ["id_parcela", "num_zona", "area_ha",
238                                             "descripcion"], zonas)
239 lista_zonas = [(z[0], z[1]) for z in zonas]
240
241 # ----- SISTEMA RIEGO
242 # caudal de 15 a 180 m3/h, coherente con riego sectorizado de 1.5 a 35 ha
243 sistemas, sis_parcela, caudal = [], {}, {}
244 for i in range(1, p["sistema"] + 1):
245     c = round(random.uniform(15000, 180000), 2)
246     sistemas.append([i, i, random.choice(TIPOS_RIEGO), c, random.choice(FUENTES),
247                    (date(2021, 1, 1) + timedelta(days=random.randint(0, 1500))).isoformat(),
248                    "Operativo" if random.random() > 0.07 else "Averiado"])
249     sis_parcela[i] = i
250     caudal[i] = c
251 n["Sistema_Riego"] = w(carpeta, "sistema_riego", ["id_sistema", "id_parcela", "tipo",
252                                             "caudal_nominal_lph", "fuente_agua", "fecha_instalacion", "estado"],
253                     sistemas)
254
255 # ----- CAMPANIA
256 # Cadenas hacia atras desde REF: la primera campania de cada parcela es la
257 # vigente y las anteriores se encadenan sin solaparse.
258 campanias, cursor = [], {}
259 i, idp = 0, 0
260 while i < p["campania"]:
261     idp = idp % p["parcela"] + 1
262     dep = dep_fundo[fundo_parcela[idp]]
263     aptos = [id_por_nombre[c] for c in APTOS[dep] if c in id_por_nombre or list(info)
264            idc = random.choice(aptos)
265            ciclo = info[idc][2]
266            if idp not in cursor:
267                siembra = REF - timedelta(days=int(ciclo * random.uniform(0.15, 0.85)))
268                estado = "En curso" if random.random() > 0.10 else "Perdida"
269            else:
270                fin_prev = cursor[idp] - timedelta(days=random.randint(10, 60))
271                siembra = fin_prev - timedelta(days=ciclo)
272                estado = "Cerrada" if random.random() > 0.08 else "Perdida"
273            fin = siembra + timedelta(days=ciclo)
274            cursor[idp] = siembra
275            i += 1
276            campanias.append([i, idp, idc, siembra.isoformat(), fin.isoformat(),
277                           random.choice([1200, 1600, 2500, 3300, 5000, 8000, 62500]), estado, 0])
278 n["Campania"] = w(carpeta, "campania", ["id_campania", "id_parcela", "id_cultivo",
279                                     "fecha_siembra", "fecha_fin_estimada", "densidad_plantas_ha", "estado",
280                                     "rendimiento_kg_ha"], campanias)
281 cultivo_vigente = {c[1]: c[2] for c in campanias if c[6] == "En curso"}
282
283 # ----- DISPOSITIVO Y SUBCLASES
284 n_disp = p["sensor"] + p["controlador"]
285 disp, sens, ctrl, tipo_de = [], [], [], {}
286 for i in range(1, n_disp + 1):
287     idp, nz = lista_zonas[(i - 1) % len(lista_zonas)]
288     if i <= p["sensor"]:
289         t = random.choices(list(PESOS_SENSOR), weights=list(PESOS_SENSOR.values()))[0]
290         modelo, fab = random.choice(SENSORES[t][1])
291         tipo_de[i] = t
292         sens.append([i, t, random.choice([10, 20, 30, 40, 60]),
293                    random.choice([5, 10, 15, 30, 60]), UNIDAD[t], NULO, NULO])

```

```

294         else:
295             modelo, fab = random.choice(CONTROLADORES)
296             ctrl.append([i, idp, round(random.uniform(4, 50), 2), random.choice(VALVULAS)])
297             disp.append([i, "SN-%07d" % (400000 + i), modelo, fab,
298                         (date(2023, 1, 1) + timedelta(days=random.randint(0, 900))).isoformat(),
299                         "Activo" if random.random() > 0.06 else
300                         random.choice(["Mantenimiento", "Inactivo"]), idp, nz])
301 n["Dispositivo"] = v(carpetas, "dispositivo", ["id_dispositivo", "codigo_serie", "modelo",
302         "fabricante", "fecha_instalacion", "estado", "id_parcela", "num_zona"],
303         disp)
304 n["Sensor"] = v(carpetas, "sensor", ["id_dispositivo", "tipo_sensor", "profundidad_cm",
305         "frecuencia_min", "unidad_medida", "rango_min", "rango_max"], sens)
306 n["Controlador"] = v(carpetas, "controlador", ["id_dispositivo", "id_sistema",
307         "caudal_max_lps", "tipo_valvula"], ctrl)
308 zona_de = {d[0]: (d[6], d[7]) for d in disp}
309
310 # ----- LECTURA_SUELO
311 n_lect = total - sum(n.values()) - p["evento"] - p["alerta"] - p["cosecha"] \
312     - p["aplicacion"] - p["mantenimiento"]
313 assert n_lect > 0, "perfil mal dimensionado"
314 con_cultivo = sorted(cultivo_vigente) or list(range(1, p["parcela"] + 1))
315 secas = set(random.sample(con_cultivo, max(1, int(len(con_cultivo) * 0.22))))
316 lecturas, vistos, cand_hum, cand_bat = [], set(), [], []
317 for k in range(n_lect):
318     sid = sens[k % len(sens)][0]
319     mide = SENSORES[tipo_de[sid]][0]
320     idp, nz = zona_de[sid]
321     idc = cultivo_vigente.get(idp)
322     hmin, hmax = (info[idc][3], info[idc][4]) if idc else (50, 75)
323     while True:
324         t = instante()
325         if (sid, t) not in vistos:
326             vistos.add((sid, t))
327             break
328     hum = None
329     if "hum" in mide:
330         base = hmin - 8 if idp in secas else (hmin + hmax) / 2
331         hum = round(max(0.5, min(99.5, random.gauss(base, 6 if idp in secas else 7))), 2)
332         tmp = round(random.gauss(temp_dep[dep_fundo[fundo_parcela[idp]]], 3.5), 2)
333         phv = round(min(9.5, max(3.5, random.gauss(6.6, 0.7))), 2) if "ph" in mide else None
334         cev = round(abs(random.gauss(1.8, 0.8)), 2) if "ce" in mide else None
335         # descarga en di nte de sierra: la bater a se reemplaza cada ~120 d as
336         bat = int(max(3, min(100, 100 - ((t - INICIO_SERIE).days + sid * 7) % 120 * 0.8
337             + random.gauss(0, 2))))
338         # faltantes por fallas de transmisi n (distintos de los nulos estructurales)
339         fila = [sid, ts(t),
340                 NULO if hum is None or random.random() < 0.02 else hum,
341                 NULO if random.random() < 0.01 else tmp,
342                 NULO if phv is None or random.random() < 0.04 else phv,
343                 NULO if cev is None or random.random() < 0.03 else cev,
344                 NULO if random.random() < 0.05 else bat]
345         lecturas.append(fila)
346         if fila[2] != NULO and hum < hmin and len(cand_hum) < p["alerta"] * 3:
347             cand_hum.append((sid, t, hum, hmin))
348         if fila[6] != NULO and bat < 15 and len(cand_bat) < p["alerta"]:
349             cand_bat.append((sid, t, bat))
350     lecturas.sort(key=lambda r: r[1]) # orden de llegada real: cronol gico
351 n["Lectura_Suelo"] = v(carpetas, "lectura_suelo", ["id_dispositivo", "fecha_hora",
352         "humedad_pct", "temperatura_c", "ph", "conductividad_ds_m",
353         "bateria_pct"], lecturas)
354 del lecturas, vistos
355
356 # ----- ALERTA
357 random.shuffle(cand_hum)
358 random.shuffle(cand_bat)
359 n_bat = min(len(cand_bat), int(p["alerta"] * 0.15))
360 alertas, ventana = [], []
361 for sid, t, val, hmin in cand_hum[:p["alerta"] - n_bat]:
362     deficit = hmin - val
363     sev = "Critica" if deficit > 20 else ("Alta" if deficit > 10 else "Media")
364     at = random.random() < 0.62
365     alertas.append([len(alertas) + 1, sid, ts(t), "Humedad baja", sev, val,
366                     "true" if at else "false",
367                     ts(t + timedelta(minutes=random.randint(20, 400))) if at else NULO,
368                     random.randint(1, p["operario"]) if at else NULO])
369     ventana.append((zona_de[sid][0], zona_de[sid][1], t))
370 for sid, t, val in cand_bat[:p["alerta"] - len(alertas)]:
371     at = random.random() < 0.8
372     alertas.append([len(alertas) + 1, sid, ts(t), "Bater a baja",
373                     "Alta" if val < 8 else "Media", val, "true" if at else "false",
374                     ts(t + timedelta(hours=random.randint(2, 72))) if at else NULO,
375                     random.randint(1, p["operario"]) if at else NULO])
376 alertas.sort(key=lambda r: r[2])
377 for k, r in enumerate(alertas, start=1):
378     r[0] = k
379 n["Alerta"] = v(carpetas, "alerta", ["id_alerta", "id_dispositivo", "fecha_hora", "tipo",
380         "severidad", "valor_registrado", "atendida", "fecha_atencion",
381         "id_operario"], alertas)
382
383 # ----- EVENTO_RIEGO
384 # 55 % de los eventos responden a una alerta de humedad en menos de 6 horas
385 eventos, claves = [], set()
386
387 def agregar_evento(idp, nz, t, modos):

```

```

388         if (idp, nz, t) in claves:
389             return
390         claves.add((idp, nz, t))
391         dur = random.randint(20, 300)
392         sis = idp # sistema de la propia parcela
393         vol = round(caudal[sis] * dur / 60 * random.uniform(0.55, 1.0), 2)
394         eventos.append((len(eventos) + 1, sis, idp, nz, random.randint(1, p["operario"]),
395             ts(t), dur, vol, random.choice(modos)))
396
397     for j in range(int(p["evento"] * 0.55)):
398         if not ventana:
399             break
400         idp, nz, t0 = ventana[j % len(ventana)]
401         agregar_evento(idp, nz, t0 + timedelta(minutes=random.randint(15, 355)),
402             ["Automatico", "Programado"])
403     while len(eventos) < p["evento"]:
404         idp, nz = random.choice(lista_zonas)
405         agregar_evento(idp, nz, instante(), ["Automatico", "Manual", "Programado"])
406     eventos.sort(key=lambda r: r[5])
407     for k, r in enumerate(eventos, start=1):
408         r[0] = k
409     n["Evento_Riego"] = w(carpeta, "evento_riego", ["id_evento", "id_sistema", "id_parcela",
410         "num_zona", "id_operario", "fecha_hora_inicio", "duracion_min",
411         "volumen_litros", "modo"], eventos)
412     del eventos, claves
413
414     # ----- COSECHA
415     # Solo se cosechan campanias cerradas o vigentes con mas de 70 % de avance.
416     tope = date(2026, 9, 15)
417     aptas = []
418     for c in campanias:
419         s, f, ciclo = date.fromisoformat(c[3]), date.fromisoformat(c[4]), info[c[2]][2]
420         inicio = s + timedelta(days=int(ciclo * 0.7))
421         if c[6] == "Cerrada" or (c[6] == "En curso" and inicio < tope - timedelta(days=5)):
422             aptas.append((c, inicio, min(f, tope)))
423     reparto = {}
424     for i in range(p["cosecha"]):
425         reparto[aptas[i % len(aptas)][0][0]] = reparto.get(aptas[i % len(aptas)][0][0], 0) + 1
426     cosechas = []
427     for c, ini, fin in aptas:
428         veces = reparto.get(c[0], 0)
429         if veces == 0:
430             continue
431         esperado = info[c[2]][8] * 1000 * area[c[1]] * random.uniform(0.65, 1.15)
432         if c[6] == "En curso":
433             esperado *= random.uniform(0.3, 0.7)
434         precio = info[c[2]][9]
435         dias = max(1, (fin - ini).days)
436         for _ in range(veces):
437             cosechas.append((len(cosechas) + 1, c[0], random.randint(1, p["operario"]),
438                 (ini + timedelta(days=random.randint(0, dias))).isoformat(),
439                 round(max(1, esperado / veces * random.uniform(0.7, 1.3)), 2),
440                 random.choices(CATEGORIAS, weights=[55, 27, 13, 5])[0],
441                 NULO if random.random() < 0.10 else round(random.uniform(8, 82), 2),
442                 NULO if random.random() < 0.06 else
443                 round(precio * random.uniform(0.75, 1.25), 2)))
444     cosechas.sort(key=lambda r: r[3])
445     for k, r in enumerate(cosechas, start=1):
446         r[0] = k
447     n["Cosecha"] = w(carpeta, "cosecha", ["id_cosecha", "id_campania", "id_operario", "fecha",
448         "cantidad_kg", "categoria", "humedad_producto_pct", "precio_kg_soles",
449         cosechas)
450
451     # ----- APLICACION_INSUMO
452     apl, claves = [], set()
453     while len(apl) < p["aplicacion"]:
454         idp, nz = random.choice(lista_zonas)
455         ins, ope, t = random.randint(1, p["insumo"]), random.randint(1, p["operario"]), instante()
456         t = t.replace(minute=random.choice([0, 15, 30, 45]))
457         if (idp, nz, ins, ope, t) in claves:
458             continue
459         claves.add((idp, nz, ins, ope, t))
460         lo, hi = DOSIS[tipo_insumo[ins]]
461         apl.append((idp, nz, ins, ope, ts(t), round(random.uniform(lo, hi), 2),
462             random.choice(METODOS[tipo_insumo[ins]])))
463     apl.sort(key=lambda r: r[4])
464     n["Aplicacion_Insumo"] = w(carpeta, "aplicacion_insumo", ["id_parcela", "num_zona",
465         "id_insumo", "id_operario", "fecha_hora", "dosis", "metodo"], apl)
466
467     # ----- MANTENIMIENTO
468     mant, claves = [], set()
469     while len(mant) < p["mantenimiento"]:
470         d, o = random.randint(1, n_disp), random.randint(1, p["operario"])
471         f = date(2025, 1, 1) + timedelta(days=random.randint(0, 620))
472         if (d, o, f) in claves:
473             continue
474         claves.add((d, o, f))
475         tipo = random.choices(["Preventivo", "Correctivo", "Calibracion"], weights=[50, 30, 20])[0]
476         obs = {"Preventivo": "Limpieza de contactos y revision de cableado",
477             "Correctivo": "Reemplazo de bateria y sellado de carcasa",
478             "Calibracion": "Calibracion con muestra gravimetrica de suelo"}[tipo]
479         mant.append((d, o, f.isoformat(), tipo, NULO if random.random() < 0.25 else obs,
480             round(random.uniform(25, 780), 2)))
481     n["Mantenimiento"] = w(carpeta, "mantenimiento", ["id_dispositivo", "id_operario", "fecha",

```

```
482         "tipo", "observacion", "costo"], mant)
483     return n
484
485
486 if __name__ == "__main__":
487     tot, dest = int(sys.argv[1]), sys.argv[2]
488     conteo = generar(tot, dest)
489     for k, v in conteo.items():
490         print("%-18s %10d" % (k, v))
491     print("%-18s %10d" % ("TOTAL", sum(conteo.values())))
```